

Universidad de Jaén

Escuela Politécnica Superior de Jaén

Asistente virtual de Trámites Administrativos del Ayuntamiento de Jaén

Estudiante: Mario Martínez Gallego

Grado: Ingeniería Informática

Tutoría del TFG: Eugenio Martínez Cámara y María Teresa Martín Valdivia
Departamento: Informática

Fecha: 22/06/2026

Licencia CC

CREA

Índice

Capítulo 1. Introducción....2
1.1. Introducción y Motivación.... 2
1.2. Objetivo....3
1.3. Hipótesis de trabajo.... 3
1.4. Estructura de la Memoria....4
Capítulo 2: Análisis del Sistema....7
2.1. Requisitos del Sistema.... 7
2.1.1. Requisitos Funcionales (RF)....7
Ilustración 2.1: Diagrama de Requisitos Funcionales....7
2.1.2. Requisitos No Funcionales (RNF)....8
Ilustración 2.2: Diagrama de Requisitos No Funcionales.... 8
2.2. Casos de Uso.... 9
Ilustración 2.3: Diagrama de Casos de Uso....9
Ilustración 2.4: Diagrama del CU-1.... 11
Ilustración 2.5: Diagrama del CU-2....12
2.3. Diagramas de Actividad....12
Ilustración 2.6: Diagrama de Actividad, CU-1....13
Ilustración 2.7: Diagrama de Actividad, CU-2....14
2.4. Diagramas de Secuencia....14
Ilustración 2.8: Diagrama de Secuencia, CU-1....15
Ilustración 2.9: Diagrama de Secuencia, CU-2....16
Capítulo 3. Planificación Temporal y Presupuesto....19
3.1. Planificación Temporal....19
3.1.1. Relación de Actividades y Dependencias para la Planificación Temporal....19
Ilustración 3.1: Diagrama de Gantt.... 21
3.2. Presupuesto Económico del Proyecto....21
3.2.1. Costes Directos de Personal....21
3.2.2. Costes Indirectos.... 23
3.2.3. Consolidación del Presupuesto Total del Sistema....23
Capítulo 4. Estado del Arte y Trabajos Relacionados....25
4.1. Fundamentos de los Modelos de Lenguaje (LLM)....25
Ilustración 4.1: Esquema de aprendizaje profundo....26
4.1.1. Contexto Histórico: De las Reglas Semánticas al Transformer.... 26
4.1.2. Fundamentos Técnicos: Proceso de transformación del texto....27
Ilustración 4.2: Tokenización....27
Ilustración 4.3: Vectorización.... 28
4.1.3. Clasificación y Estrategias del Ecosistema de IA.... 28
4.2. Arquitecturas de Generación Aumentada por Recuperación (RAG).... 29
4.2.1. El pipeline de un RAG: Ingesta, Recuperación y Generación.... 29
Ilustración 4.4: Esquema RAG....30
Ilustración 4.5: Ejemplo prompt....31
4.3. Tecnologías de Recuperación Vectorial e Indexación.... 32
4.3.1. Búsqueda semántica frente a búsqueda léxica.... 32

4.3.2. Indexación y búsqueda de similitud con FAISS..... 32
Ilustración 4.6: Ejemplo del espacio vectorial de una consulta..... 33
4.4. Marcos de Evaluación Semántica y el Paradigma LLM-as-a-Judge..... 34
4.4.1. Por qué las métricas tradicionales (BLEU y ROUGE) no sirven aquí..... 34
4.4.2. El framework Ragas y las métricas de evaluación..... 34
4.4.3. El paradigma de usar un LLM como Juez..... 35
4.5. Trabajos Relacionados..... 35
4.5.1. Inteligencia artificial en el sector legal y administrativo..... 35
4.5.2. Iniciativas nacionales de soberanía lingüística (ILENIA)..... 36
4.5.3 La Iniciativa ALIA..... 36
Ilustración 4.7: Alia..... 36
4.5.4. Aportación diferencial de este trabajo..... 37
Capítulo 5. Diseño del sistema..... 39
5.1. Arquitectura Lógica del Sistema..... 39
5.1.1. Módulos Funcionales del Sistema..... 39
Ilustración 5.1: Diagrama de componentes..... 40
5.2. Diseño de Componentes y Arquitectura de Clases..... 41
5.2.1. Patrones de Diseño Aplicados..... 41
5.2.2. Clases del Sistema..... 42
5.2.3. Modelado de Datos y Entidades..... 43
5.2.4. Diagrama de Clases UML del Sistema..... 44
Ilustración 5.2: Diagrama de Clases..... 44
5.2.5 Justificación relaciones UML..... 44
5.3. Diseño de la Interfaz de Usuario (UI/UX)..... 45
5.3.1. Boceto..... 45
Ilustración 5.3: Boceto de la Interfaz..... 46
5.3.2. Principios de Usabilidad y Experiencia de Usuario..... 46
5.3.3. Arquitectura de la Información y Distribución (Layout)..... 47
5.3.4. Componentes Visuales y Guía de Estilos..... 47
5.3.5. Control de Estados..... 48
5.3.6. Prototipo..... 48
Ilustración 5.4: Prototipo de la Interfaz..... 48
Capítulo 6. Implementación del sistema..... 49
6.1. Entorno Tecnológico y Recursos del Sistema..... 49
6.1.1. Especificación de los Componentes Utilizados..... 49
Ilustración 6.1: Logo Python..... 49
Ilustración 6.2: Logo LangChain..... 49
Ilustración 6.3: Logo OpenAI..... 50
Ilustración 6.4: Logo HuggingFace..... 50
6.2. Recursos de Infraestructura y Hardware..... 51
6.3. Flujo del desarrollo de scripts..... 51
6.3.1. Análisis Web y Descubrimiento de Campos..... 51
6.3.2. Ingesta de Trámites Administrativos..... 52
6.3.3. Procesamiento, Chunking e Indexación Vectorial..... 53

6.3.4. Interfaz y Gestión Asíncrona....54
6.2.5. Script de Automatización y Despliegue Operacional....55
Capítulo 7. Evaluación y resultados....57
7.1. Justificación y Motivación de la Evaluación....57
7.2. Metodología de Evaluación....57
7.3. Requisitos e Infraestructura para la Ejecución de la Evaluación....58
7.3.1. Construcción del Conjunto de Evaluación....58
7.3.2. Paradigma LLM-as-a-Judge e Infraestructura de Cómputo Distribuido....61
7.4. Evaluación del Módulo Recuperador (Retriever)....63
7.4.1. Evaluación Determinista de Modelos de Embeddings y Estrategias de Búsqueda....63
Tabla 7.1: Evaluación Determinista....63
7.4.2. Evaluación mediante Métricas Ragas....64
Tabla 7.2: Métricas de Recuperación Ragas....65
7.4.3. Evaluación y Clasificación de Errores en la Recuperación....65
Tabla 7.3: Resumen Errores Recuperación....67
7.5. Evaluación del Módulo Generador (Generator)....68
7.5.1. Modelos Evaluados y Protocolo de Inferencia....68
7.5.2. Evaluación mediante Métricas Ragas....68
Tabla 7.4: Métricas de Generación Ragas....69
7.5.3. Evaluación Manual de Alucinaciones en Contexto Nulo....70
Tabla 7.5: Resultados Generación sin Contexto....70
Tabla 7.6: Resumen tipos de Errores Generación sin Contexto....72
7.6. Evaluación del Software y Validación Funcional....73
7.6.1. Verificación de los Requisitos Funcionales en la Interfaz Web....73
Ilustración 7.1: Interfaz general del sistema gestionando el historial....73
Ilustración 7.2: Flujo de respuesta en doble columna....74
Ilustración 7.3: Detalle de hipervínculos oficiales a la sede electrónica....75
Ilustración 7.4: Comparativa de tiempos de respuesta....76
7.7. Balance General de la Evaluación....76
Capítulo 8. Conclusión y trabajos futuros....77
8.1 Conclusiones Generales....77
8.2 Trabajos Futuros....78
8.2.1 Refinamiento de Prompting....78
8.2.2 Evaluación de Embeddings....78
8.2.3. Implementación de un Reranker Administrativo....79
Bibliografía....81

Índice de Ilustraciones

Ilustración 2.1: Diagrama de Requisitos Funcionales....7
Ilustración 2.2: Diagrama de Requisitos No Funcionales....8
Ilustración 2.3: Diagrama de Casos de Uso....9
Ilustración 2.4: Diagrama del CU-1....11

Ilustración 2.5: Diagrama del CU-2....12
Ilustración 2.6: Diagrama de Actividad, CU-1....13
Ilustración 2.7: Diagrama de Actividad, CU-2....14
Ilustración 2.8: Diagrama de Secuencia, CU-1....15
Ilustración 2.9: Diagrama de Secuencia, CU-2....16
Ilustración 3.1: Diagrama de Gantt....21
Ilustración 4.1: Esquema de aprendizaje profundo....26
Ilustración 4.2: Tokenización....27
Ilustración 4.3: Vectorización....28
Ilustración 4.4: Esquema RAG....30
Ilustración 4.5: Ejemplo prompt....31
Ilustración 4.6: Ejemplo del espacio vectorial de una consulta....33
Ilustración 4.7: Alia....36
Ilustración 5.1: Diagrama de componentes....40
Ilustración 5.2: Diagrama de Clases....44
Ilustración 5.3: Boceto de la Interfaz....46
Ilustración 5.4: Prototipo de la Interfaz....48
Ilustración 6.1: Logo Python....49
Ilustración 6.2: Logo LangChain....49
Ilustración 6.3: Logo OpenAI....50
Ilustración 6.4: Logo HuggingFace....50
Ilustración 7.1: Interfaz general del sistema gestionando el historial....73
Ilustración 7.2: Flujo de respuesta en doble columna....74
Ilustración 7.3: Detalle de hipervínculos oficiales a la sede electrónica....75
Ilustración 7.4: Comparativa de tiempos de respuesta....76

Índice de Tablas

Tabla 7.1: Evaluación Determinista....63
Tabla 7.2: Métricas de Recuperación Ragas....65
Tabla 7.3: Resumen Errores Recuperación....67
Tabla 7.4: Métricas de Generación Ragas....69
Tabla 7.5: Resultados Generación sin Contexto....70
Tabla 7.6: Resumen tipos de Errores Generación sin Contexto....72

Capítulo 1. Introducción

En este capítulo se detalla la introducción y la motivación de este Trabajo Fin de Grado, el objetivo del mismo, la hipótesis de la cual se basa el proyecto y la estructura en capítulos del mismo.

1.1. Introducción y Motivación

A día de hoy navegar entre los trámites de una administración pública puede resultar, para muchos ciudadanos, un camino difícil, con varios obstáculos. Aunque se ha avanzado mucho digitalmente, la realidad es que el lenguaje técnico sobre trámites administrativos y cómo está organizada la información siguen siendo obstáculos difíciles de superar. Normalmente, el ciudadano promedio no suele entender los tecnicismos de un documento oficial y suele no encontrar los que necesita, qué plazos debe cumplir o qué requisitos necesita para realizar algún trámite.

Esta situación es especialmente crítica en el ámbito local, como ocurre en el **Ayuntamiento de Jaén**. Al ser la institución más cercana al ciudadano, la administración municipal debe gestionar un catálogo de trámites muy amplio y variado, que abarca desde la solicitud de una licencia de obras hasta la petición de una ayuda social o un registro sanitario de desinfección y desratización (DDD). La gran cantidad de trámites termina **saturando los canales de atención** presencial y telefónica del ayuntamiento. Esto genera una frustración en el ciudadano por no ser bien atendido y una carga de trabajo repetitiva para los empleados de la administración pública, que tienen que responder las mismas dudas continuamente.

La solución a este problema se encuentra en el uso de la inteligencia artificial, concretamente en los **Modelos de Lenguaje de Gran Escala (LLM)**. Estos modelos son herramientas con una gran capacidad para comprender las preguntas de los usuarios y generar texto de forma automática. Estos modelos pueden actuar de puente entre la administración y el ciudadano, permitiendo a cualquier persona acceder a la información de los trámites de manera sencilla, realizando preguntas con sus propias palabras y recibiendo una respuesta clara. Sin embargo, no podemos usar modelos comerciales, como *ChatGPT*, directamente en un ayuntamiento ya que nos encontramos con dos problemas principales:

1. **La privacidad y soberanía de los datos:** No es seguro ni ético enviar las consultas privadas de los ciudadanos de Jaén a servidores de empresas privadas externas. Necesitamos soberanía y control total sobre nuestra propia información.
2. **Las alucinaciones:** Sin embargo, estos modelos tienen un problema conocido como 'alucinación'. Cuando el sistema no conoce la respuesta a una pregunta, tiende a inventar la información con una apariencia de total seguridad y corrección. En un entorno público, esto es peligroso porque el asistente podría inventarse plazos de entrega o requisitos legales falsos.. En una administración pública un fallo de este calibre puede causar un problema grave al ciudadano.

Por todo ello, este TFG nace de la necesidad de crear un sistema de asistencia al ciudadano que sea preciso, seguro, independiente y auditable. La idea es aplicar una arquitectura **RAG** (*Retrieval-Augmented Generation*, o Generación Aumentada por Recuperación) para evitar que el modelo genere respuestas basadas únicamente en su conocimiento interno; en su

lugar, el sistema consulta los trámites oficiales del Ayuntamiento de Jaén antes de emitir cualquier contestación. Esto se podrá llevar a cabo utilizando infraestructuras de supercomputación europeas, así como de la propia Universidad de Jaén, las cuales ofrecen capacidades de cómputo y modelos. Este proyecto busca demostrar que con la tecnología se puede hacer que la administración sea mucho más accesible y cercana para todos.

1.2. Objetivo

El objetivo principal de este Trabajo Fin de Grado es **facilitar el acceso de la información pública de la sede electrónica del Ayuntamiento de Jaén**, exactamente los trámites administrativos del catálogo de trámites de la sede, mediante el diseño, desarrollo e implementación de un sistema inteligente de asistencia virtual interactivo (*Catálogo De Procedimientos*, n.d.).

El proyecto busca mitigar la barrera lingüística y burocrática entre la administración local y el ciudadano, garantizando la provisión de respuestas **veraces** y **fundamentadas normativamente.**

1.3. Hipótesis de trabajo

Nuestra hipótesis de partida es la siguiente:

> "La implementación de una arquitectura de Generación Aumentada por Recuperación (RAG), fundamentada en un motor de indexación denso especializado en el dominio legal y administrativo español, optimiza el acceso a los trámites administrativos de la sede electrónica y mitiga la tasa de alucinaciones de los Modelos de Lenguaje de Gran Escala (LLMs) al resolver consultas sobre la sede electrónica del Ayuntamiento de Jaén, independientemente del volumen de parámetros del modelo generador de propósito general."

A continuación profundizamos en la hipótesis, la cuál está sustentada por los siguientes puntos:

- **Contextualización y terminología específica**: Se plantea que las limitaciones de razonamiento de los modelos generalistas ante la falta de entendimiento del léxico administrativo local (palabras como «ICIO», «vado», «silencio administrativo» o «bonificación por familia numerosa») no se resuelven aumentando el tamaño del LLM, sino inyectando el contexto exacto mediante recuperación densa. La hipótesis sostiene que el uso de representaciones vectoriales relacionadas con el marco jurídico español permite una transferencia de conocimiento eficiente hacia la ventana de contexto del generador.

- **Reducción del no-determinismo y control de alucinaciones**: La restricción del espacio de búsqueda a fragmentos oficiales del Ayuntamiento de Jaén actúa como un marco de control para el modelo generativo, ya que al forzar al LLM a realizar una respuesta guiada por el contexto recuperado en lugar de depender de sus pesos paramétricos preentrenados, se reduce drásticamente la generación de datos erróneos, tales como plazos de tramitación o requisitos inventados.

- **Viabilidad de la soberanía tecnológica en la administración local**: Se plantea que la tecnología RAG permite a las administraciones locales gestionar su propia inteligencia artificial de forma independiente, demostrando que para dar información

verídica al ciudadano basta con organizar bien los datos institucionales, sin necesidad de depender de APIs comerciales cerradas.

1.4. Estructura de la Memoria

Para facilitar la lectura y comprensión de este Trabajo Fin de Grado, a continuación, se detalla la estructura del documento. El TFG se compone de **8 capítulos** que abarcan desde la introducción y fundamentación teórica hasta la implementación práctica y la evaluación experimental del sistema, terminando con una conclusión y trabajos futuros:

- **Capítulo 1.** Introducción.
  Este capítulo establece el contexto y la motivación del proyecto. En él se expone la estructura general del documento, se define el objetivo global y se formula la hipótesis en la que se basa este Trabajo Fin de Grado.

- **Capítulo 2.** Análisis del Sistema.
  Este capítulo aborda la definición del proyecto. En él se definen las acciones y servicios que el sistema debe hacer, **requisitos funcionales**, las cualidades de calidad, rendimiento, restricciones y seguridad del software, **requisitos no funcionales**, y los diferentes **casos de uso** que detallan las interacciones entre el usuario y el sistema.

- **Capítulo 3.** Planificación Temporal y presupuestos.
  Este capítulo aborda la viabilidad organizativa y económica del desarrollo del software. En él se define el cronograma de las tareas técnicas a lo largo de un año completo mediante un **diagrama de Gantt**, detallando las dependencias del ciclo de vida. Además, se calcula el **presupuesto financiero** del proyecto, desglosando **costes de personal** y los **costes indirectos** de hardware e infraestructura, añadiendo un margen para **contingencias**.

- **Capítulo 4.** Estado del arte.
  En este capítulo se explica la base teórica, científica y el contexto tecnológico que fundamenta la investigación. Se examina la evolución del **Procesamiento del Lenguaje Natural** (PLN), se detalla los fundamentos operativos y limitaciones inherentes de los **Modelos de Lenguaje de Gran Escala (LLM)**. Posteriormente, se introduce el paradigma **RAG** (*Retrieval-Augmented Generation*), describiendo el funcionamiento de sus tecnologías subyacentes. Asimismo, se justifica el uso del framework **Ragas** y el paradigma **LLM-as-a-Judge** frente a métricas léxicas tradicionales como **BLEU** o **ROUGE**. Finalmente se realiza una revisión de los **trabajos relacionados** en el sector público.

- **Capítulo 5.** Diseño del sistema.
  Este capítulo se centra en la arquitectura lógica, el modelado de datos y el diseño visual del asistente de forma abstracta. En él se detalla los **módulos funcionales**, el **flujo de datos** de los diferentes casos de uso, **diagrama de actividad** y **diagrama de secuencia**, la arquitectura software mediante el **diseño de clases** justificando patrones de diseño y, finalmente, el **diseño de la interfaz de usuario** orientada a facilitar la interacción y comparación de los modelos, detallando el proceso desde el **boceto inicial** en lápiz hasta el **prototipo final**.

- **Capítulo 6.** Implementación del sistema.
  Detalla el **entorno tecnológico** de desarrollo (lenguajes, frameworks y librerías) y los **recursos de hardware** empleados en el despliegue de la solución. Asimismo, expone la lógica y el **flujo secuencial de los scripts** del sistema, abarcando desde la fase de **extracción** de los trámites administrativos del Ayuntamiento de Jaén (scraping) y su indexación vectorial, hasta la **implementación del backend** del asistente virtual.

- **Capítulo 7.** Evaluación y resultados.
  Este capítulo contiene la parte experimental del trabajo. Está compuesto de dos secciones, la **evaluación científica** y la **evaluación del software**. En la evaluación **científica** se describe la metodología de auditoría, basada en el framework **Ragas** y el paradigma **"LLM-as-a-Judge",** aplicada sobre un dataset propio generado. Se analizan, de forma pormenorizada, las fases de **recuperación** (precisión y recuperación del contexto) y **generación** (fidelidad y relevancia). Además, se detallan diferentes técnicas y soluciones desarrolladas a lo largo de la evaluación. Esta sección concluye con la comparativa de resultados entre los modelos **ALIA (Salamandra** en el servidor de la UJA**), Llama y Apertus**. En la evaluación del **software** se comprueba si se han cumplido todos los objetivos definidos al comienzo del proyecto, es decir, el **cumplimiento** de los **requisitos funcionales** del sistema.

- **Capítulo 8.** Conclusiones y trabajos futuros.
  Capítulo final que **clausura** la memoria sintetizando las **conclusiones** generales de la investigación y **validando la hipótesis** planteada. Además, se proponen **líneas de desarrollo futuras**, orientadas al refinamiento de instrucciones para mitigar alucinaciones y la optimización de recuperación vectorial.

Capítulo 2: Análisis del Sistema

En este capítulo se definen las necesidades que el sistema debe de cubrir (**requisitos funcionales y no funcionales**) y los diferentes escenarios de uso previstos (**casos de uso**).

2.1. Requisitos del Sistema

Para garantizar que el asistente virtual sea una herramienta útil, de calidad, veraz y fiable para el ciudadano de Jaén, el desarrollo del mismo se ha guiado por una serie de requisitos técnicos rigurosos.

2.1.1. Requisitos Funcionales (RF)

Siguiendo los principios básicos de la Ingeniería del Software (Sommerville, 2011), los **requisitos funcionales** definen los servicios o tareas específicas que el sistema debe ser capaz de realizar ante determinadas entradas, es decir, describen **qué** debe de hacer el sistema (ej: el sistema debe buscar la normativa):

- **RF-1. Gestión del historial de conversación**: El sistema debe permitir al ciudadano almacenar, eliminar, visualizar y navegar por el historial de sus consultas previas en la interfaz de chat.
- **RF-2 Consultar trámites administrativos:** El sistema debe responder en lenguaje natural consultas en lenguaje natural sobre trámites administrativos del Ayuntamiento de Jaén.
- **RF-3. Identificación y citación del trámite de origen**: El sistema debe mostrar, junto a la respuesta de cada modelo, el nombre exacto o referencia del trámite administrativo de donde se extrajo la información.

A fin de proporcionar una visión en conjunto sobre las responsabilidades del sistema, en la Ilustración 2.1 se muestran los requisitos funcionales descritos.

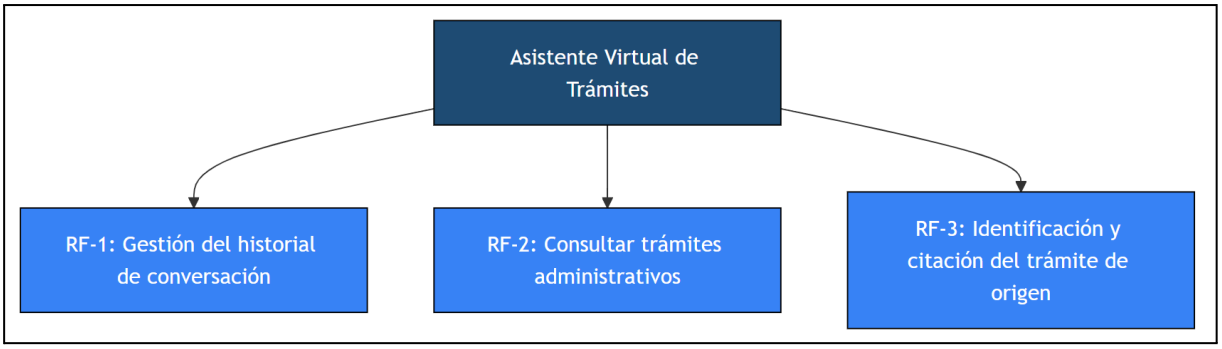


Ilustración 2.1: Diagrama de Requisitos Funcionales

2.1.2. Requisitos No Funcionales (RNF)

Los **requisitos no funcionales** definen criterios de calidad y restricciones del sistema que se está desarrollando, describen **cómo** debe funcionar el sistema (ej: la respuesta debe ser de menos de 2 segundos):

- **RNF-1. Precisión y Fidelidad:** El sistema debe garantizar que las respuestas sean exactas, que no haya "alucinaciones", para asegurar que la información legal proporcionada siempre sea correcta y fiable, **mitigando** la propagación de **información falsa** o inventada a los modelos (Ji et al., 2023)
- **RNF-2. Interoperabilidad:** El sistema debe de ser capaz de comunicarse con diferentes infraestructuras de cómputo (como los servidores de la UJA y la nube Helmholtz de Alemania), empleando el estándar de la **API de OpenaAI** para asegurar la compatibilidad entre sistemas.
- **RNF-3. Resiliencia y robustez:** El sistema debe ser capaz de gestionar problemas técnicos, problemas de red, cortes de conexión, consiguiendo que no se bloquee la interfaz del usuario.
- **RNF-4. Privacidad:** El sistema debe asegurar la privacidad y protección de la información del Ayuntamiento de Jaén y de las consultas de los ciudadanos mediante el uso de modelos de código abierto y entornos de procesamiento controlados, evitando depender de infraestructuras externas y garantizando el control sobre la exposición de datos sensibles.
- **RNF-5. Monitorización del tiempo de respuesta**: El sistema debe calcular y mostrar visiblemente en la interfaz los segundos exactos que ha tardado cada uno de los dos modelos en generar su respectiva respuesta, debido a que la latencia impacta en la percepción de la calidad del sistema interactivo (Nielsen, 1994).

A fin de proporcionar una visión en conjunto sobre las cualidades del sistema, en la Ilustración 2.2 se muestran los requisitos no funcionales descritos.

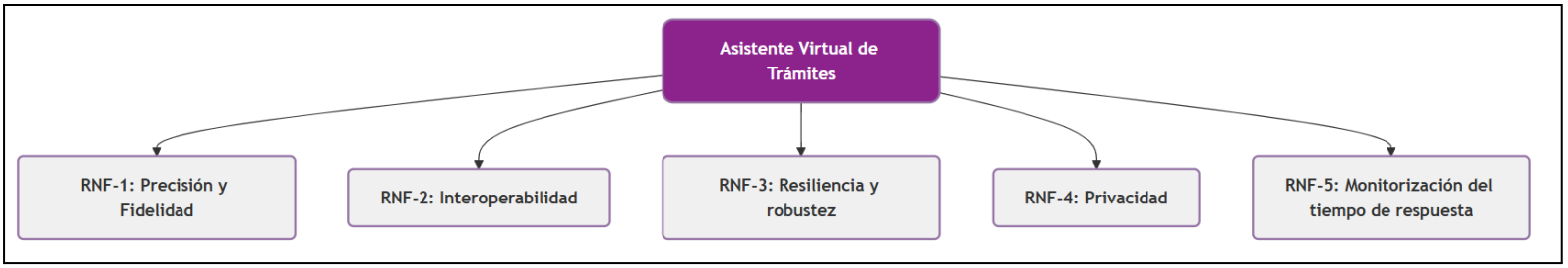


Ilustración 2.2: Diagrama de Requisitos No Funcionales

2.2. Casos de Uso

Un **caso de uso** explica cómo un usuario interactúa con el sistema. Define **qué** debe hacer el sistema sin explicar **cómo** lo va a hacer, basándose en el estándar de modelado de lenguaje unificado (UML) (Fowler, 2004).
El actor del sistema es el **ciudadano**, el cual busca realizar consultas sobre los trámites administrativos de la sede electrónica del Ayuntamiento de Jaén, además de llevar un control de sus conversaciones.

Para obtener una perspectiva global de las interacciones del usuario con el sistema antes de profundizar en su lógica interna, se presenta en la Ilustración 2.3 el diagrama general de casos de uso.

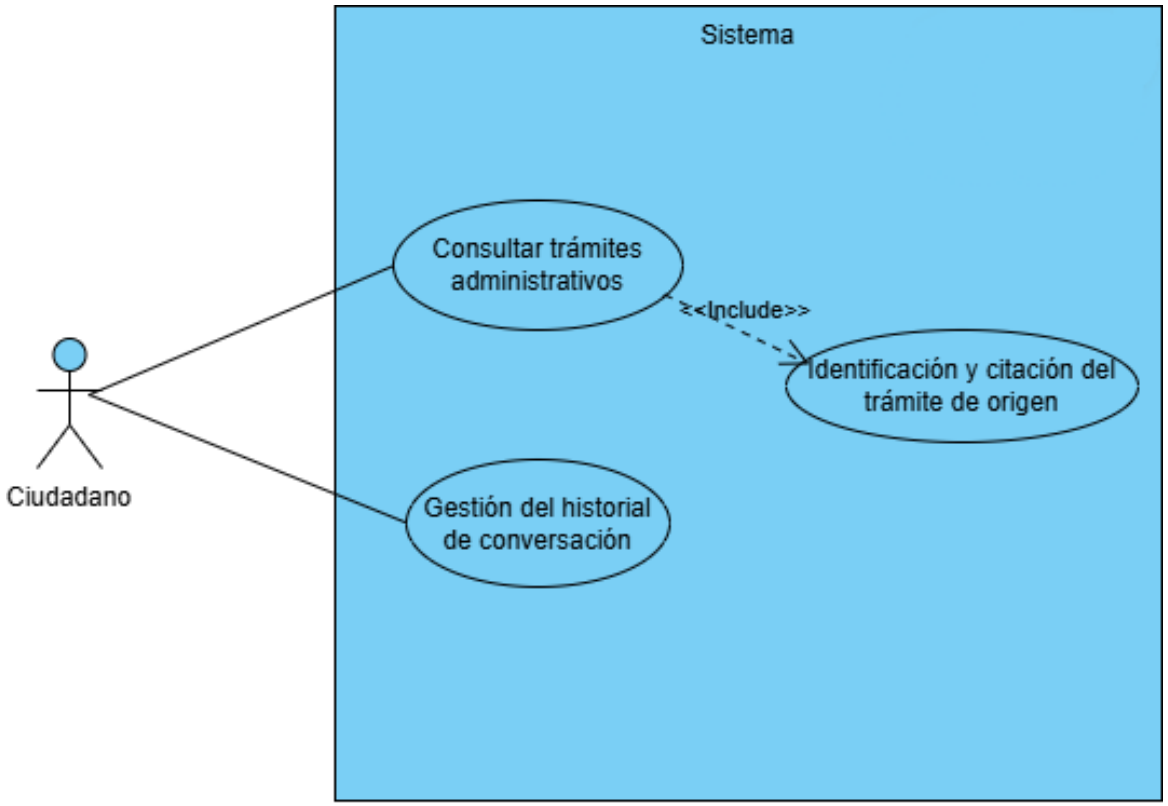


Ilustración 2.3: Diagrama de Casos de Uso

Como se observa en la ilustración superior, la arquitectura funcional se delimita mediante una frontera que encapsula los servicios del asistente virtual. El actor (Ciudadano) actúa como iniciador primario y se comunica directamente con las dos funcionalidades troncales que definen el alcance del proyecto: La capacidad de interrogar al asistente virtual sobre los trámites administrativos del Ayuntamiento de Jaén (**CU-1: Consultar trámites administrativos**), que encapsula la obligación funcional de identificar y citar la fuente del trámite de origen, y la capacidad de administrar los registros de conversaciones pasadas (**CU-2: Gestión del historial de conversaciones**)

A continuación, se detalla el flujo de eventos, precondiciones, excepciones y requisitos asociados a cada uno de estos casos de uso mediante sus respectivas tablas de especificación:

- **CU-1: Consultar trámites administrativos**

Caso de Uso	**CU-1: Consultar trámites administrativos**
Actores	Ciudadano
Descripción	El ciudadano realiza una pregunta en lenguaje natural y el sistema genera y muestra simultáneamente dos respuestas independientes en lenguaje natural, adjuntando a cada una su respectiva fuente de información y el tiempo de procesamiento.
Precondición	El ciudadano está en la interfaz de chat de la aplicación con una consulta.
Flujo principal	1. El Ciudadano introduce una consulta sobre un trámite municipal. 2. El sistema procesa la consulta y recupera la información relevante de la base de datos de trámites administrativos. 3. El sistema genera una respuesta en lenguaje natural a partir de la información recuperada. 4. El sistema muestra la respuesta en la interfaz del usuario. 5. El sistema muestra junto a la respuesta el trámite del que se obtiene la información y el tiempo de respuesta.
Flujo Alternativo	3.a. Fallo en el servidor o caída de red El sistema detecta un fallo de conexión, interrumpe el proceso, muestra un mensaje de error en lugar de la respuesta del modelo y **vuelve al paso 1.**
Postcondición	El ciudadano visualiza la información comparada en la pantalla y la consulta se registra en el historial de la conversación.

Para complementar la especificación textual del CU-1 y profundizar en las dependencias lógicas de la consulta, se presenta en la Ilustración 2.4 el diagrama de las subfuncionalidades de este proceso.

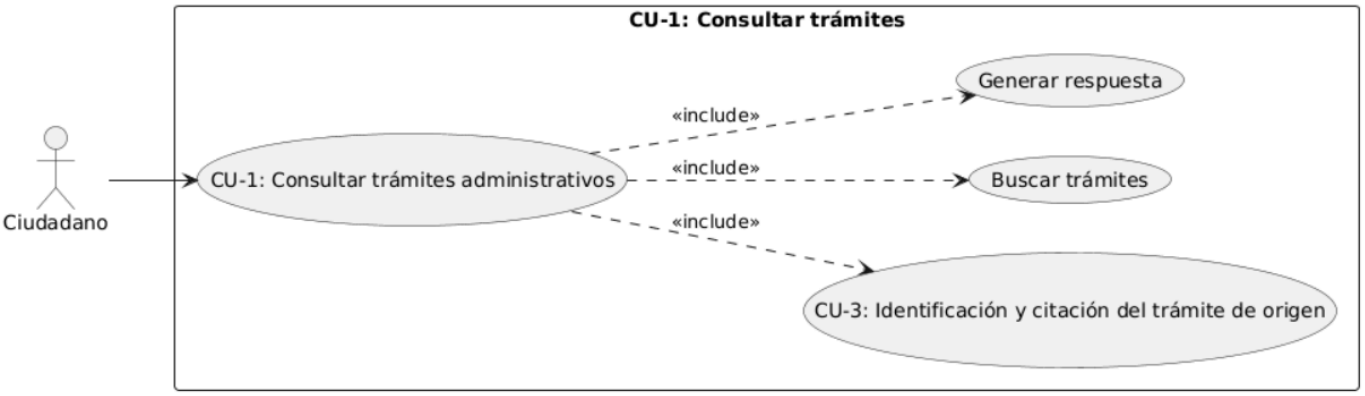


Ilustración 2.4: Diagrama del CU-1

Como se aprecia en la Ilustración 2.4, la ejecución del caso de uso principal requiere obligatoriamente delegar su carga lógica en tres subfuncionalidades atómicas e independientes mediante **relaciones de inclusión** (<<include>>). El sistema siempre debe **"Buscar trámites"** para localizar la información en el almacenamiento y **"Generar respuestas"** para computar la salida textual. Asimismo, se incluye formalmente el "CU-3: Identificación y citación del trámite de origen" como una subfuncionalidad imperativa. Este diseño asegura que el cumplimiento del requisito de transparencia y citación sea un paso indivisible y obligatorio dentro del flujo síncrono del asistente virtual.

- **CU-2: Gestión del historial de conversación**

Caso de Uso	**CU-2: Gestión del historial de conversación**
Actores	Ciudadano
Descripción	El ciudadano gestiona (navegar, visualizar, eliminar) las conversaciones que ha mantenido previamente con el sistema.
Requisitos asociados	RF-1
Precondición	El historial no debe de estar vacío
Flujo principal	1. El Ciudadano selecciona el historial de conversaciones del panel lateral. 2. El sistema recupera el historial de conversaciones almacenado. 3. El sistema muestra cronológicamente los títulos de conversaciones pasadas. 4. El Ciudadano selecciona una consulta 5. El sistema despliega en la pantalla principal todo su contenido (cuestiones, respuestas, fuentes, tiempos).
Flujo alternativo	4.a. El Ciudadano elimina la conversación. 1. El ciudadano confirma la eliminación 2. El sistema elimina de forma permanente la conversación. 3. El sistema actualiza el panel lateral eliminando el título

	de la conversación borrada. 4. El sistema redirige al usuario a la pantalla principal, **volviendo al paso 1**.
Postcondición	El sistema renderiza la conversación seleccionada por el usuario.

Del mismo modo, el comportamiento del usuario sobre el registro de datos persistentes se modela visualmente a través del diagrama de las subfuncionalidades de este caso de uso en la Ilustración 2.5.

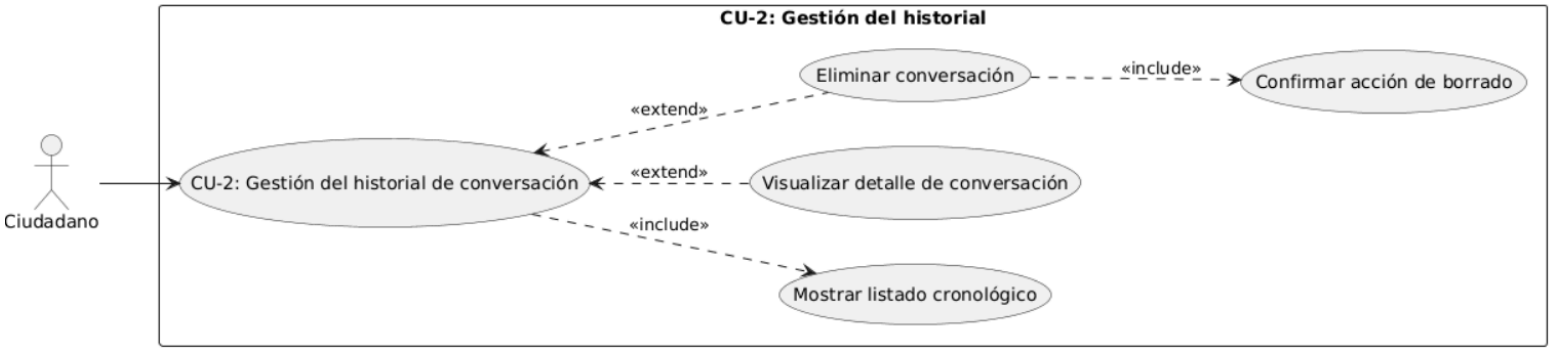


Ilustración 2.5: Diagrama del CU-2

El desglose de la Ilustración 2.5 expone la estructura interna de la administración del historial de conversaciones. El acceso al historial incluye siempre y de manera forzosa la subfuncionalidad de **"Mostrar listado cronológico"** (<<include>>) para inicializar la interfaz gráfica. A partir de este punto, el caso de uso se extiende condicionalmente (<<extend>>) hacia dos flujos independientes que quedan a elección exclusiva del ciudadano: **"Visualizar detalle de conversación"** si desea leer un registro anterior, o **"Eliminar conversación"** si desea purgar los datos. Finalmente, se ha diseñado que esta última acción de eliminación incluya obligatoriamente (<<include>>) la subfuncionalidad de **"Confirmar acción de borrado"**, garantizando un mecanismo de control redundante antes de alterar el almacenamiento físico.

2.3. Diagramas de Actividad

Para modelar conceptualmente el flujo de trabajo, las decisiones del sistema, sin especificar los objetos de código, se definen los diagramas de actividad estructurados por carriles. Delimitan las responsabilidades del **ciudadano** y del **sistema**.

CU-1 Consultar trámites administrativos

El diagrama de la Ilustración 2.6 representa el flujo de control desde que el ciudadano realiza su consulta hasta que se muestran las respuestas de los modelos, u ocurre un fallo con los modelos (escenario de excepción).

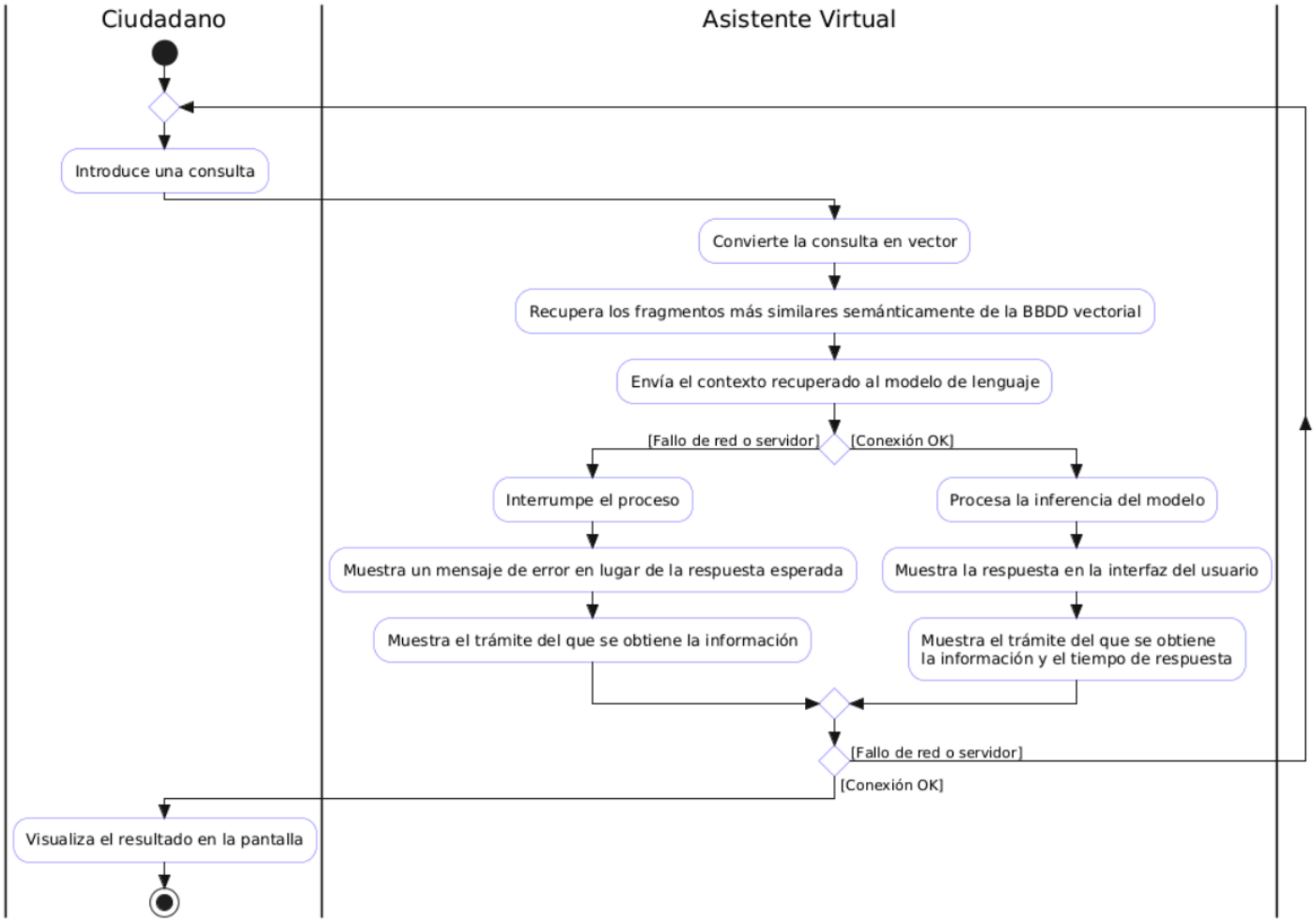


Ilustración 2.6: Diagrama de Actividad, CU-1

Desarrollo del flujo de actividad de CU-1:

- **Inicio e Ingesta de la Consulta**: El flujo de control comienza cuando el **Ciudadano** realiza una consulta. La responsabilidad se transfiere inmediatamente al **Sistema**, que procesa la consulta y la recuperación indexada de los documentos administrativos de la base de datos vectorial.
- **Control de Excepciones**: Se envía el contexto estructurado con las restricciones definidas (prompt) hacia las infraestructuras donde se alojan los modelos. Si se detecta una caída de red o timeout, el flujo se desvía, se interrumpe la ejecución del hilo principal y se inyecta un mensaje de error en lugar de la respuesta generada del modelo evitando que el software se detenga.
- **Cierre**: Una vez que ya se tienen las respuestas, ya sea con inyección del mensaje de error o la respuesta de modelos, el Sistema muestra los resultados en pantalla junto con la fuente de donde se obtuvo la información y tiempos de respuesta. El control vuelve al Ciudadano, garantizando que el cuadro de texto permanezca operativo para formular otra cuestión.

CU-2 Gestión del historial de conversación

El diagrama de la Ilustración 2.7 representa la gestión de las conversaciones pasadas, el historial de conversaciones. Controlando tanto la navegación para explorar las conversaciones como la alternativa de eliminar alguna conversación (escenario alternativo).

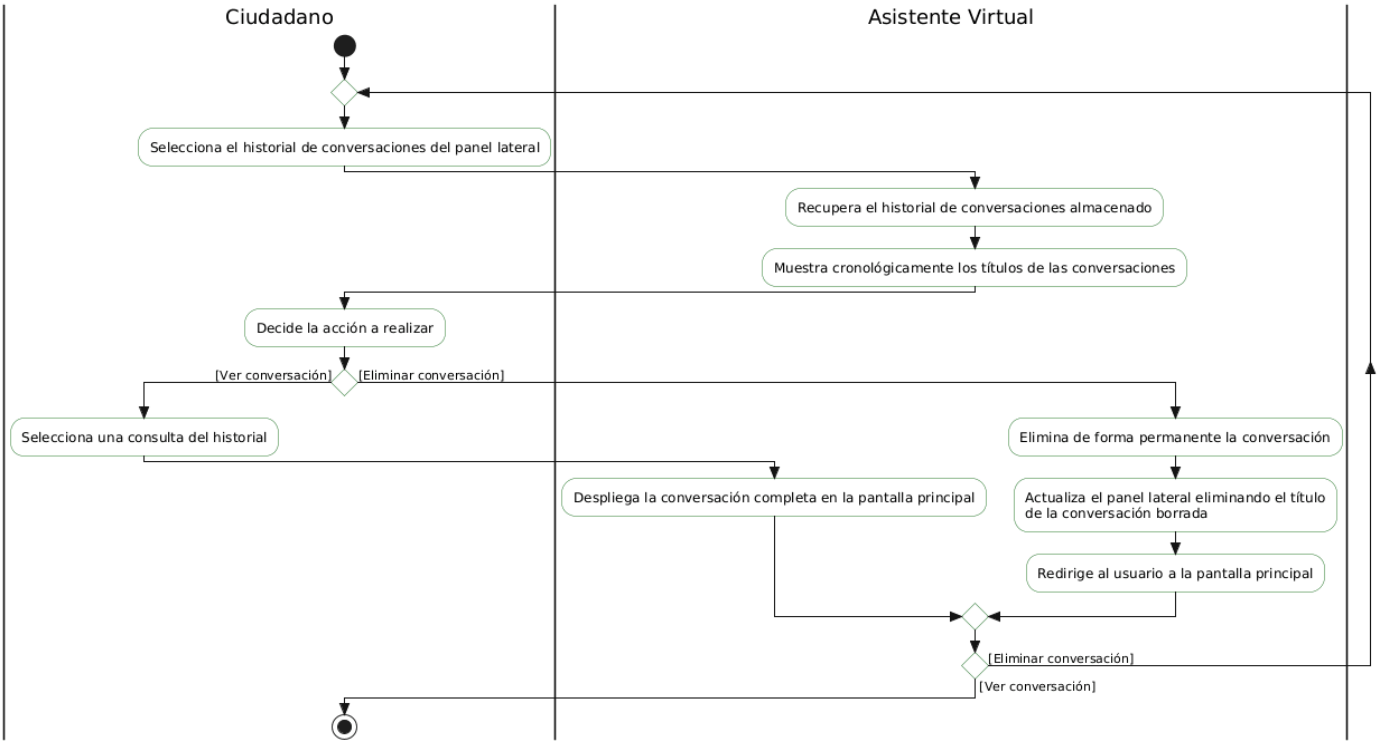


Ilustración 2.7: Diagrama de Actividad, CU-2

Desarrollo del flujo de actividad de CU-2:

- **Inicialización y Carga**: El proceso se comienza cuando el **Ciudadano** selecciona el historial de conversaciones del panel lateral. El **Sistema** asume el control para leer el historial de las conversaciones y mostrar el menú ordenando las conversaciones previas de forma cronológica.
- **Elección de Acción**: Una vez que ya está desplegado el historial con los títulos de las conversaciones, el usuario realiza su acción deseada (nodo de decisión).
    - **Seleccionar Conversación**: Si el usuario selecciona un chat previo, el Sistema procesa su identificación y despliega todo el chat de la conversación en su totalidad (preguntas, respuestas, trámites de origen y tiempos de respuestas).
    - **Eliminar Conversación (Flujo alternativo)**: Si el usuario opta por eliminar la conversación, pulsa el botón para eliminar, el Sistema pide su **confirmación** y, una vez que el Ciudadano confirma la acción, el Sistema la elimina permanentemente, actualiza el panel lateral y redirige al Cludadano a la pantalla de inicio.

2.4. Diagramas de Secuencia

Para modelar el comportamiento dinámico del software y detallar la interacción cronológica de los diferentes objetos del sistema, se representan los diagramas de secuencia asociados a los casos de uso definidos: “CU-1: Consultar trámites administrativos” y “CU-2: Gestión del historial de conversación”

CU-1 Consultar trámites administrativos

A continuación se desarrolla el flujo de datos del caso de uso “Consultar trámites administrativos”. Se presenta su respectivo diagrama de secuencia en la Ilustración 2.8 para tener una visión general del flujo.

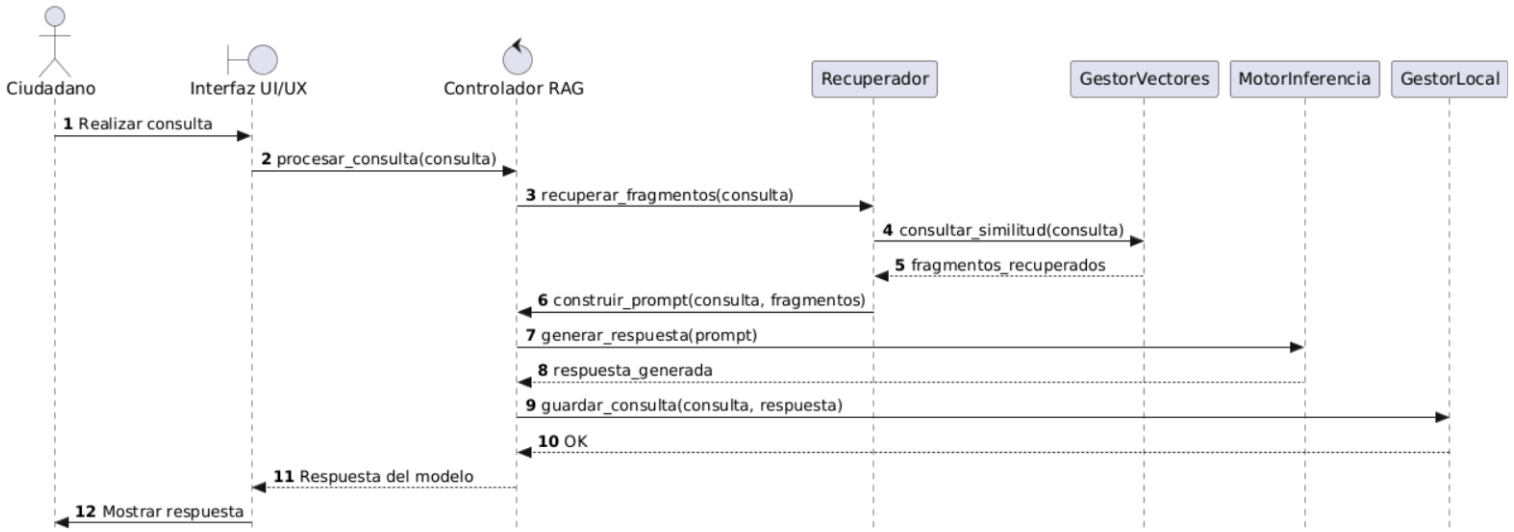


Ilustración 2.8: Diagrama de Secuencia, CU-1

A continuación se desarrolla el diagrama de secuencia del CU 1 “Consultar trámites administrativos” a través de los siguientes pasos:

1. **Petición en la Capa de Presentación (Mensajes 1 y 2)**: El flujo comienza cuando el **Ciudadano** realiza una consulta en el chat de la aplicación (*Realizar consulta*). La **Interfaz** procesa la consulta y delega la lógica de negocio al **Controlador** del sistema, que actúa de fachada, invocando el método *procesar_consulta(consulta)*.
2. **Búsqueda Vectorial (Mensajes 3, 4 y 5)**: El **Controlador RAG** redirige la petición hacia el **Recuperador** mediante el mensaje *recuperar_fragmentos(consulta)*. El Recuperador transforma la consulta en un embedding denso a través del modelo de embedding **bge-m3-es-legal-tmp-6** y y delega la búsqueda de similitud semántica a la clase entidad **GestorVectores**, invocando *consultar_similitud(consulta)*. Tras calcular el producto escalar sobre el índice FAISS, el gestor devuelve los fragmentos recuperados con mayor proximidad geométrica.
3. **Estructuración del Prompt (Mensaje 6)**: El **Controlador** obtiene los fragmentos recuperados por el **Recuperador** y procede a construir el prompt, *construir_prompt(consulta, fragmentos)*. Se inyectan los documentos normativos del Ayuntamiento de Jaén dentro de la plantilla estructurada con las restricciones y ya estaría el prompt consolidado para ser procesado por el modelo.

4. **Inferencia Concurrente (Mensajes 7 y 8)**: El **Controlador RAG** envía el flujo de datos hacia el **MotorInferencia**, ejecutando *generar_respuesta(prompt)*. El **MotorInferencia** obtiene la respuesta del modelo y se la devuelve al controlador.
5. **Persistencia de la Consulta (Mensajes 9 y 10)**: Una vez que ya se dispone tanto de la consulta como de la respuesta, el **Controlador RAG** gestiona su guardado local delegando la acción en el ***GestorLocal***, mediante el mensaje *guardar_consulta(consulta, respuesta)*. Posteriormente, la clase entidad confirma el éxito de la operación de guardado en la base de datos *(OK)* y continúa el flujo.
6. **Mensajes 11 y 12)**: Una vez persistida la consulta con su respuesta, el Controlador transfiere la información, *Respuesta del Modelo*, a la **Interfaz UI/UX**. Finalmente, se muestran por pantalla la respuesta generada, *Mostrar Respuesta*, liberando el cuadro de texto y renderizando la pantalla del ciudadano.

CU-2 Gestión del historial de conversación

A continuación se desarrolla el flujo de datos del caso de uso “Gestión del historial de conversación”. Se presenta su respectivo diagrama de secuencia en la Ilustración 2.9 para tener una visión general del flujo

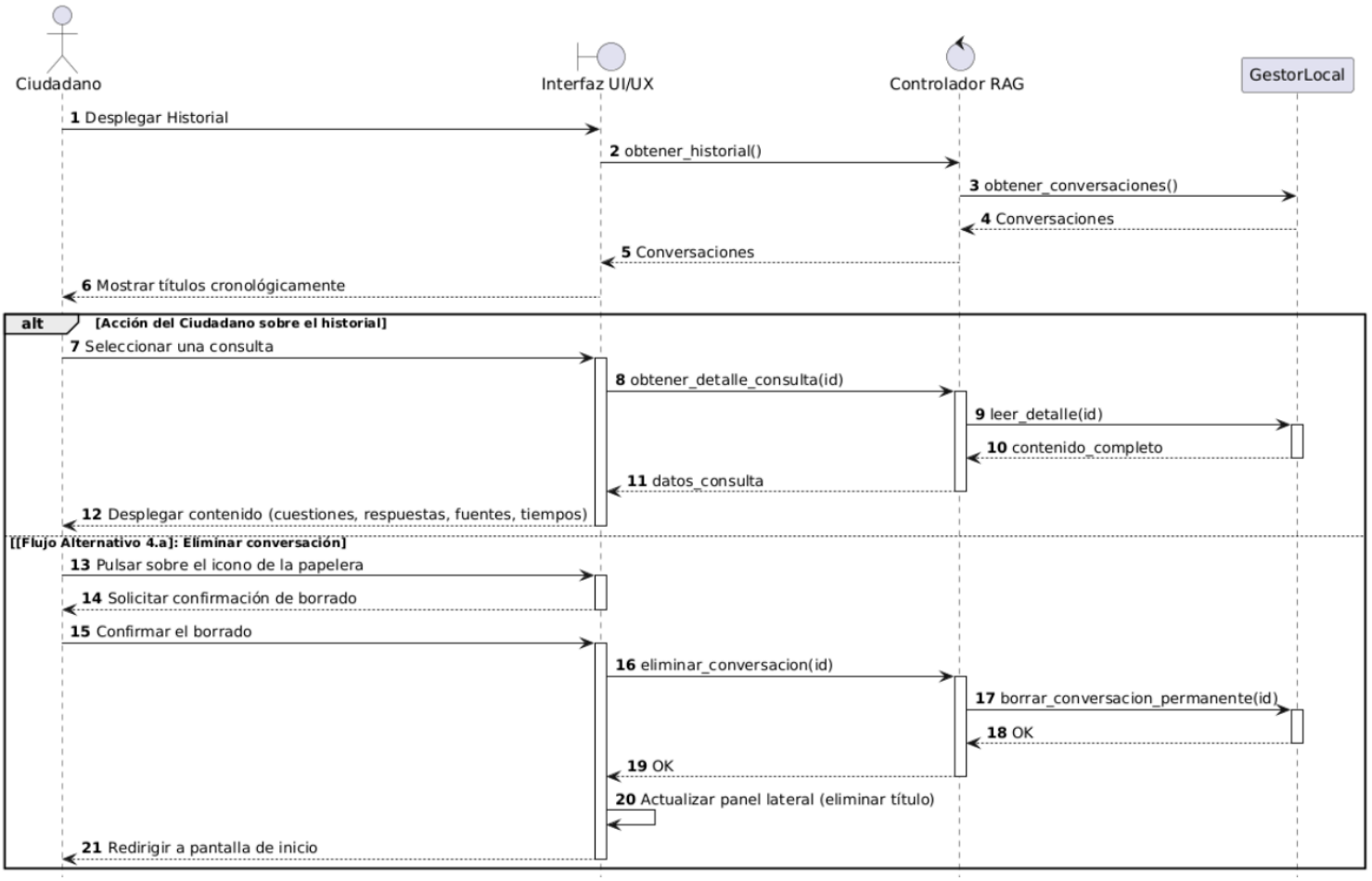


Ilustración 2.9: Diagrama de Secuencia, CU-2

A Continuación se desarrolla el diagrama de secuencia del CU-2: Gestión del historial de conversación a través de los siguientes pasos:

1. **Petición y Recuperación del Historial (Mensajes 1 a 6)**: El flujo comienza cuando el Ciudadano interactúa con la aplicación y decide consultar sus conversaciones previas (*Desplegar Historial*). La **Interfaz UI/UX** delega la petición al **Controlador** invocando el método *obtener_historial()*. El **Controlador** realiza una petición a la clase **GestorLocal** para obtener los registros, *obtener_conversaciones()*. El **gestor** devuelve las conversaciones y el **Controlador** las transfiere de vuelta a la Interfaz. Finalmente, la Interfaz procesa los datos y se encarga de mostrar cronológicamente los títulos.
2. **Consulta Seleccionada (Mensajes 7 a 12)**: En el escenario donde el Ciudadano decide explorar un chat específico del historial (*Seleccionar una consulta*), la interfaz captura el identificador de la conversación y solicita los datos de esta delegando el control al **Controlador RAG**, *obtener_detalle_consulta(id)*. El Controlador obtiene los detalles de la conversación seleccionada a través del **GestorLocal**, *leer_detalle(id)*. El Controlador empaqueta y envía estos *datos_consulta* hacia la **Interfaz UI/UX**. Por último, la interfaz renderiza la información por pantalla y muestra todo el contenido de la conversación (cuestiones, respuestas, fuentes, tiempos), reconstruyendo la en su totalidad.
3. **Eliminación de Conversación (Mensajes 13 a 21)**: Si el Ciudadano elige borrar una conversación del historial, selecciona la opción de eliminar (*Pulsar botón de borrado*). La **interfaz** delega el flujo en el **Controlador** para procesar la petición, *eliminar_conversacion(id)*. El Controlador envía la orden de borrado, *borrar_conversacion_permanente(id),* al **GestorLocal**, el cual elimina de forma permanente la conversación de la base de datos local y confirma el éxito (OK). Finalmente, la Interfaz recibe la confirmación y se actualiza, *Actualizar panel lateral (eliminar título)*, para limpiar la lista visual. La secuencia concluye redirigiendo al ciudadano a la pantalla de inicio.

Capítulo 3. Planificación Temporal y Presupuesto

Este capítulo detalla la viabilidad organizativa y económica del proyecto. Se establece el cronograma de ejecución desarrollado por distintas fases operacionales y se realiza la estimación de la carga económica total asociado al desarrollo del sistema RAG durante un periodo anual completo.

3.1. Planificación Temporal

El ciclo de vida del proyecto se ha planificado para una duración de **52 semanas (1 año)**. El desarrollo se divide en seis fases secuenciales e interdependientes, detalladas a continuación:

- **Fase 1: Investigación y Estado del Arte (Semanas 1 a 8):** Análisis del clúster ADA y la nube Helmholtz, estudio de los modelos de lenguaje (LLM) y del dominio legal-administrativo de la Iniciativa ALIA.
- **Fase 2: Ingesta y Extracción de Datos (Semanas 9 a 16):** Desarrollo y testing de los scripts necesarios para el scraper de la sede electrónica, junto con la extracción y limpieza del corpus documental de trámites administrativos del Ayuntamiento de Jaén (*Catálogo De Procedimientos*, n.d.).
- **Fase 3: Indexación Vectorial y Banco de Pruebas (Semanas 17 a 24):** Se realiza la partición estructural de los trámites, análisis y generación de embeddings, configuración de la base de datos vectorial y generación sintética del dataset de preguntas.
- **Fase 4: Desarrollo del Backend e Interfaz de Usuario (Semanas 25 a 36):** Implementación de la lógica asíncrona y gestión concurrente de múltiples modelos. En paralelo se desarrolla la interfaz de usuario web con distribución de dos columnas.
- **Fase 5: Fase Experimental y Auditoría Semántica (Semanas 37 a 44):** Ejecución del motor de evaluación masiva bajo el framework Ragas apoyándose en los clústeres ADA y Helmholtz, análisis detallado de alucinaciones y ejecución de scripts de recuperación de posibles errores.
- **Fase 6: Optimización, Documentación y Cierre (Semanas 45 a 52):** Optimización del prompt, reducción del tiempo de respuesta y redacción del documento técnico definitivo.

3.1.1. Relación de Actividades y Dependencias para la Planificación Temporal

La gestión y optimización de cualquier proyecto necesita de una secuencia lógica y ordenada de todas sus actividades. Siguiendo los estándares de gestión de proyectos establecidos en **PMBOK** *(Project Management Body of Knowledge)* (Institute, 2017), en este apartado se presenta la **Tabla de Precedencias**, una herramienta que organiza la estructura del proyecto mediante la identificación de cada tarea, su duración estimada y las relaciones de dependencia:

ID	Fase / Tarea Técnica	Duración	Semanas	Tareas Predecesoras
T1	Fase 1: Estudio técnico e investigación	8 semanas	W01 – W08	Ninguna
T2	Fase 2: Desarrollo del módulo de scraper web	8 semanas	W09 – W16	T1
T3	Fase 3: Chunking, Indexación FAISS y Embedding	4 semanas	W17 – W20	T2
T4	Fase 3: Generación del dataset sintético	4 semanas	W21 – W24	T3
T5	Fase 4: Desarrollo Backend FastAPI y Concurrencia	6 semanas	W25 – W30	T3, T4
T6	Fase 4: Frontend UI de Doble Columna y Streaming SSE	6 semanas	W31 – W36	T5
T7	Fase 5: Pruebas de integración y evaluación con Ragas	5 semanas	W37 – W41	T5, T6
T8	Fase 5: Análisis de Alucinaciones y Errores	3 semanas	W42 – W44	T7
T9	Fase 6: Pruebas de carga, optimización y memoria	8 semanas	W45 – W52	T8

Con el fin de proporcionar una hoja de ruta visual que refleje la secuenciación, las dependencias y solapamientos entre fases, en la Ilustración 3.1 se expone el cronograma del proyecto:

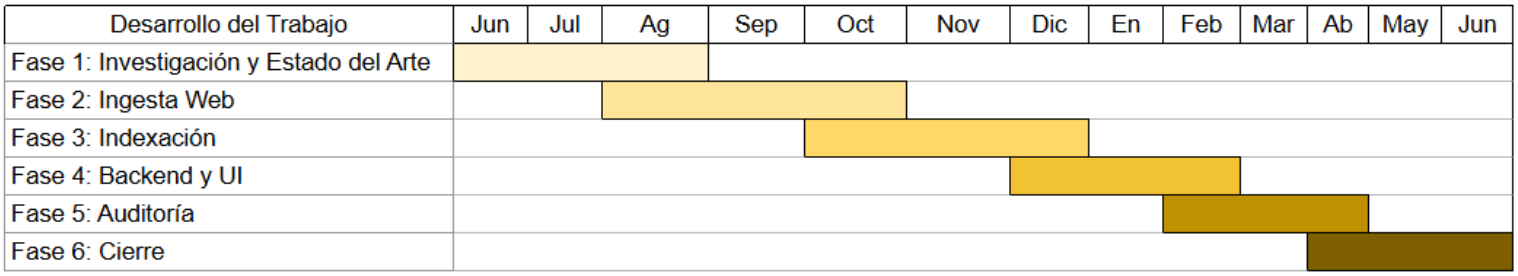


Ilustración 3.1: Diagrama de Gantt

Una vez acotada la distribución temporal del proyecto, es imprescindible calcular el costo humano y los recursos técnicos necesarios para la ejecución del proyecto. En el siguiente apartado se desarrolla el presupuesto financiero de esta planificación.

3.2. Presupuesto Económico del Proyecto

El cálculo del presupuesto se divide en costes directos de personal (ingeniería) y costes indirectos derivados de la infraestructura y recursos de cómputo utilizados, aplicando técnicas estándar de estimación de costes de software (McConnell, 2006).

3.2.1. Costes Directos de Personal

El proyecto ha sido ejecutado por un **Ingeniero Informático Junior** a jornada completa (40 horas semanales) durante un periodo de un año (52 semanas).

Se toman como referencia los siguientes parámetros económicos para la auditoría financiera del coste del desarrollo:

- **Salario Base Bruto Anual (Junior):** 25.000 €
  - *Información obtenida principalmente de randstad.es[1], la media del intervalo salarial tecnológico* (*Cuánto Cobra Un Ingeniero Informático En España*, 2026).
- **Costes de Seguridad Social a cargo de la empresa (32%):** 8.000 €
  - *La legislación laboral en España, regulada por la Orden de Cotización PJC/297/2026[2] en el Boletín Oficial del Estado* (*Orden PJC/297/2026, Por La Que Se Desarrollan Las Normas Legales De Cotización a La Seguridad Social*, 2026)*, establece que el coste empresarial ronda entre el 31% - 32%*
- **Coste Total Anual para la Empresa:** 33.000 €
- **Cómputo de Horas Anuales Laborables:** 1.760 horas
- **Coste por Hora del Ingeniero:** 18,75 €/hora

[1] Salario: https://www.randstad.es/contenidos360/orientacion-laboral/cuanto-cobra-un-ingeniero-informatico-en-espana/
[2] BOE: https://www.boe.es/diario_boe/txt.php?id=BOE-A-2026-7296

- **Coste Semanal del Ingeniero:** 634,62 €/semana

A continuación, se presenta el desarrollo semanal del gasto de personal y agrupados según la duración e impacto en las distintas fases del proyecto:

Fase Operacional	**Semanas**	**Horas Dedicadas**	**Coste Semanal (€)**	**Coste Subtotal de la Fase (€)**
Fase 1: Investigación	W01 – W08	320 h	634,62 €	5.076,96 €
Fase 2: Ingesta y Scraping	W09 – W16	320 h	634,62 €	5.076,96 €
Fase 3: Indexación y Dataset	W17 – W24	320 h	634,62 €	5.076,96 €
Fase 4: Backend e Interfaz UI	W25 – W36	480 h	634,62 €	7.615,44 €
Fase 5: Evaluación y Ragas	W37 – W44	320 h	634,62 €	5.076,96 €
Fase 6: Cierre y Optimización	W45 – W52	320 h	634,62 €	5.076,96 €
TOTAL PERSONAL	**52 semanas**	**2.120 h**	—	**33.000,24 €**

3.2.2. Costes Indirectos

Esta sección contiene tanto el hardware utilizado y los costes derivados del consumo de infraestructuras durante el ciclo de vida el proyecto:

1. **Amortización del Equipo de Desarrollo:** Ordenador portátil de desarrollo (1.200 €). Considerando una vida útil de 4 años (48 meses) y habiéndose utilizado únicamente durante los 12 meses del proyecto, el coste es de **300,00 €**.
2. **Licencia Google Colab Pro:** Entorno de computación en la nube utilizado para la fase de cálculo de embeddings y ejecución de pruebas indexadas en GPU T4. El acceso se obtuvo de forma gratuita mediante una licencia académica con una validez de un año. Coste total: **0,00 €**.
3. **Consumo de APIs de Inferencia:** Utilizada en la Fase 3 para la generación del dataset sintético. Versión gratuita de Groq con límite de peticiones y bajos niveles de cortesía, coste de **0,00 €.**
4. **Infraestructura de Supercomputación (Nodo ADA y Helmholtz Cloud):** El uso del clúster ADA de la Universidad de Jaén (SINAI) y de la nube *Helmholtz Cloud* se ha realizado bajo licencias académicas gratuitas para proyectos de investigación. Su coste es de **0,00 €** (subvencionado).

3.2.3. Consolidación del Presupuesto Total del Sistema

Sumando los costes directos e indirectos, y aplicando un **margen de reserva para contingencias del 5% para imprevistos técnicos (McConnell, 2006)** (como la resolución de fallos de red o recalcular métricas erróneas), el presupuesto final de ejecución del software es:

$$Presupuesto_{Total} = Coste\ Personal + Costes\ Indirectos + Margen$$

$$Margen = (Coste\ Personal + Costes\ Indirectos) * 0,05 = (33.000,24 + 300) * 0,05 = 1.165,01\ €$$

$$Presupuesto_{Total} = 33.000,24\ € + 300\ € + 1.165,01\ €$$

$$Presupuesto_{Total} = \mathbf{34.465,25\ €}$$

Capítulo 4. Estado del Arte y Trabajos Relacionados

En este capítulo vamos a ver toda la base teórica y científica que tiene este Trabajo Fin de Grado. Para ello, empezaremos repasando cómo funcionan por dentro los **Modelos de Lenguaje de Gran Escala (LLM)**, analizando qué es lo que pueden hacer y qué limitaciones presentan, y la evolución del **Procesamiento de Lenguaje Natural (PLN)**. Posteriormente, hablaremos del paradigma **RAG *(Retrieve-Augmented Generation)***, explicando cómo estructuramos el pipeline de ingesta, recuperación y generación de datos. También se explicarán tecnologías clave en la búsqueda y el funcionamiento del RAG como los **embeddings especializados** y la **indexación vectorial** con **FAISS**. Posteriormente, se detalla cómo funciona el framework **Ragas**, utilizado para evaluar el sistema matemáticamente mediante el paradigma ***"LLM-as-a-Judge"***. Por último, se repasan otros **trabajos relacionados** en el ámbito de la administración pública y proyectos actuales para la independencia digital de las lenguas cooficiales.

4.1. Fundamentos de los Modelos de Lenguaje (LLM)

Para empezar, hay que tener claro qué es un modelo de lenguaje, para qué sirve y cómo hemos llegado hasta la tecnología que usamos hoy en día (Jurafsky & Martin, 2026).
Un modelo de lenguaje se define como una **distribución de probabilidad** sobre secuencias de palabras o tokens. A nivel computacional, su arquitectura matemática no emula la comprensión semántica cognitiva humana, sino que opera como un sistema **predictivo** autorregresivo. Basándose en el **aprendizaje estadístico** extraído de corpus masivos de texto, el modelo calcula de manera iterativa la probabilidad de que un determinado token w_t aparezca a continuación, dado un contexto de tokens previos ($w_1, w_2, \ldots, w_{t-1}$), lo cual se puede expresar matemáticamente como :

$$P(w_1, w_2, \ldots, w_n) = \prod_{t=1}^{n} P(w_t | w_1, w_2, \ldots, w_{t-1})$$

En la actualidad, estos sistemas permiten optimizar procesos como la traducción automática, la abstracción y resumen de documentos, la generación de código fuente o, en el ámbito específico de esta investigación, la automatización de interfaces conversacionales basadas en arquitecturas de recuperación de información.

Para contextualizar el desarrollo de este proyecto, es indispensable analizar la evolución de los **Modelos de Lenguaje de Gran Escala (LLM)**. Estas arquitecturas se fundamentan en el paradigma del **aprendizaje profundo *(Deep Learning)***[3] a través de redes neuronales artificiales de múltiples capas (*Liora*, 2026). Dichas redes estructuran nodos interconectados que emulan el procesamiento sináptico biológico, lo que les permite proyectar representaciones vectoriales en espacios de alta dimensionalidad, reconocer patrones abstractos no lineales en volúmenes masivos de datos y transferir dicho conocimiento de forma autónoma hacia tareas específicas de generación de lenguaje natural, tal y como se esquematiza en la Ilustración 4.1

[3] Deep Learning: https://www.ibm.com/es-es/think/topics/deep-learning

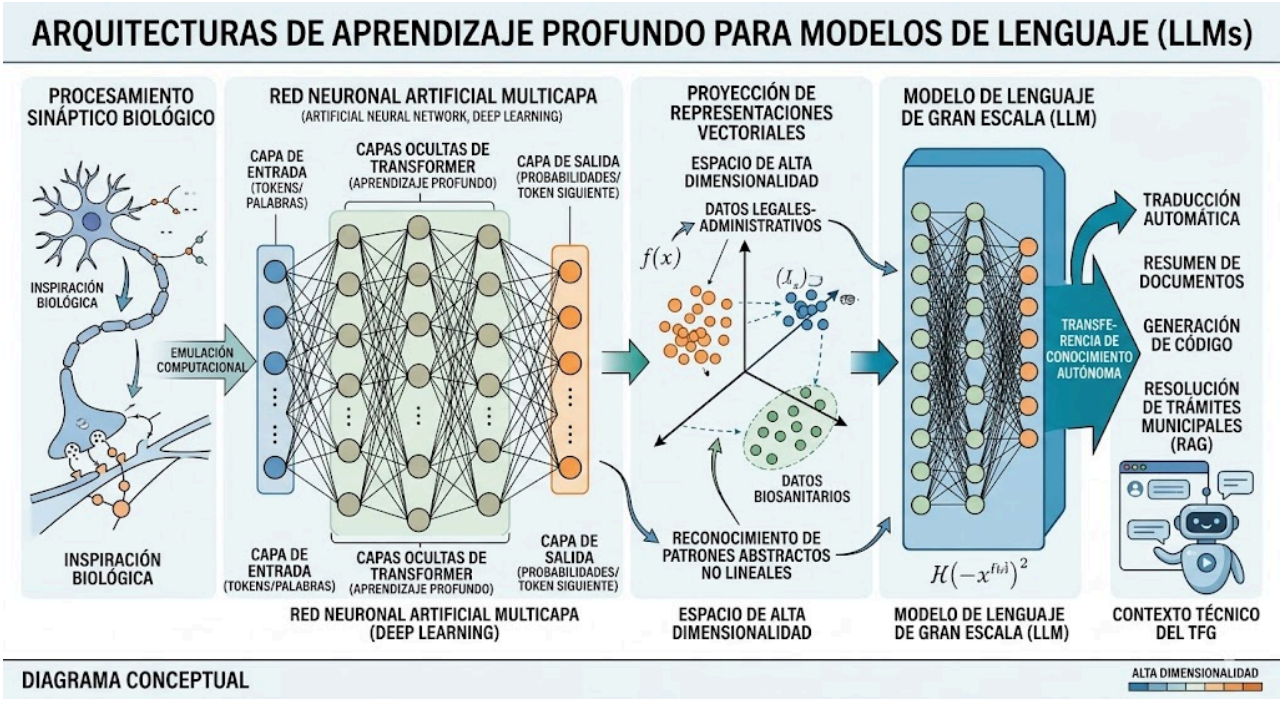


Ilustración 4.1: Esquema de aprendizaje profundo

4.1.1. Contexto Histórico: De las Reglas Semánticas al Transformer

En sus etapas iniciales, comprendidas entre las décadas de 1950 y 1980, el dominio del procesamiento lingüístico estuvo dominado por los **Sistemas Basados en Reglas**. Bajo este enfoque racionalista, expertos lingüistas y desarrolladores diseñaban **reglas gramaticales** de forma manual. Eran sistemas rígidos, manifestando una incapacidad para gestionar la ambigüedad semántica, la polisemia y las variabilidades del lenguaje natural humano.

En la década de 1990, se produjo un cambio de paradigma con la introducción de los **Modelos Estadísticos**. El análisis del lenguaje se abordó mediante aproximaciones probabilísticas basadas en la frecuencia de aparición de secuencia de palabras, formalizadas mediante modelos de **n-gramas**[4] y **Modelos Ocultos de Markov (HHM)**[5]**.** Estas tecnologías permitieron los primeros desarrollos en correctores automáticos y sistemas predictivos de texto, aunque carecían de una representación del contexto, limitada por la incapacidad de capturar las dependencias semánticas más allá de las ventanas de texto reducidas.

A comienzos del siglo XXI, la consolidación del Aprendizaje Profundo impulsó la integración de las **Redes Neuronales Recurrentes (RNN)**[6] y de las arquitecturas de Memoria a Largo Plazo y Corto Plazo (**LSTM**, *Long Short-Term Memory*)[7]. Estos modelos probabilísticos introdujeron por primera vez la idea de estado oculto o memoria recursiva, permitiendo el

[4] N-gramas: https://foqum.io/es/termino/n-grama/
[5] Modelos Ocultos de Markov: https://keepcoding.io/blog/que-es-el-modelo-oculto-de-markov/
[6] Redes Neuronales Recurrentes: https://www.ibm.com/es-es/think/topics/recurrent-neural-networks
[7] LSTM: https://la.mathworks.com/discovery/lstm.html

procesamiento secuencial de textos de longitud variable. No obstante, las **RNN** disponían de ciertas limitaciones computacionales, por un lado, el desvanecimiento o explosión del gradiente dificultaba la retención de dependencias a largo plazo, y por otro lado, la naturaleza intrínseca secuencial de su algoritmo impedía la paralelización del entrenamiento en unidades de procesamiento gráfico (GPU).

Este límite técnico se superó en el año 2017 con la publicación del artículo científico *<<Attention Is All You Need>>* (Vaswani et al., 2017), el cual introdujo la arquitectura **Transformer**[8]. El núcleo de este paradigma radica en la sustitución de los mecanismos recurrentes por el mecanismo de **auto-atención** (*Self-Attention*). Esta propiedad matemática permite calcular la importancia y la interdependencia de todos los tokens de la secuencia a la vez e independientemente de su distancia en el texto, habilitando la paralelización masiva en grandes clústeres de supercomputación y sentando los fundamentos arquitectónicos de los Modelos de Lenguaje de Gran Escala (LLM) que estructuran esta investigación.

4.1.2. Fundamentos Técnicos: Proceso de transformación del texto

El funcionamiento de los modelos basados en ***Transformers*** se divide en tres fases fundamentales. Una visión general de distintas variantes y optimizaciones de esta arquitectura puede encontrarse en diferentes artículos (Lin et al., 2022):

- **Tokenización:** Consiste en fragmentar el texto inicial en unidades mínimas de procesamiento llamadas ***tokens***, que pueden representar palabras completas o subpalabras. Podemos ver un ejemplo visual en la Ilustración 4.2

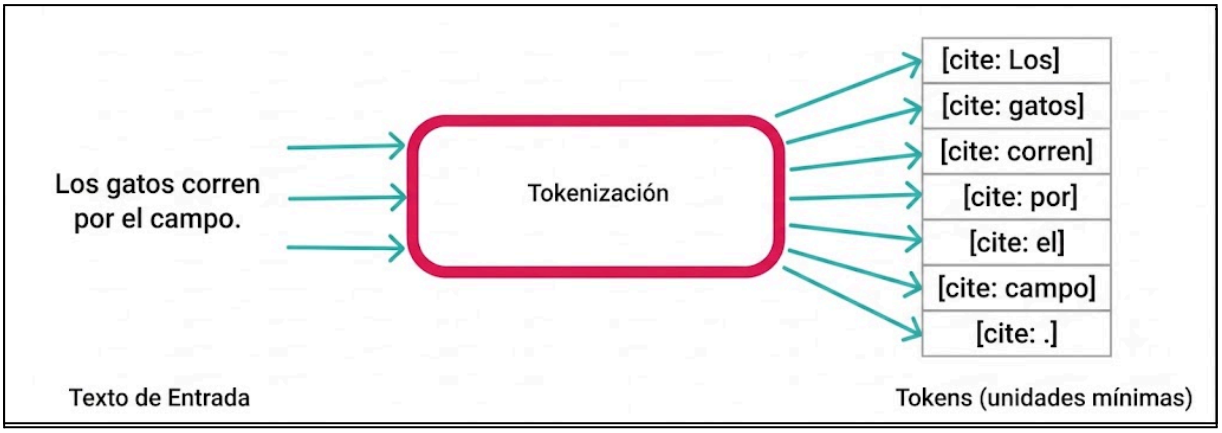


Ilustración 4.2: Tokenización

- **Representación Vectorial (*Embeddings*):** Cada *token* se traduce en un vector numérico dentro de un espacio matemático multidimensional (Base de datos vectorial). Esto permite al sistema medir y entender la cercanía del significado entre distintas palabras. Podemos ver un ejemplo sencillo de su transformación en la Ilustración 4.3

[8] Transformer: https://www.ibm.com/es-es/think/topics/transformer-model

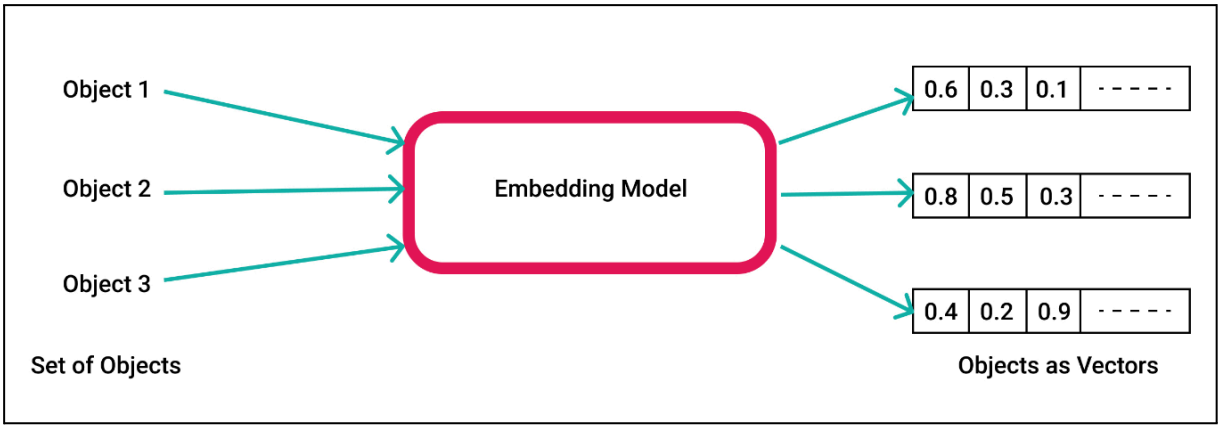


Ilustración 4.3: Vectorización

- **Pesos y Parámetros:** Las redes neuronales transmiten información mediante operaciones matemáticas reguladas por **pesos**. Estos valores se ajustan constantemente durante el entrenamiento del modelo, definiendo los miles de millones de parámetros que determinan qué palabra tiene mayor probabilidad de aparecer a continuación.

4.1.3. Clasificación y Estrategias del Ecosistema de IA

En la actualidad, la elección e implementación de un modelo de lenguaje en el sector corporativo e institucional se rige por tres factores esenciales:

- **Accesibilidad (Cerrados vs. Código Abierto)**
  Los modelos cerrados comerciales (como *GPT-4*) funcionan de manera opaca en servidores remotos, esto compromete la **privacidad de los datos** de una administración pública. Por el contrario, los modelos abiertos permiten una instalación local que asegura el **control total** de la información de forma interna (*Abierto Vs Cerrado*).
- **Dimensión (LLM vs. SLM):**
  Frente a los modelos generalistas de cientos de miles de millones de parámetros (LLM), la tendencia actual apuesta por los **SLM (*Small Language Models*)** de entre 7B y 14B parámetros. Estos modelos optimizados consiguen un equilibrio entre velocidad, fiabilidad y coste computacional (*SLM, LLM*, 2024).
- **Métodos de Adaptación**
  El Ajuste fino (*Fine-tuning*)[9] modifica la configuración interna del modelo al reentrenarlo con un corpus de texto especializado (estrategia utilizada por el **grupo SINAI para especializar a ALIA** en el dominio legal-administrativo). Por otro lado, el RAG consiste en conectar una base de datos externa para ofrecer el contexto en tiempo real sin modificar la estructura del modelo, solución desarrollada en este Trabajo Fin de Grado.

[9] Fine-Tuning: https://www.ibm.com/es-es/think/topics/fine-tuning

4.2. Arquitecturas de Generación Aumentada por Recuperación (RAG)

A pesar de la alta capacidad generativa y la flexibilidad sintáctica de las arquitecturas LLM actuales, su despliegue operativo sin mecanismos externos de control, introduce riesgos críticos en entornos de producción. Los **modelos de propósito general** no disponen de un nivel de especialización requerido para resolver consultas que exigen de una alta fidelidad en la respuesta, mientras que los **modelos ajustados** a dominios específicos disponen de una falta de rendimiento cuando son expuestos a consultas fuera de su entrenamiento. Las arquitecturas de Generación Aumentada por Recuperación (RAG) se han consolidado como el estándar actual para mitigar estos problemas, integrando recuperación de documentos y bases de datos vectoriales (Gao et al., 2023).
Esta problemática justifica el diseño y la implementación de la arquitectura de Generación Aumentada por Recuperación (RAG) desarrollada en este Trabajo Fin de Grado.

Las principales limitaciones de los Modelos de Lenguaje se estructuran en dos puntos fundamentales:

- **Alucinaciones**: Este suceso se produce cuando el modelo genera secuencias de texto coherentes y formalmente verosímiles, pero matemáticamente falsas o sin coherencia con la realidad. Al operar mediante la predicción probabilística de tokens, el sistema prioriza la fluidez gramatical sobre la veracidad, siendo incapaz de evaluar el nivel certidumbre de sus respuestas o de emitir una respuesta negativa por no disponer de conocimiento de forma nativa.
- **Límite de conocimiento:** El conocimiento de un LLM está estrictamente parametrizado y congelado en los pesos de su red neuronal tras la finalización de su fase de preentrenamiento o ajuste fino (***Fine-tuning***). Como no dispone de acceso dinámico a fuentes de información externas en tiempo real, el modelo no es capaz de gestionar datos que sufren actualizaciones constantes, como la normativa legal-administrativa que rige los trámites administrativos objeto de este estudio.

En nuestro caso, si un ciudadano le pregunta al modelo sobre unos plazos o unos documentos a entregar en el marco de un trámite del Ayuntamiento de Jaén, no nos podemos permitir que el modelo se invente la respuesta, ya que estaríamos dándole al ciudadano información falsa en un entorno legal y podría causar un grave perjuicio al ciudadano. Tampoco podemos estar reentrenando el modelo completo, haciendo ***Fine-tuning***, cada vez que el ayuntamiento cambie el contenido de algún trámite porque sería muy costoso en términos de tiempo y dinero. La solución a esto es montar un sistema **RAG (Retrieval-Augmented Generation)**.

4.2.1. El pipeline de un RAG: Ingesta, Recuperación y Generación

El paradigma de Generación Aumentada por Recuperación (RAG) (Lewis et al., 2020,) optimiza el comportamiento de los Modelos de Lenguaje de Gran Escala mediante la vinculación de una base de conocimiento externa al conocimiento del modelo. En lugar de delegar la generación de respuestas exclusivamente en los parámetros estáticos internos por la red neuronal durante su entrenamiento, el sistema obtiene la consulta del usuario, recupera los fragmentos de información con mayor relevancia semántica, en este caso, la normativa y documentación técnica de los trámites del Ayuntamiento de Jaén, y los integra de manera estructurada dentro del **prompt**. Esta inyección de contexto reduce el riesgo de

alucinaciones y **restringe el espacio de búsqueda** del modelo a fuentes documentales verificadas. Podemos observar en la Ilustración 4.4 el esquema de una arquitectura RAG:

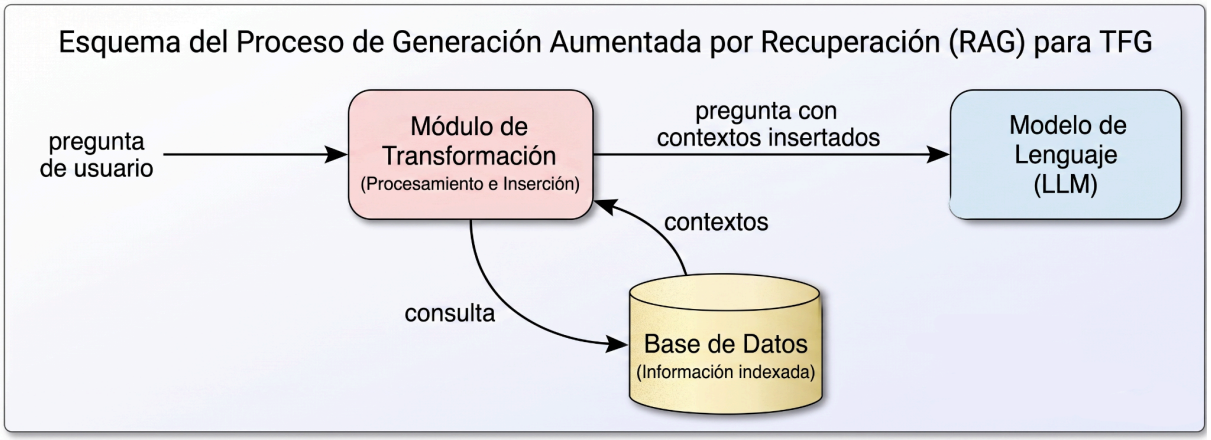


Ilustración 4.4: Esquema RAG

El pipeline de datos implementado en este sistema se divide secuencialmente en tres fases fundamentales: Ingesta de datos (Chunking), Recuperación (Retrieval) y Generación Aumentada (Generation):

1. **Ingesta de datos y troceado (*Chunking*)**
   El pipeline se inicia con la extracción automatizada de la información relativa a los trámites administrativos alojados en la sede electrónica del Ayuntamiento de Jaén. Una vez obtenido este corpus documental en bruto, se realiza una fase de limpieza y normalización textual con el fin de eliminar elementos redundantes o ruido estructural que pueda penalizar el rendimiento del sistema.
   Finalmente, el texto limpio se segmenta de forma controlada en fragmentos de menor tamaño, denominados **chunks**.
   Los trámites están organizados en tablas, cada fila correspondiente a un trámite se transforma en un chunk independiente. Este método genera fragmentos de tamaños variables pero con un límite máximo de longitud (**max_tokens**) para que no sea demasiado grande. Esta restricción es crítica ya que el uso de **ventanas de contexto** excesivamente amplias reduce la capacidad del modelo para extraer la información dentro de bloques densos de texto. Además, se reduce el número de tokens en cada llamada y maximiza la precisión en la recuperación, ya que al mantener los chunks con una única fila se garantiza aislar la información relevante para la consulta del usuario.
   Los chunks se procesan por un modelo de ***embeddings (bge-m3-es-legal-tmp-6)***, que convierte la información textual en vectores, los cuales se guardan en una base de datos vectorial, **FAISS**[10].

   Indexación y Almacenamiento con FAISS
   Estos vectores se almacenan e indexan en **FAISS (Facebook AI Similarity Search)**, una biblioteca de código abierto optimizada para la búsqueda eficiente de vecinos más cercanos (Nearest Neighbor Search) en espacios vectoriales de alta

[10] FAISS: https://faiss.ai/index.html

dimensionalidad. FAISS estructura el índice de la base de datos permitiendo realizar operaciones de comparación masiva no lineales. En este sistema, la biblioteca actúa como el motor de almacenamiento persistente, preparado para ejecutar cálculos de proximidad geométrica basados en métricas como la similitud del coseno una vez que reciba consultas en tiempo real.

2. **Recuperación (*Retrieval*):**
   Cuando el usuario realiza una pregunta al chatbot (por ejemplo, "¿Cómo puedo obtener la tarjeta del bonobús?"), el sistema no le manda la pregunta directamente al modelo. Primero, convierte la frase en un vector numérico usando el mismo modelo de **embeddings bge-m3-es-legal-tmp-6** que usamos en la ingesta. Una vez que la pregunta está transformada en vector, el sistema busca en nuestra base de datos vectorial (FAISS) los fragmentos de trámites oficiales que más se parezcan semánticamente a la pregunta del ciudadano. FAISS calcula esta semejanza de forma masiva, con todos los vectores de la base de datos, y nos devuelve un ranking con los resultados, del cual seleccionamos los **top-K** fragmentos (3 o 5 en nuestro caso). Estos trozos de texto se consideran el contexto óptimo y verídico que se le inyectará al LLM en la siguiente fase para que redacte la respuesta final.

3. **Generación Aumentada (Generation)**
   Una vez que ya tenemos los fragmentos necesarios para responder a la pregunta, juntamos la pregunta del ciudadano con estos fragmentos para montar un prompt óptimo para suministrar al LLM. En la figura 4.5 se muestra un ejemplo:

```
"""Eres un asistente virtual experto del Ayuntamiento de Jaén.
Tu objetivo es responder a las preguntas de los ciudadanos basándote
ÚNICAMENTE en la información proporcionada en el "Contexto Oficial".

Reglas estrictas:
1. Responde de forma clara, concisa, directa y educada en perfecto
español.
2. Si la información para responder a la pregunta NO está en el
contexto, di textualmente: "Lo siento, no dispongo de información
suficiente en los documentos oficiales para responder a esta
consulta." No inventes nada.
3. Si el contexto menciona plazos, órganos gestores o documentos a
presentar, inclúyelos en tu respuesta.

Contexto Oficial Recuperado:
{contexto_unido}

Pregunta del ciudadano: {pregunta}

Escribe tu respuesta a continuación:"""
```

Ilustración 4.5: Ejemplo prompt

El procesamiento del prompt por parte del modelo le permite la definición del rol que condiciona su comportamiento, transformándose en un asistente virtual especializado, se establecen reglas restrictivas minimizando la libertad para que el modelo emita una respuesta negativa en caso de no disponer de la información

necesaria. Por último, la inyección del contexto recuperado aporta el conocimiento necesario para resolver la consulta del ciudadano. Aunque los Modelos de Lenguaje de Gran Escala tienen un riesgo de alucinación ante instrucciones complejas, el diseño de este prompt garantiza robustez entre la respuesta generada y los trámites oficiales del ayuntamiento.

4.3. Tecnologías de Recuperación Vectorial e Indexación

Para la optimización de la fase de recuperación (***Retrieval***) es necesario fundamentos de álgebra lineal y sistemas de gestión de bases de datos vectoriales para garantizar una baja latencia y una alta precisión en las consultas. Los mecanismos de búsqueda léxica resultan insuficientes para interpretar la semántica subyacente en la consulta del usuario, lo que justifica los modelos de recuperación densa (Manning et al., 2008).

4.3.1. Búsqueda semántica frente a búsqueda léxica

La **búsqueda léxica** se fundamenta exclusivamente en la coincidencia exacta de cadenas de caracteres o palabras clave. Bajo este enfoque, si un ciudadano introduce el término *"multa"*, el sistema restringe la búsqueda a la coincidencia exacta de dicho lema. Por consiguiente, si el trámite oficial describe el procedimiento usando sinónimos o términos equivalentes como *sanción* o *penalidad*, el motor léxico omitirá dichos fragmentos porque no hay coincidencia exacta en los **vectores dispersos**, generando un problema en la recuperación de información .

En la administración pública esto es un problema importante, sería muy difícil que el ciudadano encuentre respuesta a sus preguntas. Si un ciudadano de Jaén pregunta al chatbot: "*¿Qué debo presentar para pagar mi multa del coche?*" y el trámite oficial del ayuntamiento se llama "*50035. Alegaciones y recursos a sanciones de tráfico*", la búsqueda tradicional no va a encontrar nada. No coincide ninguna palabra, "multa" no es lo mismo que "sanción" o "presentar" con "alegación". El sistema daría error de que no hay información cuando en realidad sí que hay.

Para solucionar dicho problema, el sistema implementa un modelo de recuperación usando **embeddings densos** para ejecutar la **búsqueda semántica**. Al procesar el corpus a través del modelo ***bge-m3-es-legal-tmp-6***, las cadenas de caracteres se proyectan en vectores densos de **1024 dimensiones** dentro de un espacio geométrico continuo.

El fundamento de esto radica en el aprendizaje por contraste, proceso mediante el cual una red neuronal ha sido optimizada para que aquellos segmentos textuales que comparten equivalencia semántica converjan en coordenadas con una alta proximidad geométrica en el espacio latente. De este modo, vectores asociados a expresiones como *"presentar para pagar mi multa del coche"* y enunciados administrativos formales como *"Alegaciones y recursos a sanciones de tráfico"* se posicionan en regiones adyacentes del espacio debido a su similitud semántica. Gracias a esto, el motor de búsqueda consigue recuperar los fragmentos documentales pertinentes, independientemente de las variaciones en el vocabulario del ciudadano o la presencia de errores ortográficos.

4.3.2. Indexación y búsqueda de similitud con FAISS

A nivel de ingeniería informática, cuando ya tienes todos los trámites divididos en miles de chunks y cada uno convertido en un vector de 1024 dimensiones, te encuentras con un problema de **complejidad temporal**. Si cuando el usuario hace una pregunta tuviéramos que comparar el vector de la pregunta con cada uno de los vectores guardados uno por uno (lo que se conoce como búsqueda por **fuerza bruta** o **Linear Scan**), el servidor fallaría en

cuanto el ayuntamiento subiera unos cuantos documentos más. Sería una solución con una complejidad de $O(N)$ que no haría posible el funcionamiento del chatbot en tiempo real.

Para evitar este cuello de botella usamos **FAISS** (*Facebook AI Similarity Search*) (Johnson et al., 2019), que es una librería de código abierto desarrollada por Meta y optimizada a bajo nivel en C++ (Jegou et al., 2017). FAISS lo que hace es agrupar todos los vectores que se parecen dentro de una estructura de datos eficiente (un **índice vectorial**) y crea caminos rápidos para descartar millones de comparaciones innecesarias, consiguiendo buscar entre miles de documentos en cuestión de milisegundos.

Para calcular si la pregunta del ciudadano se parece a un fragmento de un trámite, FAISS mide el ángulo que forman sus vectores, es decir, calcula el **producto escalar**. Este depende da la longitud de los vectores y del ángulo que forman, por ello el sistema realiza una **normalización previa de los vectores** para que no haya diferencias en la longitud, tolos los vectores son exactamente 1, al hacer esto, el producto escalar pasa a ser equivalente matemáticamente a la **similitud del coseno**[11].

El cálculo es simple: FAISS calcula el ángulo entre el vector de la consulta del usuario (q) y cada uno de los de la base de datos (d).

$$Similitud\ del\ Coseno(q,\ d) = \frac{q \cdot d}{||q||||d||} = \frac{\sum_{i=1}^{n} q_i d_i}{\sqrt{\sum_{i=1}^{n} q_i^2}\sqrt{\sum_{i=1}^{n} d_i^2}}$$

Cuando el resultado de esta operación matemática es cercano a 1 significa que el ángulo entre los vectores tiende a 0º, es decir, apuntan hacia la misma dirección, son semánticamente parecidos y están muy relacionados. FAISS ordena el conjunto de chunks de forma descendente en función de la puntuación de la similitud del coseno y devuelve los **top-K** mejores fragmentos. Estos segmentos son los que se inyectarán en la ventana de contexto en la fase de generación. Podemos observar gráficamente la representación de los vectores de consulta y el documento óptimo para su respuesta en la Ilustración 4.6

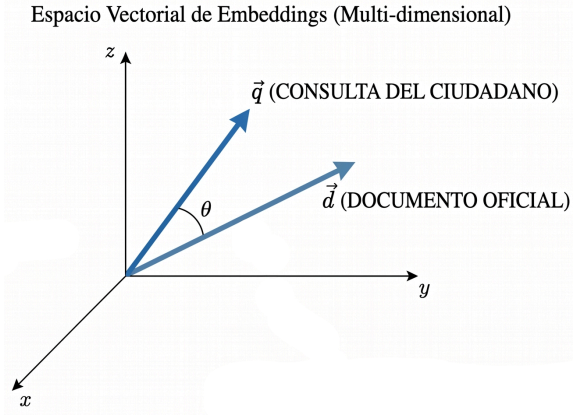


Ilustración 4.6: Ejemplo del espacio vectorial de una consulta

[11] Similitud del coseno: https://www.ibm.com/es-es/think/topics/cosine-similarity

4.4. Marcos de Evaluación Semántica y el Paradigma LLM-as-a-Judge

Uno de los desafíos críticos en el despliegue de sistemas basados en IA generativa radica en la validación de sus salidas. A diferencia del software convencional, los Modelos de Lenguaje de Gran Escala poseen una naturaleza **no determinista**, ante una misma entrada (*prompt*), el modelo puede generar múltiples respuestas sintácticas distintas que son igualmente válidas y correctas en términos de fidelidad semántica. Por consiguiente, las pruebas tradicionales de la ingeniería de software, como la pruebas unitarias o los test basados en la igualdad estricta, resulta ineficaz, dado que provocaría falsos negativos contínuos al evaluar salidas totalmente correctas pero expresadas con diferente estructura léxica.

4.4.1. Por qué las métricas tradicionales (BLEU y ROUGE) no sirven aquí

Antes del auge de los LLM, en tareas de procesamiento del lenguaje natural como la traducción automática o los sistemas de generación de resumen de textos, se utilizaban métricas basadas en la **coincidencia exacta** de palabras clave como:

- **BLEU**[12] **(*Bilingual Evaluation Understudy*)**: Compara las oraciones traducidas por máquinas con oraciones de traducciones de referencia para medir la similitud entre los textos de referencia y las predicciones.
- **ROUGE**[13] **(*Recall-Oriented Understudy for Gisting Evaluation*)**: Es un conjunto de métricas enfocado en la **recuperación**, mide cuánto del contenido de referencia (hecho por humanos) se han conseguido incluir en el texto generado por el modelo. Funciona principalmente comparando la superposición de n-gramas, palabras o secuencias entre el resumen automático y el de referencia.

Para evaluar un asistente virtual estas métricas no son útiles, por ejemplo, si un modelo responde "El coste es de cero euros" y nuestra referencia humana dice "El trámite es totalmente gratuito", el significado es el mismo pero con distintas palabras. BLEU y ROUGE darían puntuaciones mínimas porque no coinciden las palabras. Además, estas métricas son incapaces de detectar **alucinaciones** ya que un modelo puede responder unos plazos o requisitos legales incorrectos pero como coinciden gran parte de las palabras con fragmento textual oficial las métricas darían una buena puntuación.

4.4.2. El framework Ragas y las métricas de evaluación

Para evaluar nuestro sistema RAG de forma objetiva y científica, se ha usado un framework especializado llamado **Ragas**[14] (Es et al., 2023). Ragas no mira el sistema como una caja negra, sino que analiza de forma independiente las dos partes del *pipeline*: la fase de recuperación/búsqueda y la fase de generación. Más adelante se explica a fondo todo este framework, pero para tener una idea general, podemos decir que con ragas calculamos cuatro métricas clave:

- **Context Recall / Recuperación de contexto** (Fase recuperación): Mide si el motor de búsqueda ha sido capaz de encontrar **toda la información** necesaria para responder a la pregunta del usuario.

[12] BLEU: https://www.ibm.com/docs/en/watsonx/saas?topic=metrics-bleu
[13] ROUGE: https://foqum.io/es/termino/rouge/
[14] Ragas: https://docs.ragas.io/en/stable/

- **Context Precision / Precisión del contexto** (Fase recuperación): Comprueba si los fragmentos recuperados son **importantes** o si ha obtenido mucha información **irrelevante** que pueda confundir al modelo.
- **Faithfulness / Fidelidad** (Fase de generación): Mide si la respuesta se basa únicamente en la **información recuperada**, penalizando si añade algo fuera del contexto.
- **Answer Relevance / Relevancia de la respuesta** (Fase de generación): Evalúa si el modelo está respondiendo a lo que le pide el usuario o si responde con texto que no está relacionado con la pregunta.

4.4.3. El paradigma de usar un LLM como Juez

¿Cómo calculamos esas métricas sobre un dataset de preguntas sin tener que revisarlas a mano? Aquí es donde entra el paradigma **"LLM-as-a-Judge"**[15] (Zheng et al., 2023**)**. Se trata de usar un LLM como juez o evaluador.
Este enfoque consiste en utilizar modelos con mayores capacidades que los utilizados para que actúen como examinadores automatizados de los modelos del TFG. Ragas delega al juez la tarea de analizar los textos y ver que son correctos dependiendo de lo que se evalúe.

El paradigma del modelo juez se divide según la tarea específica:

- **Gemma-3-12B-it** (Juez de Recuperación): Se utiliza de forma local en el cluster **ADA** para el **recall** y el **precision**. Entra dentro de este paradigma ya que aprovechamos su capacidad de seguir instrucciones para verificar si los fragmentos extraídos contienen la información correcta.
- **Qwen-3.6-35B** (Juez de Generación): Debido a su gran cantidad de parámetros y su alta capacidad de razonamiento, se encarga de evaluar las respuestas finales del chatbot, midiendo la **fidelidad** y la **relevancia de las respuestas**.

4.5. Trabajos Relacionados

Para terminar este capítulo se presenta un resumen de las soluciones actuales en el ámbito de la automatización y despliegue de modelos de lenguaje, identificando la justificación de desarrollo de este TFG.

4.5.1. Inteligencia artificial en el sector legal y administrativo

La automatización de procesos y la gestión de documentos mediante IA ha crecido de forma exponencial en los últimos años. Hay muchas publicaciones sobre el uso de RAG para optimizar la búsqueda de jurisprudencia o agilizar contratos bastante grandes.

Sin embargo, el problema de estos trabajos radica en su dependencia de Interfaces de Programación de Aplicaciones (APIs) comerciales de código cerrado, principalmente los modelos **GPT de OpenAI.** En la administración pública esto tiene una serie de problemas como son: la **privacidad** de los datos y el cumplimiento **normativo**, debido a que se envían datos confidenciales hacia servidores de Estados Unidos y pierden totalmente la privacidad.
Asimismo, estas arquitecturas presentan un problema en la representatividad lingüística, al haber sido preentrenadas principalmente con un corpus anglosajones. Por consecuencia de esta diferencia cultural y semántica, los modelos globales disponen de una degradación en su precisión y capacidades de transferencia de conocimiento cuando son expuestos al léxico

[15] LLM-as-a-Judge: https://processtalks.com/tecnologia-llm-as-a-judge-y-asistentes-rag/

especializado en las órdenes jurídicas, técnicas y administrativas del Estado español, resultando ineficaces para la gestión de trámites municipales específicos.

4.5.2. Iniciativas nacionales de soberanía lingüística (ILENIA)

Como respuesta a esta independencia tecnológica, en España han surgido proyectos clave como en el caso del proyecto **ILENIA**[16] ***(Impulso de las Lenguas EN Inteligencia Artificial)***, coordinado a nivel nacional por la SEDIA (*Secretaria de Estado de Digitalización e Inteligencia Artificial*). Su objetivo es generar **recursos digitales abiertos** (modelos de lenguaje, herramientas de traducción y corpus de texto y voz) en las lenguas del Estado para que las empresas desarrollen aplicaciones de IA multilingües. El desarrollo de esta iniciativa se descentraliza en 4 centros de investigación especializados, estando cada uno de ellos encomendado a la creación de recursos para cada lengua específica del Estado:

- **AINA** (Cataluña - liderado por el BSC-CNS) para el **catalán**.
- **NÓS** (Galicia - Universidade de Santiago de Compostela) para el **gallego**.
- **GAITU / NEL-GAITU** (País Vasco - Centro HiTZ) para el **euskera**.
- **VIVES** (Comunidad Valenciana - Centro CENID) para el **valenciano**

De esto nace el proyecto **ALIA** y el desarrollo del modelo **Salamandra**, usado en este TFG.

4.5.3 La Iniciativa ALIA

El desarrollo de este Trabajo Fin de Grado se alinea con la estrategia de la iniciativa **ALIA**[17]. ALIA es una iniciativa pionera en la Unión Europea que busca proporcionar una **infraestructura pública** de recursos de IA, como modelos de lenguaje abiertos y transparentes. Su misión es fomentar el impulso del **castellano** y **lenguas cooficiales**, catalán y valenciano, euskera y gallego, en el desarrollo y despliegue de la IA en el mundo.
Es un proyecto estratégico de la Secretaría de Estado de Digitalización e Inteligencia Artificial (**SEDIA**[18]), financiado con fondos públicos en el marco de la Estrategia Nacional de Inteligencia Artificial 2024.



Ilustración 4.7: Alia

[16] ILENIA: https://planderecuperacion.gob.es/noticias/conoce-proyecto-lenia-impulso-lenguas-inteligencia-artificial-ia-prtr
[17] ALIA: https://alia.gob.es/
[18] SEDIA: https://digital.gob.es/ministerio/organigrama/sedia

Se trata de un proyecto abierto que se distingue por la transparencia, en línea con el Reglamento de IA y la apertura para impulsar la innovación y la adopción de las tecnologías, garantizando la fiabilidad tecnológica y la inclusión social y económica.

La familia de modelos ALIA cuenta con la verificación de la **Agencia Española de Supervisión de la Inteligencia Artificial (AESIA)**[19] y está alineada con los estándares de transparencia que establece el Reglamento de IA.

En el caso del entrenamiento de la familia de modelos ALIA, el procesamiento de varios miles de millones de palabras necesita del uso de miles de horas de MareNostrum 5, uno de los superordenadores más potentes del mundo, ubicado y gestionado por el **Barcelona Supercomputing Center**.

Dentro del proyecto nacional ALIA, el grupo SINAI de la Universidad de Jaén genera herramientas en tres áreas estratégicas (Biosanitario, Patrimonial-cultural y Legal-administrativo). Este trabajo se centra en el ámbito **legal-administrativo**, sector en el cual la aportación del grupo SINAI constituye el pilar fundamental para los siguientes componentes del proyecto:

- **Uso del Modelo Especializado en ADA**: Para la fase de generación de respuestas de este proyecto, se utiliza el modelo específico para el dominio jurídico-administrativo desarrollado por el grupo SINAI, ***ALIA-es-legal-7B-Instruct*** en hugging face. Este modelo se encuentra desplegado y accesible en el nodo 08 de la infraestructura de supercomputación **ADA** (UJA) bajo el nombre de ***Salamandra-7B-It-legal-BSC***.
- **Creación de un Dataset propio**: Es fundamental indicar que este trabajo no utiliza el conjunto de datos generado por el grupo SINAI para el proyecto ALIA. En su lugar, de forma independiente y como parte del desarrollo de este TFG, se ha diseñado y construido un dataset propio y exclusivo de 843 preguntas basadas en los trámites reales del Ayuntamiento de Jaén. Se trata de un banco de pruebas para la auditoría del RAG.
- **Representación Vectorial (Embeddings)**: El grupo SINAI tiene desarrollado, gracias a la iniciativa ALIA, modelos de representación **(ALIA-embeddings-legal)** de este dominio. Sin embargo, para este trabajo se evalúa un modelo embedding de terceros. Para ello se realizará una comparativa entre varios modelos para obtener el modelo de representación que consiga la mayor tasa de recuperación de información, justificando así la decisión final.

4.5.4. Aportación diferencial de este trabajo

A pesar del impacto de estas iniciativas, la mayoría de los desarrollos actuales están enfocados en soluciones orientadas a la Administración General del Estado o a la indexación masiva de Boletines Oficiales, tales como el BOE. Este Trabajo Fin de Grado difiere de esta tendencia ya que en él se evalúa el rendimiento de estas tecnologías en un entorno más cercano a la ciudadanía: **la administración pública local**. Tomando como núcleo de la operación el **Ayuntamiento de Jaén**. Este proyecto pone a prueba la arquitectura RAG (*Retrieval-Augmented Generation*) frente a un corpus compuesto por trámites administrativos municipales reales. Además, se evalúa directamente el

[19] AESIA: https://aesia.digital.gob.es/es

comportamiento del modelo ***Salamandra*** ante un dataset de 843 cuestiones, calculando sus métricas de precisión y fidelidad con las obtenidas por arquitecturas de código abierto globales y de referencia en la industria como el modelo ***Llama*** de Meta.

Capítulo 5. Diseño del sistema

En este apartado vamos a definir cómo se organiza el sistema a nivel lógico y cómo es el flujo de la información a través de los diferentes componentes. Se busca entender el diseño de la arquitectura de forma conceptual, nos centramos en los módulos funcionales y en los flujos de datos sin especificar todavía en aspectos concretos de programación o configuraciones de red.

5.1. Arquitectura Lógica del Sistema

La arquitectura dispone de un diseño con un enfoque modular, separando las responsabilidades del sistema (extracción, almacenamiento vectorial, orquestación conversacional, interfaz de usuario y motor de auditoría) para garantizar que el software sea escalable y mantenible.

5.1.1. Módulos Funcionales del Sistema

Buscando la consecución del objetivo de este TFG, la arquitectura lógica se divide en cinco módulos principales, relacionados entre ellos:

- **Ingesta y Extracción de Datos:** Es el módulo el cuál se encarga del scraping y descubrimiento de campos del catálogo de trámites alojado en la sede electrónica del Ayuntamiento de Jaén. Su función principal es transformar las estructuras de datos de la web donde está la información de los trámites (tablas HTML) en texto plano normalizado, ejecutando posteriormente una estrategia de fragmentación (***chunking***) orientada por filas, cada fila un fragmento.
    - Relación de Interfaces: No proporciona ninguna interfaz. Requiere de una interfaz externa de comunicación que proporciona el servidor del **Ayuntamiento de Jaén** para la extracción del HTML, y requiere la interfaz de servicios del **servidor vectorial** para almacenar los fragmentos procesados.
- **Servidor Vectorial (FAISS):** Es el almacenamiento indexado de todos los trámites. Recibe los fragmentos que ha obtenido el módulo de ingesta, estos fragmentos se transforman en vectores numéricos de 1024 dimensiones, gracias al modelo de ***embeddings***, y se organizan en una estructura de datos indexada óptima, ***BBDD vectorial FAISS***. En la fase de consulta se encarga de buscar los fragmentos de texto necesarios para por similitud semántica, calculando el producto escalar.
    - Relación de Interfaces: Proporciona la interfaz para la carga inicial desde el **módulo de ingesta** y la interfaz para la **recuperación semántica** de contextos en tiempo de ejecución.
- **Controlador RAG (Backend RAG):** Es el cerebro lógico del sistema, coordina el flujo. Cuando un usuario realiza una cuestión, pregunta al servidor vectorial, recupera el contexto necesario para resolver la cuestión, monta la plantilla del **prompt** estructurado (***Prompt Engineering***) con las reglas estrictas de control y gestiona las peticiones externas. Su particularidad es que **bifurca la consulta**, enviando a la vez el mismo prompt a dos entornos de ejecución de **LLM diferentes**, *Llama* y *Salamandra*.
    - Relación de Interfaces: Proporciona la interfaz expuesta hacia el cliente (**Capa UI/UX**). Requiere la interfaz del **servidor vectorial FAISS**, las APIs externas al **clúster ADA** y la **nube Helmholtz**, y la interfaz del **motor de evaluación**.

- **Interfaz Gráfica de Doble Columna (Capa UI/UX):** Es el punto de interacción con el usuario. Recibe la entrada de texto del ciudadano y renderiza las respuestas en formato síncrono dividido en dos columnas independientes, una para *Llama* y otra para *Salamandra*, mostrando además metadatos técnicos como el tiempo de generación de la respuesta y la fuente de donde ha sacado la respuesta (el trámite).
    - Relación de Interfaces: No proporciona interfaces hacia otros módulos. Requiere de forma estricta la interfaz del **Controlador (Backend)** para canalizar las consultas del usuario.
- **Auditoría y Evaluación Semántica (Ragas Engine):** Es el componente que se encarga de realizar las pruebas del software. Lee de forma secuencial el dataset con los 843 casos de prueba, ejecuta el pipeline de auditoría de recuperación y generación para todos los modelos, e invoca de forma distribuida a los LLM que actúan como jueces (*Qwen* en generación y *Gemma* en recuperación) para calcular las métricas estadísticas de precisión, recuperación, fidelidad y relevancia.
    - Relación de Interfaces: Proporciona la interfaz de **auditoría** (permitiendo al backend ejecutar auditorías en bloque). Requiere la conexión con los servicios de inferencia de los modelos jueces.

A continuación se muestra en la Ilustración 5.1 un diagrama para tener una visión general de los componentes, que forman el sistema, con sus respectivas relaciones:

Diagrama de componentes

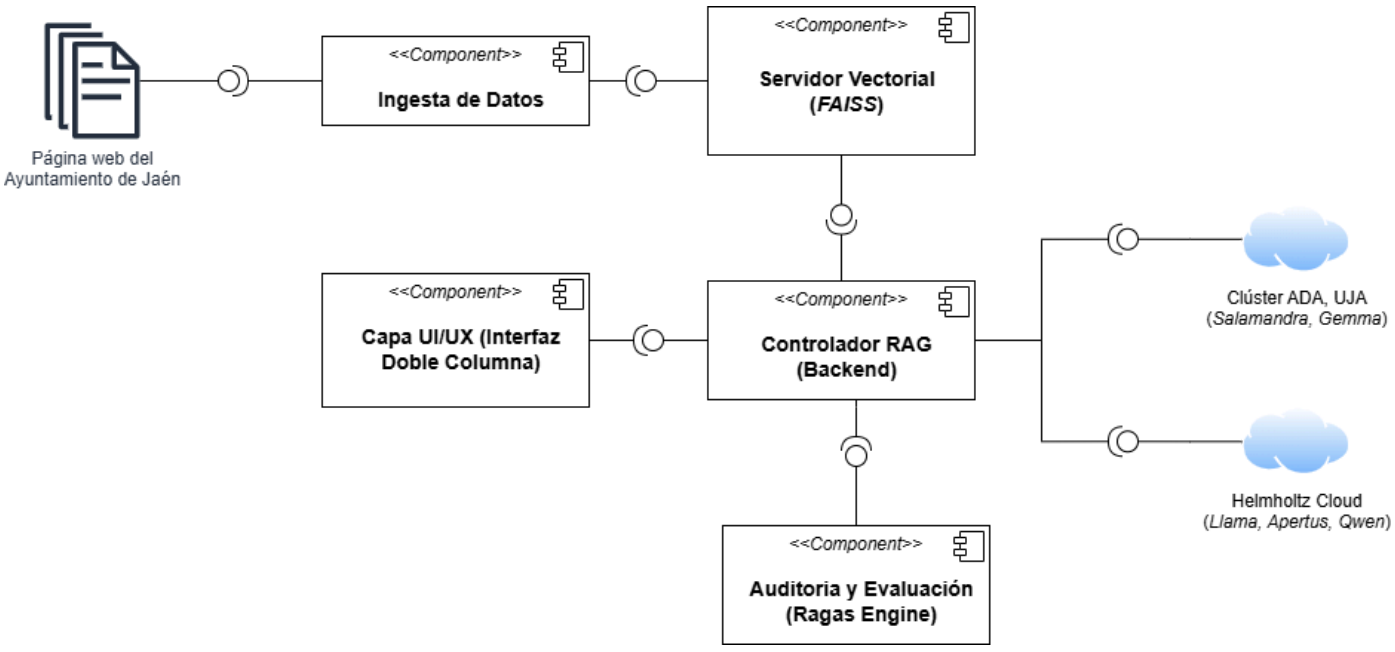


Ilustración 5.1: Diagrama de componentes

En el ámbito de la Ingeniería del Software, la representación de la estructura estática de un sistema se plasma mediante piezas autónomas y modulares con responsabilidades únicas y bien definidas. En este contexto arquitectónico, cada bloque funcional actúa como un componente.

Esta disposición garantiza un desacoplamiento operativo; mientras el módulo de ingesta se centra de forma exclusiva en la extracción de la información de los documentos web, el servidor vectorial opera de manera independiente resolviendo las operaciones de álgebra lineal necesarias por los vectores semánticos.

El uso de este diagrama nos permite ver una abstracción de alto nivel idónea para entender la organización global del sistema, aislando los detalles de implementación de bajo nivel o de la lógica interna de los subsistemas, como ocurre con las dependencias y librerías externas integradas en el motor de evaluación (Ragas Engine).

Por tanto, esta arquitectura se clasifica como un diagrama de componentes al proyectar la estructura del software real en bloques lógicos independientes y reutilizables. Esta estrategia de diseño optimiza la mantenibilidad del código, la escalabilidad de la plataforma y nos facilita la comprensión de la arquitectura RAG desarrollada.

5.2. Diseño de Componentes y Arquitectura de Clases

Una vez definido el flujo de datos global, en este apartado definimos el diseño lógico del software con la **Programación Orientada a Objetos (POO)**.

El objetivo de este diseño es garantizar dos principios básicos de la ingeniería del software: **alta cohesión** (cada clase tiene una única responsabilidad bien definida) y **bajo acoplamiento** (las clases son lo más independientes posible). Al haber definido el sistema con los módulos del punto anterior, conseguimos un software modular donde cualquier cambio en la infraestructura externa, no afecta al resto del sistema.

5.2.1. Patrones de Diseño Aplicados

Para garantizar la construcción de un sistema robusto, con alta cohesión, bajo acoplamiento y con la flexibilidad necesaria para mitigar problemas de concurrencia y caídas de red, se han aplicado los siguientes patrones de diseño estándar (GoF y arquitectónicos):

- **Controlador (*Controller*):** Patrón arquitectónico aplicado en el **Controlador RAG**.
  El Controlador RAG centraliza la recepción de los eventos del sistema que se disparan desde la **Interfaz UI/UX** (como ***procesar_consulta***, ***obtener_historial*** o ***eliminar_conversacion***). Su función exclusiva es **coordinar** el flujo de control, delegando el procesamiento específico de tareas a los subsistemas especializados y aislando la capa de presentación de la lógica de negocio.
- **Fachada (*Facade*):** Patrón GoF que se asume por el propio **Controlador RAG** de cara a la capa frontend. La **Interfaz** no conoce la existencia ni interactúa de forma directa con la base de datos vectorial FAISS, el generador de prompts o APIs de inferencia. El controlador actúa como una “caja negra” que expone interfaces limpias, con una única llamada de la interfaz, el sistema activa todo el desarrollo del proceso, desacoplando la infraestructura.
- **Estrategia (Strategy)**: Patrón GoF aplicado en el **MotorInferencia**. El sistema encapsula las llamadas a los Modelos de Lenguaje como estrategias de ejecución intercambiables. Cambiar de modelo o de infraestructura no rompe el flujo, ya que la estructura del mensaje y los hiperparámetros permanecen constantes, solo cambia el modelo y el tipo de cliente (ada o helmholtz).

- **Tubería y Filtros (*Pipe and Filter*):** Patrón arquitectónico que estructura el flujo secuencial y unidireccional del pipeline RAG. El dato inicial (la pregunta) se somete a una secuencia de filtros de transformación modular: vectorización semántica, filtrado algebraico en la base de datos para recuperar los fragmentos relevantes y el flujo concluye con la encapsulación de estos fragmentos con las reglas en la plantilla del prompt para finalmente dar una respuesta. Esto permite modificar o cambiar el modelo de embeddings o la base de datos sin alterar el resto de la tubería
- **Degradación Elegante / Fallback (Táctica de Tolerancia a Fallos)**: Patrón arquitectónico estructural implementado en la lógica del **MotorInferencia** y controlado por el **Controlador RAG**, el cual gestiona el bloque condicional de excepción, Excepción 3.a, del diagrama de secuencia del CU 1. Ante un fallo de conexión o rate limits, el sistema intercepta dicho error de forma controlada. En lugar de provocar un colapso del software, se desvía el flujo principal de generación de la respuesta y se inyecta un mensaje de error en lugar de la respuesta del modelo, manteniendo el hilo de ejecución activo y el cuadro de texto habilitado para futuras consultas.

5.2.2. Clases del Sistema

A partir de los módulos del sistema definidos en etapas previas, se compone la lógica de negocio del backend con las siguientes clases, indicando sus atributos y métodos bajo el estándar de diseño **orientado a objetos**:

ControladorRAG

Actúa como la fachada del backend y controlador del sistema. Gestiona el ciclo de vida de las consultas (CU-1) y coordina la persistencia de las conversaciones en el historial (CU-2).

- **Métodos:**
  - **+procesar_consulta(pregunta: String): Void**. Recibe la consulta de la Interfaz, procesa el pipeline de recuperación, invoca la inferencia de modelos y maneja excepciones de red.
  - **-construir_prompt(consulta: String, fragmentos: List<String>): String**. Método privado. Conecta los fragmentos recuperados de la base de datos vectorial junto con la consulta y la plantilla estructurada del prompt.
  - **+obtener_historial(): List<Conversacion>**: Solicita al gestor local el historial cronológico.
  - **+obtener_detalle_consulta(id: Integer): DetalleConversacion**. Recupera el contenido completo de una conversación del historial.
  - **+eliminar_conversacion(id: Integer): Boolean.** Gestiona el borrado de una conversación completa del historial.

Recuperador

Clase encargada de interactuar con la base de datos vectorial, abstrae la complejidad de indexación y la búsqueda semántica.

- **Atributos:**
  - **-modelo_embeddings: String.** Identificador del modelo de representación semántica (vectores).
  - **-k_documentos: Integer.** Número de fragmentos que se van a recuperar en cada consulta.

- **Métodos:**
  - **+recuperar_fragmentos(consulta: String): List<String>**. Gestiona la vectorización e invoca al *GestorVectores* para obtener los fragmentos más relevantes.

GestorVectores
Clase entidad que encapsula el acceso directo y las operaciones de bajo nivel con la base de datos vectorial FAISS.

- **Atributos:**
  - **-ruta_indice: String.** Ruta física de los índices vectoriales.
- **Métodos:**
  - **+consultar_similitud(consulta_vectorizada: Vector): List<String>**. Ejecuta la búsqueda algebraica en FAISS y devuelve los textos coincidentes.

GestorLocal
Clase entidad que maneja la persistencia y recuperación de datos en la base de datos local (historial de conversaciones).

- **Métodos:**
  - **+guardar_consulta(consulta: String, respuesta: String): Boolean**. Persiste una nueva interacción en la base de datos.

MotorInferencia
Clase que se encarga de gestionar las conexiones con las APIs de las diferentes infraestructuras que aportan los LLMs.

- **Atributos:**
  - **-cliente_ada: Object**. Instancia autenticada que se conecta al clúster ADA de la UJA.
  - **-cliente_aleman: Object.** Instancia autenticada que se conecta a la infraestructura en la nube de Helmholtz.
  - **-config_parametros: Map**. Configuración con hiperparámteros fijos para los clientes.
- **Métodos:**
  - **+generar_respuesta(prompt: String): Map<String, String>**. Ejecuta la llamada en paralelo a ambos endpoints y obtiene las respuestas o inyecta la excepción en caso de caída de red (Excepción 3.a).

5.2.3. Modelado de Datos y Entidades

Para asegurar que el intercambio de información entre la capa de persistencia (BBDD), el backend y la capa de presentación (Interfaz UI/UX) se realice con bajo acoplamiento y tipado seguro, se han diseñado dos **Objetos de Transferencia de Datos (DTO).**

- **Conversacion**: DTO diseñado exclusivamente para optimizar la carga del panel lateral de la aplicación. Contiene únicamente los metadatos esenciales de una sesión de chat para evitar transferencias pesadas de texto a través de la red
  - **Atributos**:
    - **id: Integer**. Identificador único de la conversación.
    - **titulo: String**. Título de la conversación.
    - **fecha_creacion. DateTime**: Fecha que permite ordenar las conversaciones de forma cronológica.

- **DetalleConversacion**: DTO que encapsula una conversación completa del historial de conversaciones. Se invoca únicamente cuando el ciudadano selecciona una conversación específica para su lectura.
    - **Atributos:**
        - **id: Integer**. Identificador de la conversación.
        - **lista_mensajes. List<Mensaje>**: Colección indexada que agrupa todas las interacciones de preguntas y respuestas ocurridas en ese conversación.
- **Mensaje**: Entidad que modela de forma exacta un mensaje dentro del flujo del asistente. Mapea de forma directa la estructura del objeto persistido en el historial de conversaciones.
    - **Atributos**:
        - **question: String**. La consulta del usuario.
        - **modelo_1: String**. Texto de la respuesta generada por el modelo 1.
        - **modelo_2: String.** Texto de la respuesta generada por el modelo 2.
        - **sources: List<Map>**. Colección de diccionarios con el nombre de los trámites municipales recuperados de FAISS y sus correspondientes URLs de origen.

5.2.4. Diagrama de Clases UML del Sistema

A continuación se muestra en la Ilustración 5.2 el diagrama de clases, que muestra la estructura estática del sistema:

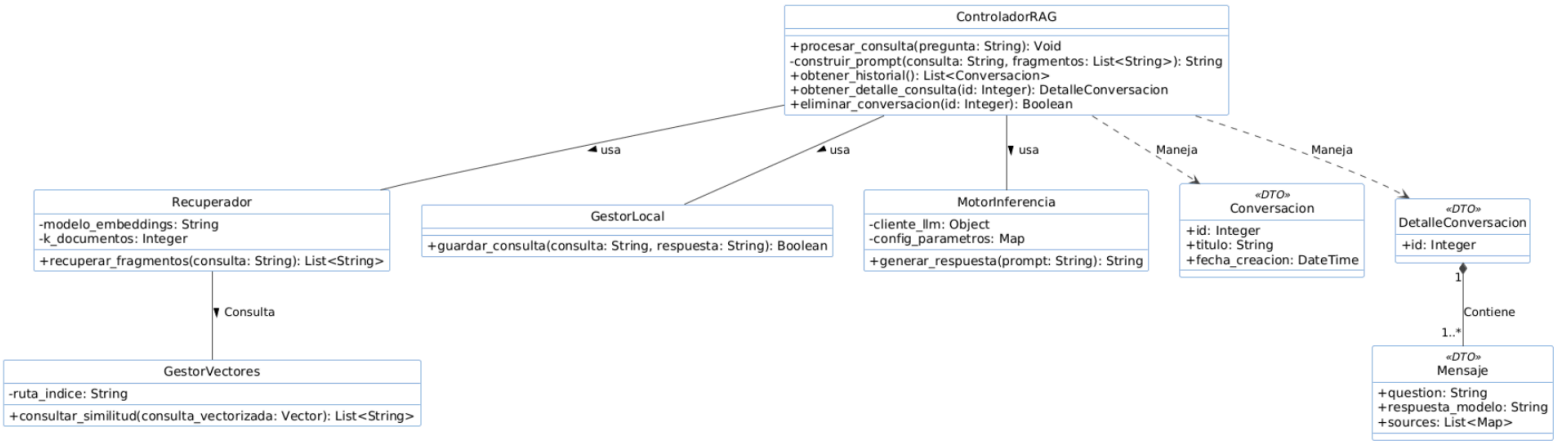


Ilustración 5.2: Diagrama de Clases

5.2.5 Justificación relaciones UML

Asociación: Representa una relación estructural donde un componente conoce y utiliza a otro para poder cumplir con su responsabilidad general.

- **ControladorRAG -- Recuperador**: El ControladorRAG gestiona el flujo principal. Para aplicar la técnica RAG, tiene un vínculo estructural con el Recuperador, delegándole la tarea de buscar y extraer los fragmentos de texto relevantes antes de generar una respuesta.

- **ControladorRAG -- MotorInferencia**: Una vez obtenido el contexto, el controlador necesita un modelo de lenguaje (LLM). Se relaciona con el MotorInferencia para enviarle el prompt final y recibir la respuesta redactada, completando así la fase de "Generación" del pipeline.
- **ControladorRAG -- GestorLocal**: El controlador gestiona la aplicación (como ver el historial o eliminar chats). Para ello, se asocia con el ***GestorLocal***, que es la clase entidad encargada de abstraer y manejar de forma segura la persistencia en la base de datos local.
- **Recuperador -- GestorVectores**: El recuperador tiene la responsabilidad de buscar similitudes semánticas (embeddings). Sin embargo, no accede físicamente a los datos, sino que se vincula con el ***GestorVectores***, delegándole a este las operaciones algebraicas de bajo nivel sobre los índices de la base de datos vectorial FAISS.

Composición: Indica una relación “todo-parte” donde la dependencia es absoluta. Si el contenedor, el “todo”, desaparece, sus partes también deben ser destruidas.

- **DetalleConversacion <>-- Mensaje**: Un objeto DetalleConversacion representa una conversación de chat abierta, la cual está compuesta obligatoriamente por una colección de interacciones (Mensaje). Si el detalle de la conversación se elimina o se cierra, los mensajes individuales que contenía dejan de tener contexto o sentido por sí solos y se destruyen junto con él.

Dependencia: indican que una clase usa a otra de manera efímera, relaciones de uso temporales. Normalmente porque la recibe como parámetro, la crea como variable local o la devuelve como resultado de un método.

- **ControladorRAG ..> Conversacion / DetalleConversacion**: El controlador no contiene estas clases como variables. Las instancias de estos DTOs entran y salen a través de sus métodos (por ejemplo, obtener_historial() retorna una lista de Conversacion). Se utiliza una relación de dependencia porque si la estructura del DTO cambia (como renombrar un atributo), la lógica interna del controlador tendrá que actualizarse obligatoriamente para seguir funcionando.

5.3. Diseño de la Interfaz de Usuario (UI/UX)

Una vez definida la arquitectura lógica y el flujo de datos del sistema es fundamental diseñar la **capa de presentación,** interfaz de usuario *(User Interface, UI)* y experiencia del usuario *(User Experience, UX)*. El objetivo de la interfaz es simplificar el acceso a los trámites del ayuntamiento, se busca transformar un proceso complejo de búsquedas vectoriales y respuestas de inteligencia artificial, en un chat sencillo, rápido y comprensible para toda la población.

5.3.1. Boceto

Como paso previo al desarrollo, se ha elaborado un boceto inicial con papel, lápiz y goma para una idea principal de la interfaz (ver Ilustración 5.3).

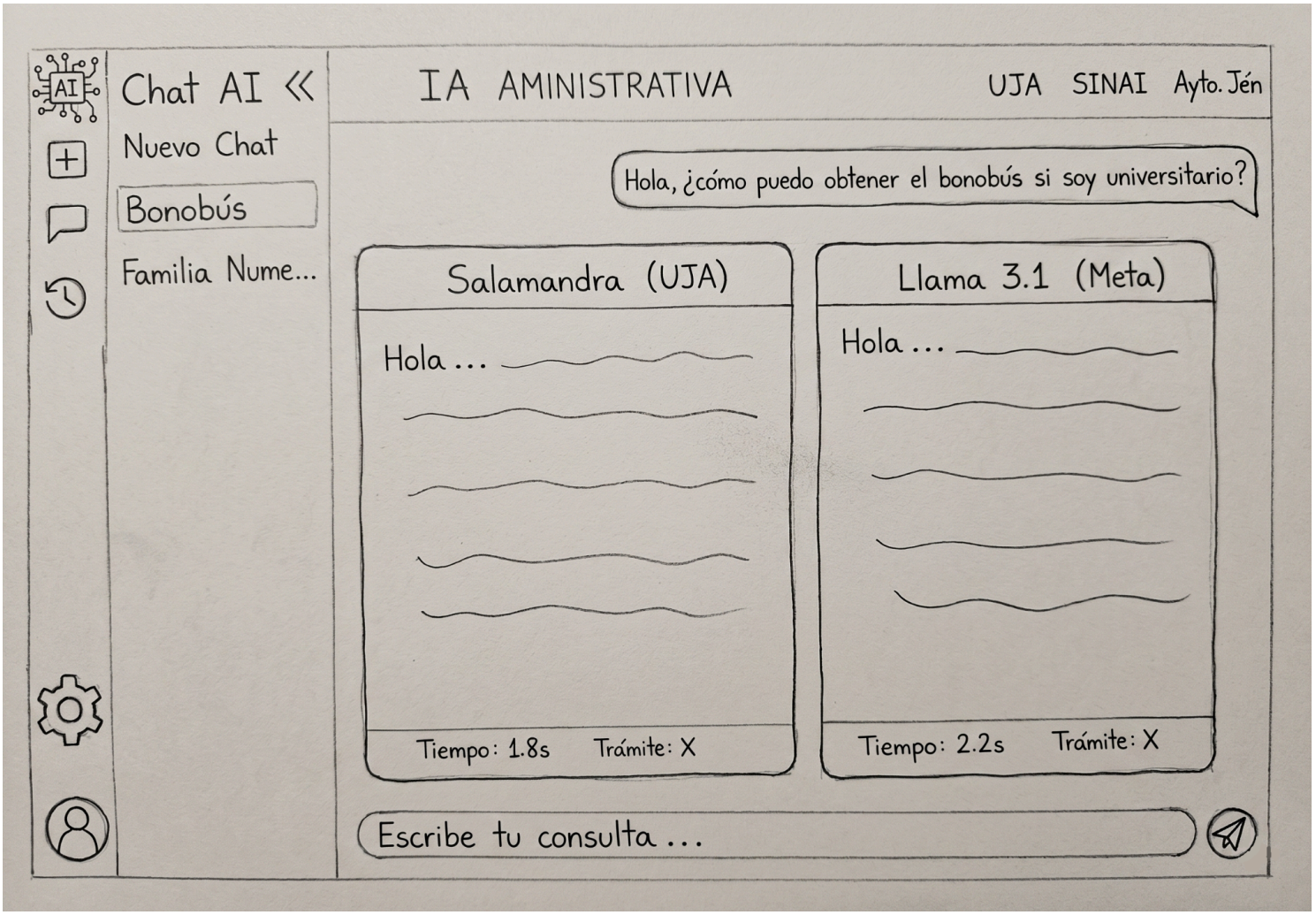


Ilustración 5.3: Boceto de la Interfaz

5.3.2. Principios de Usabilidad y Experiencia de Usuario

La interacción de la aplicación se rige por tres principios de usabilidad orientados a facilitar el análisis comparativo de texto:

1. **Diseño en Paralelo (*Side-by-Side Layout*):** Se sustituye la clásica interfaz de chat basada en un único flujo, en una interfaz dividida en dos columnas para mostrar las respuestas de los modelos en paralelo.
2. **Transparencia y auditoría:** Garantiza la confianza institucional al incluir metadatos explícitos en cada respuesta, detallando el tiempo de respuesta y la fuente de donde ha sido extraída la información, incluyendo hipervínculos al trámite original.
3. **Identidad Corporativa:** El diseño incorpora en su cabecera la identidad de la Universidad de Jaén, del grupo de investigación SINAI y del Ayuntamiento de Jaén. Esto dota a la aplicación de un entorno formal y académico, coherente con el contexto de este TFG.

5.3.3. Arquitectura de la Información y Distribución (Layout)

La interfaz se distribuye mediante un sistema de rejillas (grid) compuesto por tres zonas principales:

- **Panel Lateral (Sidebar):** Situado a la izquierda con una estética clara y minimalista. Gestiona el menú, opción para iniciar una nueva consulta (*Nuevo Chat*), un historial de las conversaciones sobre los trámites consultados, como *"Bonobús"* o *"Familia Numerosa"*. En la parte inferior incluye la configuración y el perfil del usuario. Además cuenta con un botón para comprimir *(<<)* la barra y maximizar el espacio de la lectura.
- **Área Central de Visualización:** Ocupa el centro de la pantalla con un fondo grisáceo/blanco. Se muestra la consulta del usuario, debajo el espacio se divide en dos columnas idénticas donde se mostrarán las respuestas de los modelos, una respuesta en cada columna, y así poder compararlos en tiempo real.
- **Barra inferior de Entrada de Datos (Footer Input):** Bloque estático ubicado en la parte inferior de la pantalla. Contiene un campo de texto redondeado con el marcador de posición o con el texto *"Escribe tu consulta…"*. A la derecha también está el botón de envío representado con un avión de papel.

5.3.4. Componentes Visuales y Guía de Estilos

Para garantizar la legibilidad del texto y un aspecto visual limpio, se ha definido una paleta de colores y una tipografía centradas en su funcionalidad:

Paleta de Colores

- **Contenedores**: Un **blanco** puro *(#FFFFFF)* para las tarjetas de respuestas y la barra lateral, contrastando sutilmente con el fondo de la aplicación. Así el texto oscuro resalta al máximo.
- **Cabecera de Modelos:** Para facilitar la identificación rápida de los modelos, cada tarjeta tiene un degradado suave en la cabecera. Tonos **rosados** para el modelo **Salamandra** y tonos **azulados** para **Llama**.
- **Tipografía**: Se emplea una combinación de fuentes ***Sans-Serif***. **Outfit** para encabezados por su peso y claridad, mientras que **Inter** se emplea para el cuerpo de texto de las respuestas por su legibilidad. El texto base 15px, reduciendo a 12px para los metadatos con un tono **gris**.

Tarjeta de Respuesta

La tarjeta de respuesta es el componente más complejo del frontend ya que es donde se muestra todo el resultado de la consulta que se hace al backend, se compone de tres bloques verticales:

1. **Header:** Muestra un avatar que identifica al modelo(una salamandra para el modelo de la UJA o un robot para el modelo de Meta) junto con el nombre del modelo en negrita.
2. **Cuerpo de Respuesta Estructurado:** Renderiza el ***output*** proveniente del backend.
3. **Footer de Métricas:** Una franja inferior gris que actúa como panel de control. Muestra 2 variables fundamentales para el rendimiento del sistema:
    - **Tiempo:** Latencia exacta de la petición medida en segundos (ej. 1.8s).

- **Fuente:** Nombre del trámite del cual se ha extraído la información para generar la respuesta, enlazable al portal web oficial (ej. *10032-1. Tarjeta Bonobús para Estudiantes Universitarios*).

5.3.5. Control de Estados

Para gestionar la comunicación asíncrona entre el frontend y el ***ControladorRAG*** del backend, la interfaz pasa por 3 estados:

- **Estado de Reposo:** La aplicación está a la espera. La barra lateral es navegable, el campo de texto está activo y se muestra la vista principal de bienvenida.
- **Estado de Carga Asíncrona:** Al enviar la pregunta, el sistema inyecta las tarjetas en la vista. Inicialmente, se muestra que está cargando. Posteriormente, se establece la conexión para observar cómo aparecen las palabras en tiempo real a medida que los modelos generan la respuesta, sin necesidad de esperar que termine el proceso completo.
- **Estado Finalizado:** Una vez finalizado la generación de la respuesta, el texto final que estructurado junto con los metadatos de tiempo y fuentes en el pie de la tarjeta. El sistema actualiza el panel lateral para reflejar la nueva conversación, en caso de que sea nueva y no un mensaje de otra conversación, y deja activa la entrada de texto para la siguiente consulta.

5.3.6. Prototipo

A continuación se muestra en la Ilustración 5.4 el prototipo para que tengamos una imagen más detallada de todo lo desarrollado en los apartados anteriores:

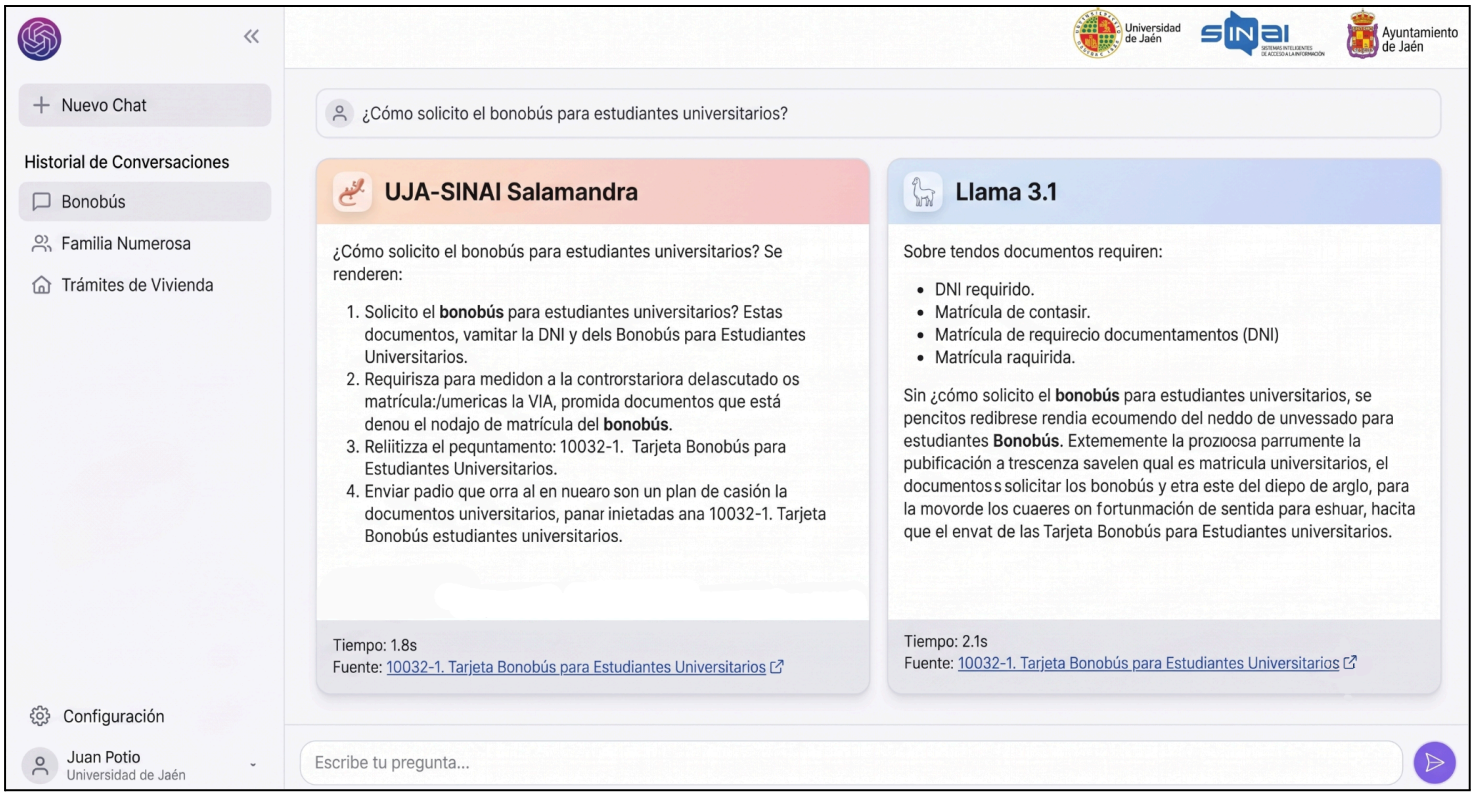


Ilustración 5.4: Prototipo de la Interfaz

Capítulo 6. Implementación del sistema

Este capítulo detalla el desarrollo y el despliegue práctico del asistente virtual basado en el pipeline RAG. El desarrollo del capítulo se articula en dos bloques: en primer lugar, se acota el entorno tecnológico de desarrollo, especificando de forma concisa el lenguaje de programación, los frameworks, las librerías y los recursos de infraestructura de supercomputación y almacenamiento local que sostienen la plataforma; en segundo lugar, se describe secuencialmente el flujo de trabajo de los scripts del sistema, desarrollando el propósito específico, las herramientas usadas y la lógica de cada módulo. Aborda desde la fase inicial de scraping y extracción de los trámites administrativos hasta la orquestación asíncrona del motor de inferencia en tiempo real.

6.1. Entorno Tecnológico y Recursos del Sistema

El desarrollo, despliegue y ejecución del asistente virtual RAG se ha consolidado bajo un entorno tecnológico unificado, optimizado para el procesamiento de lenguaje natural y la persistencia de datos. A continuación se delimitan, clasifican y desarrollan los componentes de software del sistema, especificando qué tipo de recurso es y su rol específico.

6.1.1. Especificación de los Componentes Utilizados

Intérpretes y Lenguajes Base:

- **Python (v3.12)**: Es el **entorno de ejecución** del proyecto. Proporciona el soporte necesario para gestionar los tipos de datos de la aplicación y garantiza la **compatibilidad** con las extensiones utilizadas



Ilustración 6.1: Logo Python

Frameworks de Desarrollo y Automatización:

- **FastAPI (v0.115.12)**: Framework encargado de generar el backend (Ramírez) mediante programación asíncrona. Procesa las peticiones del usuario y gestiona el ciclo de vida de los **endpoints** de control (/api/chat y /api/conversations).
- **LangChain Community (v0.4.1)**: Framework especializado en la arquitectura RAG (Chase). Es el intermediario que permite a la aplicación interactuar de manera genérica con el **almacenamiento vectorial** sin depender de sus APIs internas.



Ilustración 6.2: Logo LangChain

- **Selenium WebDriver (v4.40.0)**[20]: Framework de automatización utilizado en la ingesta de datos. Permite **interactuar dinámicamente** con los menús, botones y elementos de la sede electrónica, simulando una **interacción humana**, para descubrir los trámites ocultos tras el código JavaScript.

Librerías y SDKs Especializados:

- **OpenAI SDK**[21] **(v2.24.0 - AsyncOpenAI (Clase))**: Kit de Desarrollo de Software (SDK). Es una librería que funciona como intermediaria de red. Se encarga de gestionar y empaquetar las conexiones HTTP/HTTPS para **enviar las consultas** del ciudadano con contexto al clúster y **recibir las respuestas** de forma continua (streaming de tokens).



Ilustración 6.3: Logo OpenAI

- **FAISS CPU (v1.10.0)**[22]**:** Librería de alto rendimiento desarrollada por Meta. Realiza los cálculos de producto escalar sobre la matriz de embeddings directamente en la memoria RAM, resolviendo la recuperación de información en milisegundos.
- **LangChain HuggingFace (v1.2.0), Sentence-Transformers (v3.4.1)** y **PyTorch (v2.6.0)**: Librerías especializadas en machine learning y cálculo tensorial (Wolf et al.). Su propósito es instanciar localmente el modelo de **embeddings** dariolopez/bge-m3-es-legal-tmp-6 para **vectorizar en tiempo real** las consultas del ciudadano.



Ilustración 6.4: Logo HuggingFace

- **Requests (v2.32.4) y BeautifulSoup4 (v4.13.5)**[23]: Librerías utilizadas para el scrapeo web. Requests **descarga el código fuente** HTML de forma directa y eficiente, mientras que la BeautifulSoup4 **parsea y limpia** el documento para aislar el texto de los trámites administrativos.
- **PyPDF2 (v3.0.1)**[24]: Librería de manipulación de documentos en formato PDF. Se encarga de abrir, decodificar y extraer el texto plano de los archivos que el ayuntamiento adjunta en las tablas de los trámites.

[20] Selenium: https://www.selenium.dev/blog/2026/selenium-4-40-released/
[21] SDK: https://developers.openai.com/api/docs/libraries
[22] FAISS: https://pypi.org/project/faiss-cpu/
[23] BeautifulSoup4: https://pypi.org/project/beautifulsoup4/
[24] PyPDF: https://pypi.org/project/PyPDF2/

Módulos Nativos:

- **AsyncIO**[25]: Librería estándar de Python que controla la concurrencia asíncrona de la aplicación, permitiendo que la transmisión de tokens de los modelos se ejecuten simultáneamente sin congelar el hilo principal.
- **JSON**: Librería estándar de Python que codifica y decodifica estructuras de datos. Se utiliza para la persistencia local y para formatear los datos de las sesiones.

Servidores de Red y Herramientas DevOps:

- **Uvicorn (v0.34.0)**: Servidor web basado en la especificación ASGI *(Asynchronous Server Gateway Interface)*. Herramienta encargada de escuchar en la dirección y puerto local configurados (127.0.0.1:8000), administrar los sockets de red y canalizar las llamadas hacia la aplicación FastAPI.

6.2. Recursos de Infraestructura y Hardware

Para que el software sea operativo, el sistema se acopla a tres recursos indispensables de almacenamiento e infraestructura de supercomputación:

- **Recurso de Inferencia (Clúster ADA - UJA)**: Nodo de supercomputación remoto gestionado por el grupo de investigación SINAI de la Universidad de Jaén. Actúa como el motor que aloja y ejecuta los modelos **Gemma-3-12B**, usado en la evaluación, **Salamandra-7B-It**, usado en la evaluación e implementación, y **Llama-3.1**, usado en la evaluación e implementación.
  La aplicación se conecta de forma segura mediante
  - **Protocolo HTTP interno**
  - **Endpoint** http://ada01.ujaen.es:8080/v1
  - Utilizando autenticación por **API Key**, token alfanumérico proporcionado por los administradores del grupo SINAI.
- **Recurso de Recuperación (Índice FAISS Local)**: El **Índice FAISS** es un directorio físico (faiss_index) integrado en el servidor de la aplicación. Almacena de forma binaria los embeddings de los textos de los trámites administrativos, permitiendo búsquedas de similitud matemática rápidas a nivel local.
- **Recurso de Persistencia**: Archivo **JSON** físico plano (conversations.json) utilizado como base de datos local ligera para guardar el historial de las consultas, respuestas, fuentes y tiempo de respuesta de las sesiones.

6.3. Flujo del desarrollo de scripts

Una vez definidos los recursos hardware y el entorno de ejecución, se detalla el pipeline de desarrollo del sistema. Se describe el propósito, los componentes usados y la lógica del script, el flujo de ejecución completo.

6.3.1. Análisis Web y Descubrimiento de Campos

Propósito

Automatizar la extracción de las etiquetas de las filas que componen las tablas de los trámites administrativos. El conjunto resultante actúa como molde estructural que garantiza que la fragmentación de la información posterior (*chunking*) sea limpia y alineada con su trámite correspondiente.

[25] AsyncIO: https://docs.python.org/es/3/library/asyncio.html

Componentes, Frameworks y Librerías Utilizados

- **Framework Selenium WebDriver:** Manejo de la interacción con el catálogo de los trámites para simular los clics en los desplegables de la web.
- **Librería Requests:** Descarga los HTML de los trámites. En cuanto se obtienen los enlaces a los trámites se cierra Selenium y se pasa el control a ***requests*** para bajarse el código HTML.
- **BeautifulSoup4:** Se utiliza para parsear y analizar el código HTML de cada trámite y obtener los campos de las tablas, rastrea la estructura de la página web mediante filtros para obtener el texto limpio.
- **Estructura set() de Python:** Almacenamiento optimizado en memoria para la eliminación automática de registros duplicados.

Lógica del Código (obtenerCampos.py)

1. **Extracción de los Identificadores:** El script inicializa Selenium, localiza el elemento de paginación **CATSERV_LISTA_length** mediante su atributo **NAME** y ejecuta **select_by_value('-1')** para cargar todos los trámites en una sola vista, almacenando sus IDs en una lista dinámica.
2. **Extraer campos:** El script recorre la lista de IDs haciendo peticiones HTTP con **requests**. **BeautifulSoup** busca la lista principal (***<dl>***) e itera sobre los términos de la clase ***<dt class="entry-label">***, que es el lugar donde están escritos los títulos de cada fila de cada trámite.
3. **Antibloqueo y Persistencia:** Para evitar que el cortafuegos del ayuntamiento bloqueara la IP,, al final de cada iteración se introduce una **pausa aleatoria**, entre 0.5 y 1. Antes de esto se guardan los campos obtenidos en el set creado anteriormente.

6.3.2. Ingesta de Trámites Administrativos

Propósito

Realizar la ingesta masiva de todos los datos de los trámites y normalizarlos. El script descarga el texto de los trámites y procesa los documentos PDF adjuntos, guardando un archivo .txt estructurado por trámite y delimitando los campos con marcadores **###** para facilitar la segmentación semántica posterior.

Componentes, Frameworks y Librerías Utilizados

- **Framework Selenium WebDriver:** Se utiliza igual que en el script anterior obtenerCampos.py, con el fin de obtener los IDs de los trámites y navegar por ellos.
- **Librería Requests (requests.Session):** Una vez obtenidos los enlaces se abre una sesión de **requests**. Esta reutiliza la misma conexión TCP para obtener las páginas de los trámites, desactivando la validación SSL **(verify=False)** para evitar que el script falle por problemas de certificados del Ayuntamiento.
- **Librerías BeautifulSoup4 y PyPDF2 (PdfReader):** Extraen los datos. **BeautifulSoup** parsea el árbol DOM del HTML, y crea una copia equivalente para que lo lea Python, y **PyPDF2** se encarga de leer los flujos binarios de los PDFs adjuntos
- **Módulo google.colab.drive:** Infraestructura utilizada para montar nuestro almacenamiento de Drive en la ruta ***/content/drive***, para guardar los archivos directamente en la nube.

Lógica del Código (Scraper.py)

1. **Selección de datos:** Reutiliza el listado de **IDs** previos y abre una sesión **HTTP persistente**. **BeautifulSoup4** valida que las cabeceras <dt> coincidan con la lista de títulos obtenidos anteriormente antes de extraer el contenido textual de cada celda
2. **Extracción de PDFs:** Si BeautifulSoup encuentra un enlace (<a>) que apunta a un PDF, el script realiza una petición HTTP con **stream=True**. En lugar de guardar el PDF en el disco, obtiene los bytes directamente en la memoria RAM usando un buffer de **BytesIO**, clase de la librería ***io***. **PdfReader** procesa ese buffer, saca el texto del documento y lo guarda en el documento del trámite entre etiquetas controladas.
3. **Inyección de Contexto y Formateo**: Para asegurar que el buscador RAG no alucine, se inyecta la variable **contexto_tramite,** que contiene el nombre del trámite y su URL, al final de cada campo para indicar a qué trámite pertenece ese campo. Además se usan delimitadores para las palabras (**|||TOKEN|||**) para no romper la estructura, luego estos token se cambiarán por saltos de línea dejando el texto ordenado.
4. **Guardado de archivos:** Algunos trámites del Ayuntamiento tienen nombres que hacen que el sistema operativo no permita el guardado, lanza una excepción al intentar crear el archivo **.txt.** Para que el script no fallara si se detectaba este error, se limpia el nombre dejando solo valores alfanuméricos o el id del trámite.

6.3.3. Procesamiento, Chunking e Indexación Vectorial

Propósito

Transformar los archivos de texto planos, obtenidos en el scrapeo, en un repositorio indexado vectorialmente para su recuperación en tiempo real. Implementa una segmentación guiada por la estructura de las tablas de los trámites, inyecta metadatos contextuales en cada chunk y genera archivos binarios (*faiss* y *pkl*) para búsquedas.

Componentes, Frameworks y Librerías Utilizados

- **Framework LangChain:** Encapsula fragmentos de texto unidos a metadatos (ID, URL, título) en objetos estructurados de tipo *Document*.
- **Librería HuggingFace Embeddings:** Carga el modelo especializado de representación semántical ***bg3-m3-es-legal-tmp-6***.
- **Librería PyTorch (Entorno CUDA):** Cargar los embeddings en la memoria de la tarjeta gráfica GPU T4 de Google Colab reduciendo así el tiempo de cálculo.
- **Base de Datos Vectorial (FAISS):** Motor local de base de datos de vectores que opera directamente sobre la memoria RAM para eliminar latencias de red.
- **Librería Pickle de Python:** Guarda el texto de los trámites en un archivo binario ***.pkl*** para dejarlo preparado para la búsqueda tradicional BM25 que evaluaremos.

Lógica del Código (indices.py)

1. **Chunking Estructural:** Fragmentación del texto de los archivos mediantes expresiones regulares utilizando los delimitadores **###** puestos durante el scrapeo. Esto garantiza que cada chunk equivale a un campo de la tabla, si ese campo tiene una longitud mayor que el máximo establecido del chunk pues se divide, con esto no se mezcla información de distintos campos.

2. **Inyección Contexto y Limpieza:** Se extrae el id del trámite y la URL. Para evitar que el buscador se confunda con fragmentos sueltos, se reescribe el texto antes de vectorizarlo, añadiendo la cabecera

   TRAMITE: {nombre}\nCAMPO: {titulo}\n{contenido}

   Con esto nos aseguramos que el fragmento siempre guarde de donde proceden sus datos.

3. **Splitter Recursivo:** La fragmentación del texto es principalmente por campos pero si hay alguno que exceda los **2000 carácteres** se fragmenta para evitar un contexto excesivo para el modelo de lenguaje. Si supera dicho tamaño, el objeto ***RecursiveCharacterTextSplitter*** de LangChain divide el fragmento en bloques menores a 1200 con un desbordamiento (overlap) de 200.
4. **Normalización y Persistencia:** Al inicializar el modelo de embedding se activa el parámetro ***normalize_embeddings=True***. El software aplica una normalización L2 a los vectores convirtiéndolos en vectores unitarios, permitiendo que FAISS calcule la similitud del coseno mediante un producto escalar. El proceso concluye guardando en disco el índice vectorial (*index.faiss*) y los objetos estructurados serializados con pickle (bm25_chunks.pkl).

6.3.4. Interfaz y Gestión Asíncrona

Propósito

Desplegar el backend de la aplicación del asistente virtual y gestiona la comunicación en tiempo real con el ciudadano. Recibe la consulta desde la interfaz, realiza la búsqueda vectorial en la base de datos local, construye el prompt y lanza peticiones concurrentes a los modelos Llama y Salamandra. Finalmente devuelve las respuestas mediante **streaming** para rellenar las columnas de la interfaz, además se guarda el historial de las conversaciones.

Componentes, Frameworks y Librerías Utilizados

Para desarrollar este script se han usado las siguientes herramientas:

- **Framework FastAPI:** *Framework* que crea APIs en Python. Se utiliza para levantar el servidor, exponer los *endpoints* HTTP y muestra el diseño visual final.
- **Asyncio:** Módulo de Python que realiza varias tareas a la vez. Pregunta a los dos modelos juntos y evita que el sistema se bloqueé.
- **AsyncOpenAI:** Cliente asíncrono que gestiona las peticiones HTTP hacia el clúster ADA (UJA) donde están los modelos generadores.
- **FAISS y HuggingFace Embeddings:** Se cargan para realizar las búsquedas de similitud en milisegundos.
- **Módulo json:** Se encarga de guardar los datos en un archivo local *conversations.json.*

Lógica del Código (app.py) y Análisis Cualitativo

1. **Inicialización:** Al levantar el servidor ASGI mediante Uvicorn, la aplicación ejecuta una sola vez la carga en memoria RAM de los pesos del modelo de embeddings y del índice binario FAISS, para evitar cuellos de botella en peticiones sucesivas
2. **Gestión de Endpoints:** Se crean rutas estructuradas que permiten al cliente

inicializar el chat **(/api/conversations** via POST), gestionar el menú lateral cronológico (GET), recuperar conversaciones pasadas (GET a **/api/conversations/{id}**) y eliminar registros físicos de conversaciones (DELETE).

3. **Recuperación y Construcción del Prompt:** Al realizar una consulta en el endpoint **/api/chat**, el método **similarity_search_with_score** obtiene los 3 fragmentos semánticamente más próximos a la consulta en FAISS. Los textos se inyectan dentro de una plantilla de instrucciones con reglas explícitas penalización de alucinaciones.
4. **Concurrencia y Streaming:** El controlador duplica el prompt e instancia dos tareas asíncronas paralelas hacia los modelos Salamandra y Llama. Las tareas depositan los tokens generados de forma concurrente en una estructura de cola compartida (asyncio.Queue). Finalmente, un generador de eventos vacía la cola y envía los datos en tiempo real al navegador empleando el protocolo **Server-Sent Events (SSE)**, guardando el registro final en conversations.json de forma local.

6.2.5. Script de Automatización y Despliegue Operacional

Propósito

Automatizar el despliegue local en Windows asegurando que las dependencias estén completamente aisladas. Se crea un entorno virtual la primera vez que se ejecuta. El sistema arranca en paralelo y de forma segura el servidor backend junto con el frontend en el navegador.

Componentes, Frameworks y Librerías Utilizados

Para desarrollar este script se han usado las siguientes herramientas:

- **Python (v3.12)**: Forzado mediante el comando **py -3.12**.
- **Módulo venv**: Herramienta estándar de Python para la creación de entornos virtuales.
- **Gestor de paquetes pip**: Utilizado para la instalación automatizada y silenciosa de los paquetes.
- Comandos de procesamiento por lotes de Windows: Uso de directivas de consola como **if not exist, start /b**, **timeout** y **cmd** para la gestión de subprocesos paralelos.

Lógica del Código (iniciar_comparador.bat)

1. **Verificación del entorno**: Al ejecutarse, el script comprueba si la carpeta del **entorno virtual** (.venv) está creada en el directorio raíz. En caso de no estar creada, invoca al intérprete para construir el entorno virtual e instala de forma síncrona todas las librerías del archivo **requirements.txt** empleando **pip**. Si la carpeta existe, no instala nada y activa directamente el entorno virtual de forma silenciosa.
2. **Arranque**: El script utiliza **start /b cmd /c** para dividir la ejecución en dos hilos paralelos. Esto evita que el navegador intente cargar la web antes de que el backend responda, aplicando un retraso de seguridad (**timeout /t**) antes de abrir la URL local.
3. **Inicialización del Servidor ASGI** :Mientras el navegador espera, el hilo principal arranca el servidor Uvicorn **(python -m uvicorn app:app**) para activar la API REST de FastAPI en **127.0.0.1:8000**. Así cuando concluye el tiempo de espera, el backend ya está listo para dar soporte a la interfaz gráfica.

Capítulo 7. Evaluación y resultados

Para garantizar el funcionamiento del asistente de trámites administrativos de la sede electrónica del Ayuntamiento de Jaén, se ha diseñado e implementado un marco integral de validación. Este capítulo detalla el proceso de evaluación técnica, el cual se divide en dos grandes partes: la **Evaluación Científica**, centrada en la calidad del algoritmo del pipeline RAG mediante las métricas de recuperación y generación del framework Ragas, y la **Evaluación del Software**, orientada a verificar el cumplimiento de los requisitos funcionales de la aplicación.

7.1. Justificación y Motivación de la Evaluación

El despliegue de sistemas basados en IA Generatia dentro de un ámbito administrativo legal exige una tasa de fidelidad extrema. Los **Modelos de Lenguaje de Gran Escala (LLM)** tienen una naturaleza estocástica y no determinista, ante una misma entrada de datos (prompt), el modelo puede generar múltiples variantes sintácticas formalmente correctas. Esto inhabilita los métodos clásicos de ingeniería de software, como las pruebas unitarias basadas en la igualdad de cadenas de caracteres, debido a que provocaría falsos negativos ya que evaluaría respuestas semánticamente iguales pero léxicamente distintas.

Asimismo, en tareas de procesamiento de lenguaje natural (PLN) aplicadas a dominios críticos, las métricas tradicionales de superposición de n-gramas como **BLEU** *(Bilingual Evaluation Understudy)* o **ROUGE** *(Recall-Oriented Understudy for Gisting Evaluation)* resultan insuficientes. Estas herramientas evalúan la **coincidencia exacta de palabras clave**, penalizando el uso de sinónimos válidos (por ejemplo, calificar negativamente la equivalencia entre "trámite gratuito" y "coste de cero euros") y, lo más importante, mostrando una incapacidad para detectar alucinaciones en textos con alta coincidencia sintáctica pero donde la información es falsa. Por consiguiente, se justifica la necesidad de implementar una evaluación especializada que sea inmune a las diferencias lingüísticas y evalúe la veracidad legal de las salidas.

7.2. Metodología de Evaluación

Para evaluar con precisión el comportamiento del asistente virtual, la evaluación científica no aborda el sistema como una caja negra, sino que se ejecuta de forma desacoplada, analizando de manera independiente las dos fases principales que forman la arquitectura RAG. Este enfoque metodológico se realiza a través del framework **Ragas**, el cual requiere la estructuración de un conjunto de datos compuesto por cuatro variables esenciales: la consulta del usuario (**question**), el contexto recuperado (**contexts**), la respuesta generada por el modelo (**answer**) y la respuesta de referencia u óptima (**ground_truth**).

A partir de estas variables, el sistema delega en dos arquitecturas específicas que actúan como jueces evaluadores independientes para calcular las cuatro métricas seleccionadas para esta investigación

1. **Evaluación del Recuperador (Retriever)**:Analiza la calidad de la información técnica extraída de la base de datos vectorial FAISS. Para este bloque, el modelo lingüístico utilizado como juez evaluador es **Gemma-3-12B-it**, el cual se encarga de calcular dos métricas cuantitativas::
    - **Precisión del Contexto *(Context Precision)***: Se calcula utilizando la question, los contexts y la ground_truth. El juez Gemma evalúa si el motor de búsqueda es capaz de **priorizar los fragmentos de texto útiles** y

posicionarlos en los primeros lugares del ranking de similitud, **penalizando** la presencia de **ruido** en las primeras posiciones.

- **Recuperación del Contexto *(Context Recall)***: Se calcula utilizando la question, los contexts y la ground_truth. Esta métrica mide la exhaustividad del buscador basándose en la respuesta ideal; el juez Gemma **descompone la respuesta** de referencia en **afirmaciones** individuales y verifica qué porcentaje de dichas afirmaciones se encuentra respaldado y presente dentro de los contextos recuperados.

2. **Evaluación del Generador (Generator)**: Evalúa la calidad semántica y la seguridad de la respuesta final en lenguaje natural redactada por los modelos de lenguaje de gran escala que serán juzgados (***Salamandra, Llama, Apertus***). Para este bloque, el modelo lingüístico utilizado como juez evaluador es **Qwen-3.6-35B**, encargado de calcular dos métricas basadas en lógica lingüística:
   - **Fidelidad *(Faithfulness)***: Se calcula utilizando la question, los contexts y la answer. Actúa como el detector principal de **alucinaciones**; el juez Qwen mide si las **afirmaciones** fácticas emitidas por el asistente virtual **se fundamentan de manera exclusiva en el contexto** inyectado por FAISS, **penalizando cualquier invención de datos ajena a los documentos** proporcionados.
   - **Relevancia de la Respuesta *(Answer Relevancy)***: Se calcula utilizando la question y la answer. Evalúa el nivel de **alineación de la salida con la intención original del ciudadano**; el juez Qwen genera varias **preguntas inversas** a partir de la respuesta dada y calcula la **similitud del coseno** de sus representaciones vectoriales con respecto a la **consulta original**, penalizando respuestas redundantes o superfluas.

7.3. Requisitos e Infraestructura para la Ejecución de la Evaluación

De acuerdo con la metodología desarrollada, la ejecución de las pruebas requiere de dos activos importantes:

- Corpus estructurado de preguntas y respuestas verificadas.
- Un entorno dotado de un modelo lingüístico que actúe como entidad examinadora, uno para el recuperador y otro para el generador.

7.3.1. Construcción del Conjunto de Evaluación

Debido a la ausencia previa de un banco de preguntas validado para la sede electrónica del Ayuntamiento de Jaén, se diseñó e implementó un pipeline automatizado para construir un dataset de evaluación propio. El proceso de construcción del dataset se dividió en dos fases secuenciales: la definición **de perfiles de usuario** y la generación contextualizada de **consultas**.

Fase 1. Ingesta y Vectorización de Perfiles de Usuario

Para dotar al conjunto de datos de evaluación de variabilidad lingüística y simular las intenciones de búsqueda de ciudadanos reales, el primer paso metodológico consistió en establecer una **base de datos diversa de perfiles de usuario**. La función de estos perfiles es determinar el **rol** (System Prompt) que adoptará el Modelo de Lenguaje (LLM) durante la fase de generación, obligándolo a formular preguntas sintéticas **adaptadas a las características de dicho perfil** al recibir un fragmento de texto (chunk) normativo.

Para la obtención de estas identidades, se hizo uso de **PersonaHub** (Chan et al. 2024), una arquitectura de datos que proporciona una colección masiva de perfiles sintéticos. Específicamente, se descargaron 20.000 registros desde su repositorio oficial público (disponible en: https://huggingface.co/datasets/proj-persona/PersonaHub). Estos perfiles se procesaron y almacenaron en un índice vectorial para permitir su posterior emparejamiento semántico con la tipología de cada trámite administrativo del Ayuntamiento de Jaén.

El desarrollo de esta fase se apoyó en los siguientes componentes:

- **Librería Hugging Face Datasets**: Empleada para invocar la función ***load_dataset*** y descargar 20.000 registros del dataset público ***PersonaHub***.
- **Modelo de Embeddings Multilingüe**: Dado que los perfiles extraídos de PersonaHub se encontraban originalmente en inglés, se seleccionó el modelo **paraphrase-multilingual-MiniLM-L12-v2** a través de *HuggingFaceEmbeddings* por su capacidad para proyectar representaciones semánticas cross-lingües.
- **Motor Vectorial FAISS**: Utilizado para la creación del almacén persistente (**Vectorstore**) donde se indexaron los vectores matemáticos de los 20.000 perfiles, habilitando su posterior búsqueda por similitud geométrica.

Fase 2. Generación Sintética y Adaptación Contextual
Durante las pruebas preliminares de generación, se identificó un fallo de distribución: la gran mayoría de los perfiles de PersonaHub resultaban anómalos o **fuera de dominio** para el contexto de una administración local española (por ejemplo: "analista y comentarista de críquet", "productor de música pop japonés" o "biólogo marino").

Para mitigar este error, se inyectó una lista estática complementaria con perfiles más propios de la ciudad de Jaén (ej: “presidente de una cofradía de Jaén”, “mujer trabajadora de mediana edad” o “vendedor ambulante”). A partir de este entorno se desarrolló un script (*dataset.py*) utilizando los siguientes componentes:

- **Framework LangChain**: Encargado de controlar el flujo de datos mediante la concatenación lógica **prompt | llm | StrOutputParser()**, inyectando las reglas de comportamiento y parseando las salidas del modelo hacia texto limpio.
- **API de Inferencia de Groq**[26]: Debido a la indisponibilidad temporal de acceso al clúster de supercomputación ADA durante esta fase del proyecto, se utilizó la **plataforma de inferencia Groq**. Se empleó el modelo **Llama-3.1-8b-instant** para simular la formulación de la pregunta del ciudadano, y el modelo **Llama-3.3-70b-versatile** para redactar la respuesta ideal (Ground Truth).
- **Búsqueda Híbrida Vectorial**: Integración del **índice local FAISS** con los embeddings de HuggingFace para **emparejar** la base de datos de **PersonaHub** y la **lista jiennense**.

Lógica de Ejecución: El script itera sobre los fragmentos documentales extraídos fila a fila de las tablas de trámites del Ayuntamiento. Para cada fragmento, se interroga al índice vectorial FAISS para recuperar el **perfil de ciudadano más relacionado semánticamente con el trámite**. Si el perfil seleccionado proviene de **PersonaHub** (en inglés), una cadena

[26] Groq: https://groq.com/

de **LangChain lo traduce y lo adapta** culturalmente al contexto jiennense. Finalmente, se invoca a la API de Groq bajo instrucciones sistémicas: el modelo ciudadano debe formular una duda orgánica sobre el trámite **evitando muletillas** artificiales (como "según el texto" o "según este trámite"), y el modelo asistente debe **generar la respuesta basada en un 100 % en el fragmento original**, devolviendo un objeto estructurado (json_object).

Mecanismos de Control de Calidad y Resiliencia
Para garantizar la integridad del dataset frente a interrupciones de red o alucinaciones de los LLMs, el script se implementa con una serie de protecciones:

- **Filtro de Evasivas**: **Rechaza** automáticamente **respuestas vacías** o que contengan **fórmulas de denegación** (ej. "información no disponible"), forzando un **máximo de 2 reintentos** de generación.
- **Gestión de Rate Limiting**: Al emplear la capa gratuita de la API de Groq, el sistema captura las excepciones **HTTP 429 (Too Many Requests)**, para su solución se paraliza el hilo de ejecución durante 60 segundos.
- **Persistencia Progresiva**: El sistema realiza un guardado progresivo en disco para proteger los registros ya generados ante posibles fallos.

Estructura del Dataset y Ejemplos
La ejecución exitosa del dataset dio como resultado un archivo JSON consolidado con **843 casos de prueba** estructurados de la siguiente forma:

- **pasaje**: El texto extraído de la fila de la tabla del Ayuntamiento.
- **contexto**: La descripción del perfil del ciudadano adaptado que realiza la pregunta.
- **pregunta**: La pregunta redactada por el modelo adaptado al contexto.
- **respuesta**: La respuesta basada 100 % en el pasaje.
- **origen_perfil**: Etiqueta de control para saber si el perfil del usuario vino de la lista de Jaén o de PersonaHub.

A continuación se presentan 4 casos de estudio extraídos del dataset final:

- Ejemplo 1 (Contexto local)
    - **Pasaje**: Trámite: 10760-R. Renovación de licencia de venta ambulante en mercadillo municipal\nNormativa básica: Ordenanza Municipal…
    - **Contexto**: Vendedor ambulante con licencia de puesto en mercadillo.
    - **Pregunta**: ¿Qué documentos exactamente tengo que presentar para renovar mi licencia de venta ambulante en el mercadillo municipal?.
    - **Respuesta:** Para la renovación de tu licencia de venta ambulante en el mercadillo municipal, debes consultar la Normativa básica…
    - **Origen_perfil:** Lista Jaén.

- Ejemplo 2 (Contexto especializado)
    - **Pasaje**: Trámite: 20009. Informe de Arraigo Social\nNormativa básica: Ley Orgánica 4/2000…
    - **Contexto**: Soy abogada en materia de inmigración en Jaén…
    - **Pregunta**: ¿Cuál es la normativa básica que rige el trámite de Informe de Arraigo Social?.
    - **Respuesta:** Ley Orgánica 4/2000 sobre derechos y libertades de los Extranjeros en España y su Integración Social.
    - **Origen_perfil:** PersonaHub (Adaptado).

- Ejemplo 3 (Contexto profesional)
    - **Pasaje**: Trámite: 10002. Uso o Aprovechamiento de los Espacios Escénicos\nInformación Adicional: Documentación a aportar:…
    - **Contexto**: Un coordinador de eventos para un recinto que alberga diferentes eventos deportivos…
    - **Pregunta**: ¿Qué tipo de documentación específica debo aportar para solicitar el uso o aprovechamiento de los espacios escénicos en el recinto deportivo y de entretenimiento donde trabajo como coordinador de eventos?.
    - **Respuesta:** Para solicitar el uso o aprovechamiento de los espacios escénicos, debes aportar un proyecto o dossier de la actividad o espectáculo a realizar, donde se informe de la actividad, si es benéfica.
    - **Origen_perfil:** PersonaHub (Adaptado).

- Ejemplo 4 (Contexto universitario)
    - **Pasaje:** Trámite: 10032-1. Tarjeta Bonobús para Estudiantes Universitarios\nQuien lo puede Presentar: Estudiantes universitarios de la UJA…
    - **Contexto**: Estudiante de la Universidad de Jaén interesada en unirse al equipo de fútbol femenino y busca información sobre las instalaciones.
    - **Pregunta:** ¿Puedo solicitar la Tarjeta Bonobús para Estudiantes Universitarios si soy estudiante de la Universidad de Jaén?.
    - **Respuesta:** Sí, como estudiante de la Universidad de Jaén, puedes solicitar la Tarjeta Bonobús para Estudiantes Universitarios…
    - **Origen_perfil:** PersonaHub (Adaptado)

7.3.2. Paradigma LLM-as-a-Judge e Infraestructura de Cómputo Distribuido

La evaluación científica automatizada se fundamenta en el paradigma *LLM-as-a-Judge* implementado a través del framework Ragas. Este enfoque metodológico utiliza el empleo de modelos de lenguaje con capacidades cognitivas y volúmenes de parámetros significativamente superiores a los del sistema en pruebas, permitiendo que actúen como árbitros capaces de evaluar de forma asíncrona la lógica conceptual de los textos legales.

Para procesar las 843 consultas y ejecutar los algoritmos de evaluación, el sistema se acopló a una infraestructura distribuida de supercomputación remota, incorporando mecanismos de resiliencia diseñados a medida:

Infraestructuras de Inferencia

La comunicación con ambos clústeres se unifica usando la librería cliente de **OpenAI** en Python.

- **Servidor ADA (Universidad de Jaén)**: Aloja localmente en el nodo 08, bajo la gestión del grupo de investigación SINAI, el modelo **Gemma-3-12B-it** (Gemma Team) (juez del módulo recuperador), la variante especializada objeto de estudio **ALIA-es-legal-administrative-7B-Instruct** (identificada internamente como **Salamandra-7B-It**) y el modelo global de referencia **Llama-3.1-8B-Instruct.**
  Las conexiones se unificaron mediante **peticiones HTTP**, apuntando al **endpoint http://ada01.ujaen.es:8080/v1** bajo la autenticación de una **API Key** alfanumérica

privada.

- **Plataforma Helmholtz Cloud (Alemania)**: Proveedor de infraestructura científica de alto rendimiento que, a través del nodo del servicio Blablador de la Asociación Helmholtz, sirvió como entorno para alojar el modelo juez de generación masiva **Qwen-3.6-35B** (Qwen Team) (cuyos 35.000 millones de parámetros garantizan el análisis de matices lógicos complejos para evaluar la generación de los modelos evaluados) y el modelo europeo **Apertus-8B-Instruct-2509**. La comunicación empleó el protocolo seguro **HTTPS** apuntando al endpoint **https://api.helmholtz-blablador.fz-juelich.de/v1**, autenticado mediante un **Token de Acceso Personal (PAT)**.

Mecanismos de Resiliencia

Se han aplicado los siguientes mecanismos de resiliencia para manejar el dataset:

- **Ajuste Dinámico de Inferencia (Monkey-patching)**: Debido a que el modelo juez ***Qwen-3.6-35B*** requiere de una fase de computación interna orientada al **razonamiento lógico** explicativo (thinking process), **consumía la totalidad de los tokens** de salida antes de emitir la puntuación numérica. Para solucionarlo, se interceptaron en tiempo de ejecución los parámetros de inicialización por defecto del framework Ragas mediante técnicas de **monkey-patching**, forzando el límite de **max_tokens a 8192**.
- **Inferencia Incremental y Persistencia**: El script implementa un **sistema de guardado físico inmediato** en disco tras la resolución de cada consulta individual. Esto dota al motor de tolerancia a fallos, permitiendo **reanudar los procesos** de evaluación masiva ante pérdidas de conexión sin duplicar costes computacionales.
- **Control de Tasa Asíncrono (Rate Limiting)**: Aunque la lógica de red se estructuró con llamadas no bloqueantes mediante la librería asyncio, se programó un **retardo de un segundo** entre peticiones consecutivas enviadas al clúster **Helmholtz**, priorizando la estabilidad del servidor remoto sobre la velocidad bruta de ejecución, ya que debe haber un segundo entre peticiones para no ser **bloqueados** del servidor.
- **Gestión del Túnel de Seguridad SSL:** Durante las pruebas iniciales, los cortafuegos de la red local interrumpían las conexiones salientes hacia Alemania debido a una alerta de tipo **Hostname mismatch** en el carné de identidad digital del certificado del servidor. Para asegurar la continuidad del experimento, se configuró el cliente de red avanzado **httpx** para omitir la validación de la identidad inicial **(verify=False)**, delegando la seguridad de forma íntegra en la clave criptográfica privada de la **API Key**.
- **Política de Reintentos y Recálculo Selectivo**: Ante **microcortes de red o saturación temporal de los clústeres**, el software ejecuta hasta **tres reintento**s espaciados. En caso de persistir la incidencia, la celda de la métrica correspondiente es marcada explícitamente con la etiqueta ***"ERROR"***. Una vez finalizada la evaluación de la generación, un script complementario de postprocesamiento **clona el archivo original** mediante la librería shutil como medida de respaldo, detecta selectivamente las filas con error e invoca al juez de forma aislada para **completar el dataset** de forma eficiente.

7.4. Evaluación del Módulo Recuperador (*Retriever*)

El rendimiento de un sistema RAG se encuentra acotado por la calidad de la información inyectada en su fase inicial, es decir, la información que obtiene y se inyecta al modelo. Cualquier desviación o deficiencia en el recuperador introduce ruido semántico en el prompt, degradando la respuesta final del LLM.

7.4.1. Evaluación Determinista de Modelos de Embeddings y Estrategias de Búsqueda

Antes de calcular las métricas probabilísticas del framework Ragas, se implementó una **evaluación determinista**. El script ejecuta las 843 preguntas del Ground Truth contra el buscador bajo **tres enfoques metodológicos** de recuperación: **búsqueda léxica** mediante el algoritmo estadístico **BM25**, **búsqueda densa/vectorial** mediante **FAISS** y búsqueda combinada de carácter **Híbrido**. Para la aproximación híbrida, se extraen en paralelo los **mejores candidatos** de ambos motores y se **unifican** sus puntuaciones en un ranking consolidado aplicando el algoritmo **Reciprocal Rank Fusion (RRF)**[27] (Cormack et al.), parametrizado bajo la expresión matemática:

$$RRF_Score(d) = \sum_{m \epsilon M} \frac{1}{k_{rrf} + r_m(d)}$$

Donde M es el conjunto de motores de búsqueda, $r_m(d)$ representa la posición del documento d en el ranking del motor m, y k_{rrf} es una constante de suavizado fijada en 60.

Para medir la eficacia, el script emplea el módulo de expresiones regulares **re.search** para verificar de forma exacta si el **fragmento devuelto** por el buscador coincide con el **nombre de la fila** y el **trámite** esperado, evaluando el comportamiento en ventanas de contexto de tres y cinco documentos (Top-3 y Top-5). Los resultados se exponen en la siguiente tabla de control:

Modelo de Embeddings	BM25_ R@3	BM25_ R@5	FAISS_ R@3	FAISS_ R@5	Híbrido_ R@3	Híbrido_ R@5
sentence_similarity_spanish_es	30,84%	39,15%	43,06%	51,48%	40,33%	51,48%
dariolopez/bge-m3-es-legal-tmp-6	30,84%	39,15%	**67,97%**	**75,44%**	52,67%	64,65%
dariolopez/roberta-base-bne-finetuned-msmarco-qa-es-mnrl-mn	30,84%	39,15%	47,09%	55,99%	42,35%	54,09%
wilfredomartel/bge-m3-es-legal-v3	30,84%	39,15%	57,53%	65,48%	50,77%	62,04%

Tabla 7.1: Evaluación Determinista

[27] RRF: https://www.elastic.co/docs/reference/elasticsearch/rest-apis/reciprocal-rank-fusion

El análisis cuantitativo de la Tabla 7.1 revela un **estancamiento** del algoritmo de palabras clave **BM25**, limitado a una tasa de acierto fija del **30,84 %** en Top-3. Este fenómeno se fundamenta en la desconexión léxica entre el lenguaje coloquial del ciudadano y la terminología formal de la administración; si el usuario introduce la palabra "multa" y el registro municipal se titula "Alegaciones a sanciones de tráfico", el emparejamiento por caracteres idénticos omite el pasaje de forma sistemática.

Al utilizar la indexación por proximidad geométrica en **FAISS**, el modelo especializado en el dominio jurídico-administrativo español **bge-m3-es-legal-tmp-6**, basado en la arquitectura multilingüe *BGE-M3* (Chen et al.), sobresale notablemente, alcanzando un **67,97 %** en Recall@3 y un **75,44 %** en Recall@5, superando con holgura a las arquitecturas generalistas o de ajuste macro como RoBERTa.

La evaluación demuestra que la **búsqueda híbrida empeora** el rendimiento neto respecto a la búsqueda densa pura (cayendo del 75,44 % al 64,65 % en Top-5). Esto constata que el motor por palabras clave **BM25** introduce **ruido léxico** que penaliza la precisión del espacio continuo del embedding legal. Por consiguiente, se justifica matemáticamente la selección del modelo **bge-m3-es-legal-tmp-6** operando de manera exclusiva bajo indexación vectorial **FAISS** en una ventana eficiente de **Top-3** documentos.

7.4.2. Evaluación mediante Métricas Ragas

Para evaluar el comportamiento del recuperador bajo el espectro formal del framework **Ragas**, se ejecutó un script *(rec_Ragas.py)*. Este componente carga el índice FAISS en memoria y canaliza las consultas inyectando las salidas al juicio de Gemma-3-12B-it, analizando de forma matemática dos métricas fundamentales de recuperación de información:

- **Precisión del contexto** (*Context Precision*): Métrica que evalúa la capacidad del recuperador para colocar los fragmentos de texto útiles en las **primeras posiciones** del ranking. En el ámbito administrativo, es fundamental que el trámite más relevante se sitúe al principio para evitar que el modelo generador se confunda con información secundaria o ruido de otros fragmentos irrelevantes. Se expresa matemáticamente como la media ponderada del cálculo Precision@k para cada rango indexado:

$$Context\ Precision@K\ = \frac{\sum_{k=1}^{K} (Precision@k \times v_k)}{Nº\ de\ elementos\ relevantes}$$

  Donde v_k actúa como un indicador binario (1 si el fragmento en la posición k es un acierto verídico para la resolución, 0 en caso contrario), y $Precision@k$ se desglosa como el cociente entre los verdaderos positivos y la suma de estos con los falsos positivos acumulados hasta dicho rango:

$$Precision@k\ = \frac{Verdaderos\ Positivos@k}{Verdaderos\ Positivos@k + Falsos\ Positivos@k}$$

- **Recuperación del Contexto** *(Context Recall)*: Mide la exhaustividad semántica del motor, verificando que no se haya omitido ningún **hecho esencial** para fundamentar la respuesta de referencia. Para su evaluación, el juez lingüístico **descompone** la cadena de texto de la **respuesta ideal** (Ground Truth) en un **conjunto de afirmaciones** atómicas individuales, verificando de forma lógica qué porcentaje de ellas se encuentran contenidas o pueden deducirse de los fragmentos textuales inyectados en el contexto:

$$Conext\ Recall\ = \frac{Nº\ de\ afirmaciones\ de\ la\ referencia\ soportadas\ por\ el\ contexto}{Nº\ total\ de\ afirmaciones\ en\ la\ referencia}$$

Resultados

Tras completar el cálculo a sobre la totalidad del conjunto de prueba, se consolidaron los resultados expuestos en la siguiente tabla:

Métrica de Evaluación	Puntuación Media (%)
Context Precision *(Precisión del Contexto)*	**86,56 %**
Context Recall *(Recuperación del Contexto)*	**83,26 %**

Tabla 7.2: Métricas de Recuperación Ragas

Los índices de la Tabla 7.2 demuestran científicamente la alta eficiencia del motor densificado. La nota de **86,56 % en Precisión** del Contexto prueba de forma sólida que el recuperador no solo localiza los datos normativos correctos en más de 8 de cada 10 consultas, sino que es **altamente efectivo ordenando** el fragmento idóneo en la primera línea de la carga útil (payload) del prompt, **minimizando la dispersión** semántica.

7.4.3. Evaluación y Clasificación de Errores en la Recuperación

Dado que el motor de búsqueda vectorial muestra un margen de error residual, se desarrolló el script de diagnóstico avanzado auditoria_gemma.py. Este módulo aísla de forma automatizada aquellas **consultas del dataset** que obtuvieron una calificación de **Context Precision = 0.0**. El script realiza una extracción por **expresiones regulares** de los **metadatos esperados**, replica la búsqueda en caliente de los tres chunks asociados por consulta y genera un archivo de salida JSON. El análisis manual y la clasificación de los 195 fragmentos arrojaron la siguiente distribución cuantitativa:

- **Coincide sólo el Trámite (93 chunks)**: El buscador localiza con éxito el documento administrativo global, pero extrae una fila errónea correspondientes a campos homólogos internos de la tabla, evidenciando los límites de los embeddings ante estructuras tabulares densas.
- **No coinciden ni el Trámite ni el Campo (73 chunks)**: Fallo absoluto de alineación en el espacio geométrico indexado.
- **Coincide sólo el Campo (27 chunks)**: El sistema extrae una sección semánticamente idéntica (por ejemplo, el bloque titulado "Normativa básica") pero

perteneciente a un trámite municipal completamente ajeno a la intención de búsqueda del usuario.
- **Error de formato en la evaluación (2 chunks)**: Casos donde el recuperador localizó con éxito la información correcta, pero sufrió una penalización debido a una alucinación sintáctica en el formato de salida JSON emitido por el modelo juez.

El estudio cualitativo en profundidad de este subconjunto de fallos permitió tipificar y agrupar las deficiencias en cuatro clases de errores:

- **Preguntas de Acceso (15 casos)**: Consultas orientadas a conocer como se entra a la web (ej. ¿Cómo accedo al trámite X?), donde el ciudadano espera la respuesta "Acceso mediante sistema Cl@ve". Sin embargo, la respuesta a dicha consulta se almacena en el campo de **"Nivel de identificación en tramitación online"**, la presencia de la palabra "acceso" forzó a FAISS a recuperar el trámite correcto pero fallando en la fila en el 100% de las muestras. Ejemplo de una consulta:
  - Pregunta_ciudadano: "¿Cómo puedo **acceder** al trámite de Servicio Comida a Domicilio?".
  - Respuesta_esperada_ground_truth: "Puedes acceder mediante sistema Cl@ve".
  - Trámite esperado: "20002. Servicio Comida a Domicilio"
  - Campo esperado: "Nivel de identificación en trámitación online".

- **Preguntas con Ambigüedad Sintáctica (14 casos)**: Preguntas sintácticamente malas que contienen errores de la fase de generación del dataset, incluyendo muletillas como **"Según el texto"** o **"Según este trámite"** sin aportar información del trámite. FAISS falló encontrar el trámite correcto en su totalidad de las muestras, aunque en 7 ocasiones recuperó la tipología del campo correcta. Ejemplo de una consulta:
  - Pregunta_ciudadano: "¿Quién puede presentar **este trámite**, según los requisitos establecidos?".
  - Respuesta_esperada_ground_truth: "Según los requisitos establecidos, solo pueden presentar este trámite los ciudadanos que dispongan de tarjeta de aparcamiento para personas con movilidad reducida.".
  - Trámite esperado: "10705. Licencia Reserva Espacio para Estacionamiento de Vehículos de Personas con Movilidad Reducida".
  - Campo esperado: "Quien lo puede Presentar"

- **Preguntas Fuera del Dominio por Alucinación (6 casos)**: Consultas **fuera del dominio administrativo** inyectadas por fallos de los modelos generadores de Groq (ej. preguntas relativas a "camiones monstruo" o "periodistas de críquet"). El motor de recuperación devolvió los 3 documentos más similares semánticamente, provocando un 100% de error total por no tener relación de dominio las consultas con la base de datos vectorial. Ejemplo de una consulta:
  - Pregunta_ciudadano: "¿Dónde se encuentra Velika Vas y cómo está conectado con la autopista Ljubljana-Zagreb?".
  - Respuesta_esperada_ground_truth: "Velika Vas se encuentra en una ubicación que podría usarse para ilustrar conceptos geográficos,

específicamente su conexión con la autopista Ljubljana-Zagreb.".
    - Trámite esperado: "40018. Campaña municipal de natación escolar".
    - Campo esperado: "Modelos de solicitud".

- **Ambigüedad de Campos Homólogos (25 casos)**: Consultas buenas donde el motor FAISS no fue capaz de encontrar los fragmentos correctos.
  Ejemplo, consulta 703:
    - Pregunta_ciudadano: "¿Cuándo es necesario presentar la solicitud de licencia administrativa para animales potencialmente peligrosos?".
    - Respuesta_esperada_ground_truth: "Todo el año".
    - Trámite esperado: "50003. Licencia Administrativa Para Animales Potencialmente Peligrosos".
    - Campo esperado: "Plazos de Presentación".

Resumen de los Tipos de Errores

Como balance del proceso de auditoría, los errores identificados en las 65 consultas con precisión nula (Context Precision = 0.0) se consolidaron para identificar los principales cuellos de botella del motor de búsqueda indexado. En la Tabla 7.3 se presenta el desglose cuantitativo y cualitativo de cada tipología de error sobre el pipeline RAG.

Error	Casos	Porcentaje	Descripción
Ambigüedad de Campos Homólogos	25	38,46 %	Colapso geométrico en el espacio latente debido a celdas tabulares con texto altamente repetitivo. Los *embeddings* convergen en **coordenadas próximas**, provocando falsos positivos.
Preguntas de Acceso	15	23,08 %	El motor FAISS sobrepone la similitud de **términos léxicos aislados** (ej. “acceso”) frente a la verdadera intención semántica de la consulta.
Preguntas con Ambigüedad Sintáctica	14	21,54 %	Preguntas mal formuladas que **omitan detalles claves del trámite**, impidiendo la proyección de un vector de consulta con suficiente fuerza discriminatoria.
Consultas Fuera de Dominio (Alucinación)	6	9,23 %	Consultas con **ruido inyectadas por error** que fuerzan el emparejamiento por proximidad geométrica (Alucinación en la generación de la pregunta).
Errores de Formato	5	7,69 %	Desviaciones estocásticas del LLM que generó la pregunta añadiendo todo su **procesamiento interno** como pregunta o respuesta.

Tabla 7.3: Resumen Errores Recuperación

7.5. Evaluación del Módulo Generador (Generator)

La etapa generativa se encarga de transformar la documentación recuperada por el motor FAISS e inyectada en el modelo en una respuesta en lenguaje natural estructurada, coherente y libre de alucinaciones para el ciudadano

7.5.1. Modelos Evaluados y Protocolo de Inferencia

El estudio comparativo somete a una evaluación simultánea a tres arquitecturas de lenguaje con diferentes estrategias de entrenamiento:

- **ALIA-es-legal-administrative-7B-Instruct (Salamandra)**: Modelo de **especialización** desarrollado con la colaboración del grupo de investigación SINAI de la Universidad de Jaén. Su arquitectura base parte de Salamandra-7B (Barcelona Supercomputing Center), sobre la cual se ejecutó un **preentrenamiento** continuo especializado para dar lugar a la variante **ALIA-legal-administrative-7B-Base**, finalizando con un entrenamiento de instrucción (Instruction tuning) para consolidar la versión analizada en este proyecto.
- **Llama-3.1-8B-Instruct**: Modelo de **propósito general** desarrollado por Meta (Llama Team), incorporado como la referencia estándar de la industria en código abierto para medir el valor neto que aporta la especialización jurídica de ALIA frente a un sistema global expuesto a volúmenes masivos de datos.
- **Apertus-8B-Instruct**: Arquitectura de código abierto desplegada localmente dentro del nodo de la nube de la Unión Europea en la infraestructura del clúster alemán de Helmholtz.

Para ejecutar la inferencia, se construyó un script *(generación_respuestas.py.)*, en el cual se extrae de forma limpia el Top-3 de fragmentos vectoriales de FAISS y los inyecta en una plantilla de prompt altamente restrictiva que fuerza un **rol experto**, obliga al uso del español peninsular y dicta la **disculpa** explícita e invariable "Lo siento, no dispongo de información suficiente en los documentos oficiales para responder a esta consulta" si la solución no se encuentra explícitamente en los pasajes. Las arquitecturas se parametrizaron bajo una temperatura fría de 0.1 para **atenuar la creatividad estocástica** y se programó una pausa mandatoria de **un segundo** para proteger las cuotas del clúster Helmholtz, generando un JSON estructurado con las respuestas de los tres modelos indexadas por el identificador único de la pregunta. La estructura del archivo JSON es la siguiente:

- **id_pregunta**: Identificador de la pregunta.
- **pregunta**: La pregunta del dataset de la que se genera la respuesta.
- **respuesta_esperada_ground_truth**: La respuesta ideal a la pregunta.
- **documentos_recuperados_faiss**: Array de 3 *chunks* que devolvió FAISS (contexto).
- **respuestas_generadas**: Diccionario con las respuestas generadas por los 3 modelos (Llama-3.1 (UJA), Salamandra (UJA), Apertus (Alemania).

7.5.2. Evaluación mediante Métricas Ragas

El objeto JSON con las respuestas consolidadas se remitió de forma asíncrona al script desarrollado para calculas las métricas de generación de Ragas *(gen_Ragas.py)*, el cual invoca al modelo de alta capacidad **Qwen-3.6-35B** operando como juez generativo mediante la interfaz de AsyncOpenAI. El evaluador analiza las salidas bajo las dos dimensiones troncales de calidad del framework Ragas para la generación de la respuesta:

- **Fidelidad (Faithfulness)**: Actúa como el indicador directo para la detección de **alucinaciones**. El juez descompone la respuesta generada por el modelo en **afirmaciones individuales** y verifica de manera lógica si cada una de ellas puede **demostrarse** de forma matemática y exclusiva utilizando únicamente los **documentos del contexto** inyectado por FAISS. Su cálculo responde a la siguiente expresión matemática:

$$Faithfulness\ Score\ = \frac{Nº\ de\ afirmaciones\ en\ la\ respuesta\ soportadas\ por\ el\ contexto}{Nº\ total\ de\ afirmaciones\ identificadas\ en\ la\ respuesta}$$

- **Relevancia de la Respuesta (Answer Relevancy)**: Mide la capacidad del modelo para ceñirse a las **necesidades del usuario**, penalizando la redundancia y la inyección de datos superfluos. El juez analiza la respuesta generada y redacta de forma inversa N **variantes de preguntas** $N = 3$ que habrían dado lugar a dicha salida. Posteriormente, se calcula el **promedio de la similitud del coseno** entre los embeddings de las **preguntas inversas generadas** (E_{gi}) y el vector de la **consulta original del ciudadano** (E_0) empleando el modelo especializado bge-m3-es-legal-tmp-6:

$$Answer\ Relevancy\ = \frac{1}{N}\sum_{i=1}^{N}\frac{E_{gi}\cdot E_0}{\left|E_{gi}\right|\left|E_0\right|}$$

Resultados
Tras completar la inferencia y aplicar las técnicas de resiliencia y postproesamiento ante errores de formato, el juez *Qwen-3.6-35B* consolidó las métricas medias expuestas en la siguiente tabla de resultados:

Modelo Generador	**Relevancia Media de la Respuesta** *(Answer Relevancy)*	**Fidelidad Media al Contexto** *(Faithfulness)*
Llama-3.1-8B-Instruct	0,6122	**0,6809**
Salamandra-7B-lt (ALIA-legal)	**0,6992**	0,5163
Apertus-8B-Instruct	0,4153	0,4588

Tabla 7.4: Métricas de Generación Ragas

El análisis cruzado de la Tabla 7.3 revela un fenómeno de transferencia de conocimiento de alto interés científico. El modelo especializado **Salamandra-7B-lt** obtiene la calificación más alta en **Relevancia de la Respuesta (0,6992)**, lo que demuestra que sus respuestas están fuertemente alineadas con la **intención de búsqueda y el vocabulario** administrativo del ciudadano. Sin embargo, su índice de **Fidelidad** sufre una degradación notable **(0,5163)**. Esta caída se justifica debido a que la arquitectura de ALIA está profundamente entrenada

en el dominio legal y administrativo español; al procesar el prompt, el modelo tiende a **responder utilizando sus propios pesos sinápticos internos** cargados de conocimiento preexistente, **desobedeciendo** la norma de ceñirse de forma exclusiva al fragmento de FAISS e introduciendo datos correctos pero no presentes en el contexto inyectado (lo que Ragas tipifica matemáticamente como un fallo de fidelidad).

Es crucial destacar que, en las iteraciones y actualizaciones más recientes del modelo, este comportamiento ha evolucionado hacia una estricta **concisión estructural y brevedad de la respuesta**, mitigando el antiguo "sesgo de sobreasistencia" en el que la arquitectura generaba bloques extensos de texto superfluo en un intento de guiar al ciudadano a pesar de no contar con contexto de calidad. En su estado actual, ante un escenario de información insuficiente, Salamandra-7B-It restringe su longitud de salida: o bien ejecuta correctamente la directriz de denegación explícita, o bien computa una respuesta directa, correcta y concisa basada estrictamente en sus pesos paramétricos internos, eliminando la redundancia y optimizando la latencia de generación.

Por el contrario, **Llama-3.1-8B-Instruct** se consolida como el modelo **más fiel al contexto inyectado (0,6809)**, mostrando un **comportamiento disciplinado** ante las instrucciones del prompt y limitando su generación a los datos provistos, aunque su **relevancia** específica decae a **0,6122** debido a su **naturaleza generalista**. Finalmente, el modelo europeo Apertus se posiciona en último lugar en ambas dimensiones, evidenciando dificultades críticas para interpretar y procesar la complejidad léxica de la terminología administrativa española.

7.5.3. Evaluación Manual de Alucinaciones en Contexto Nulo

Para comprobar la resistencia de los modelos y evaluar su comportamiento en escenarios críticos donde el componente recuperador falla por completo, **sin contexto relevante**, se diseñó un entorno de pruebas aislado mediante un script *(generar_respuesta_Prec0.py)* en el cual el código aísla el subconjunto de 65 preguntas del dataset que obtuvieron una nota de **Context Precision = 0.0**, inyectando de forma deliberada los tres fragmentos más similares pero completamente **irrelevantes** (ruido puro), forzando a los modelos a decidir entre acatar la orden de denegación del prompt o generar texto de forma descontrolada. El análisis manual de este experimento dio como resultado las siguientes tasas de comportamiento:

Modelo	Tasa de Éxito	Tasa de Fallo
Llama-3.1-8B-Instruct	**55% (36 preguntas)**	45% (29 preguntas)
Apertus-8B-Instruct	54% (35 preguntas)	46% (30 preguntas)
Salamandra-7B-It	34% (24 preguntas)	**66% (43 preguntas)**

Tabla 7.5: Resultados Generación sin Contexto

En este marco de evaluación, se define como ***"Éxito"*** bajo dos escenarios: el primero, cuando el modelo **acata las reglas** de seguridad y emite la disculpa literal, y el segundo (específico para la validación de este TFG), cuando el modelo **desobedece el prompt** pero **resuelve la consulta** del ciudadano de forma correcta gracias a los datos reales aprendidos en su fase de preentrenamiento continuo (fenómeno denominado "alucinación útil"). Se

cataloga como ***"Fallo"*** cualquier escenario donde el modelo **ignora la restricción** y genera una **respuesta normativa falsa o incoherente**.

El análisis cualitativo detallado de la evaluación se documenta a través de los siguientes tres ejemplos de casos de estudio:

Caso de Estudio 1 (Obediencia frente a Conocimiento interno):

- **Consulta**: ¿Qué pasa si el Ayuntamiento no me contesta a tiempo sobre mi solicitud de aportación de documentación a expedientes?
- **Contexto**: Fragmentos del trámite 12303. Declaración responsable para ejecución de obras (No es el trámite correcto).
- **Respuesta de Llama-3.1**: "Lo siento, no dispongo de información suficiente en los documentos oficiales para responder a esta consulta."
- **Respuesta de ALIA / Salamandra**: "Si el Ayuntamiento no responde a tiempo sobre tu solicitud de aportación de documentación a expedientes, es importante tener en cuenta lo siguiente: 1. Plazo de Resolución... 2. Denegación Tácita: En muchos casos, cuando un procedimiento administrativo no se resuelve en el plazo establecido, se entiende que la solicitud ha sido denegada..."
- **Evaluación**: Llama-3.1 **cumple las restricciones** de seguridad con éxito. Por otro lado, ALIA / Salamandra no sigue el prompt y omite la disculpa, sin embargo, responde en base a su preentrenamiento, generando una respuesta correcta y útil para el ciudadano.

Caso de Estudio 2 (Discrepancia entre el almacenamiento y el ajuste):

- **Consulta**: ¿Cuál es el modelo de autoliquidación para el Impuesto sobre Construcciones, Instalaciones y Obras (ICIO)?
- **Contexto**: Pasajes de la oficina virtual de la Diputación de Jaén relativos a la recaudación del Impuesto de Vehículos (IVTM) (Coincide el campo pero no el trámite).
- **Respuesta de Llama-3.1**: "Lo siento, no dispongo de información suficiente en los documentos oficiales para responder a esta consulta."
- **Respuesta de ALIA / Salamandra**: "El modelo de autoliquidación para el Impuesto sobre Construcciones, Instalaciones y Obras (ICIO) es el Modelo 600. Este modelo se utiliza para la presentación de la autoliquidación del ICIO..."
- **Evaluación**: Ante el contexto nulo, **Llama-3.1 se rige al prompt**. **ALIA / Salamandra no lo sigue** para responder autónomamente basándose en sus pesos sinápticos locales. Responde con gran coherencia sobre el ICIO pero la respuesta difiere de la información que hay en el dataset (Esperaba el Modelo 060, no el 600), demostrando que el conocimiento interno puede diferir de las normativas locales.

Caso de Estudio 3 (Híbrido):

- **Consulta**: ¿Dónde puedo solicitar cita previa para presentar la solicitud de la Tarjeta Bonobús para Menores de 5 a 14 años (Gratuita) en caso de que prefiera la presentación presencial?
- **Contexto**: Requisitos de edad e identificación del Bonobús, omitiendo la URL física de la cita previa.
- **Respuesta de Llama-3.1**: "Lo siento, no dispongo de información suficiente en los documentos oficiales para responder a esta consulta."

- **Respuesta de ALIA / Salamandra**: "Lo siento, no dispongo de información suficiente en los documentos oficiales para responder a esta consulta. Sin embargo, puedo ofrecerte una respuesta general basada en la información proporcionada: [...] Para solicitar una cita previa presencialmente, es recomendable contactar directamente con el Ayuntamiento de Jaén o visitar su página web oficial..."
- **Evaluación**: Hay una mezcla en la respuesta de ALIA / Salamandra, ya que pide disculpa literal, pero añade una sección secundaria basada en su conocimiento para orientar correctamente al ciudadano.

Resumen de los Tipos de Errores

El análisis cualitativo de las respuestas emitidas en un escenario de contexto nulo, $Context\ Precision = 0.0$, permite categorizar el comportamiento de los modelos frente a las restricciones del prompt. Mientras que las arquitecturas generalistas (como Llama-3.1) tienden a un acatamiento estricto de las reglas del prompt, el modelo especializado Salamandra-7B-It presenta un abanico de desviaciones derivadas de su entrenamiento en el dominio jurídico.En la Tabla 7.6 se estructuran y definen las cuatro tipologías de comportamiento observadas, clasificando el tipo de error (o acierto paramétrico).

Tipo de Comportamiento	Descripción	Caso de Referencia
Cumplimiento Estricto	**Acatamiento absoluto de las directrices**. El modelo detecta la ausencia de información útil y emite la disculpa programada	Casos 1, 2 y 3 (Llama)
Alucinación Útil	**Desobediencia de las reglas del prompt**. El modelo activa sus pesos sinápticos y resuelve la consulta usando su conocimiento latente preentrenado de forma **verídica y correcta**.	Caso 1 (Salamandra)
Alucinación Negativa	**Desobediencia del prompt**, donde el conocimiento del modelo colisiona con los trámites locales. El modelo afirma un dato jurídicamente válido a nivel general pero es **falso a nivel municipal**.	Caso 2 (Salamandra)
Híbrida	Comportamiento mixto. El modelo cumple su restricción de seguridad emitiendo la disculpa, pero concatena un bloque de texto adicional basado en su conocimiento interno para guiar al usuario.	Caso 3 (Salamandra)

Tabla 7.6: Resumen tipos de Errores Generación sin Contexto

Balance del análisis cualitativo:

La síntesis expuesta en la Tabla 7.6 confirma la existencia de un claro compromiso en el despliegue de modelos ajustados a un dominio (como ALIA-legal). La elevada concentración de pesos paramétricos vinculados a la terminología jurídica otorga al modelo una fuerte "inercia generativa". Esto provoca que, ante consultas administrativas, el modelo prefiera **priorizar su propio conocimiento latente frente a las instrucciones de contención del sistema** (desviaciones de Alucinación Positiva y Colisión Local). Por consiguiente, queda demostrado científicamente que la integración de un modelo especializado en una

arquitectura RAG municipal requiere un **diseño de prompts mucho más restrictivo** o la implementación de capas de validación intermedias para mitigar su autonomía generativa en contextos de información insuficiente.

7.6. Evaluación del Software y Validación Funcional

La evaluación del software complementa la evaluación científica mediante un análisis de caja negra y la inspección de la interfaz de usuario UI/UX. Su objetivo fundamental es demostrar que la lógica de negocio implementada en el backend cumple con los requisitos funcionales del sistema y alguno no funcional, garantizando una experiencia interactiva fluida, transparente y robusta para el ciudadano.

7.6.1. Verificación de los Requisitos Funcionales en la Interfaz Web

A continuación, se describe detalladamente cómo el diseño de la arquitectura de software y los componentes en producción permiten el cumplimiento de los Requisitos Funcionales en la fase de diseño:

RF-1. Gestión del Historial de Conversación

Este requisito se cumple gracias al sistema de rutas FastAPI y la persistencia local del archivo ***conversation.json***. Al arrancar el sistema la interfaz lanza una petición HTTP GET hacia el endpoint ***/api/conversations***, el cual lee el diccionario indexado en disco y transmite los metadatos de las conversaciones activas (ID, título extraído y marca de tiempo). El frontend procesa estos datos y renderiza el panel de navegación cronológico dinámico. El usuario puede **navegar** entre conversaciones pasadas haciendo clic sobre ellos o **eliminar** de manera permanente el registro del disco duro interactuando con el control de borrado, el cual ejecuta una petición **HTTP DELETE** síncrona que actualiza el estado de la base de datos local.

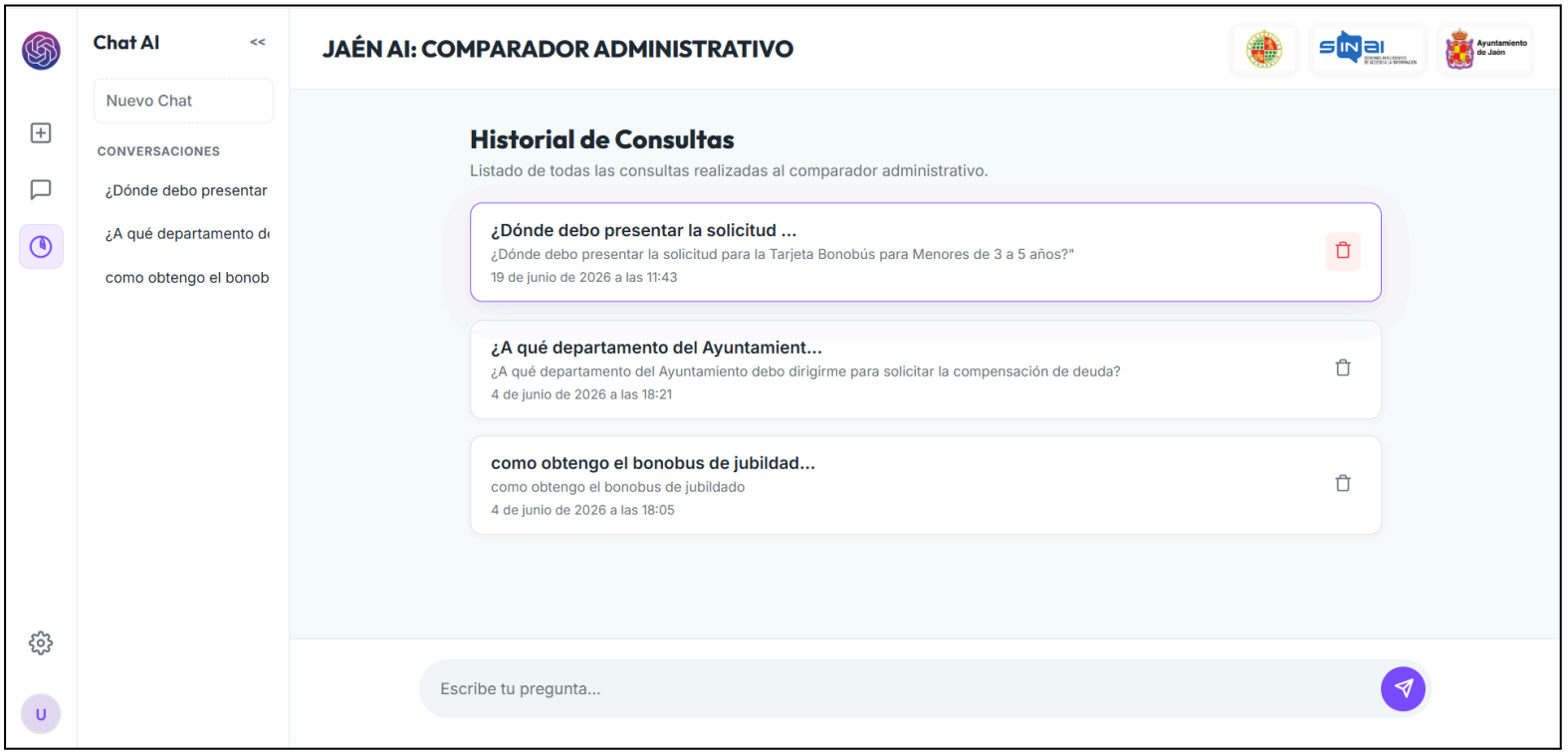


Ilustración 7.1: Interfaz general del sistema gestionando el historial.

RF-2. Consulta con Respuesta Simultánea
El desarrollo de la doble columna interactiva en producción se implementa gracias a la asincronía basada en el bucle de eventos de **AsyncIO** y el protocolo **Server-Sent Events (SSE)**. Cuando el ciudadano introduce una consulta en la caja de texto y ejecuta la petición POST hacia el endpoint ***/api/chat***, el backend duplica el prompt con el contexto inyectado por FAISS e instancia dos tareas asíncronas en paralelo mediante la función ***asyncio.create_task()*** hacia los endpoints remotos del clúster ADA.

Los modelos Llama-3.1-8B-Instruct y ALIA-es-legal-administrative-7B-Instruct (Salamandra) procesan la inferencia simultáneamente, depositando los caracteres generados en una estructura de **cola asíncrona compartida** (asyncio.Queue). Un generador web vacía esta cola de manera continua y transmite las **palabras** al navegador en **tiempo real** mediante un flujo de **streaming** asíncrono, permitiendo al ciudadano observar de manera simultánea cómo se escriben de forma paralela ambas columnas **sin congelar** el renderizado de la interfaz gráfica. Podemos observar en la Ilustración 7.2 la generación de la respuesta a una consulta cuando no se han terminado de generar completamente las respuestas.



Ilustración 7.2: Flujo de respuesta en doble columna.

RF-3. Identificación y Citación del Trámite de Origen
La transparencia y fiabilidad del sistema se consolidan gracias a la inyección de los metadatos desde la recuperación vectorial. Cuando el motor FAISS recupera los 3 fragmentos semánticamente más cercanos utilizando el embedding bge-m3-es-legal-tmp-6, el objeto Document de **LangChain extrae de forma síncrona las claves de metadatos** id, titulo y url que fueron inyectadas originalmente durante la fase de scraping normativo.

Cuando los modelos terminan de generar la respuesta, el backend adjunta los trámites obtenidos. A continuación, el frontend recibe estos datos y despliega enlace hacia la ruta oficial del trámite. Esto permite al usuario comprobar el origen y acceder directamente a la página web del Ayuntamiento de Jaén para comprobar la veracidad de los datos. Podemos observar en la Ilustración 7.3 que el nombre del trámite de la respuesta de Salamandra está subrayado debido a que estaba el cursor seleccionando lo para demostrar que es un enlace.

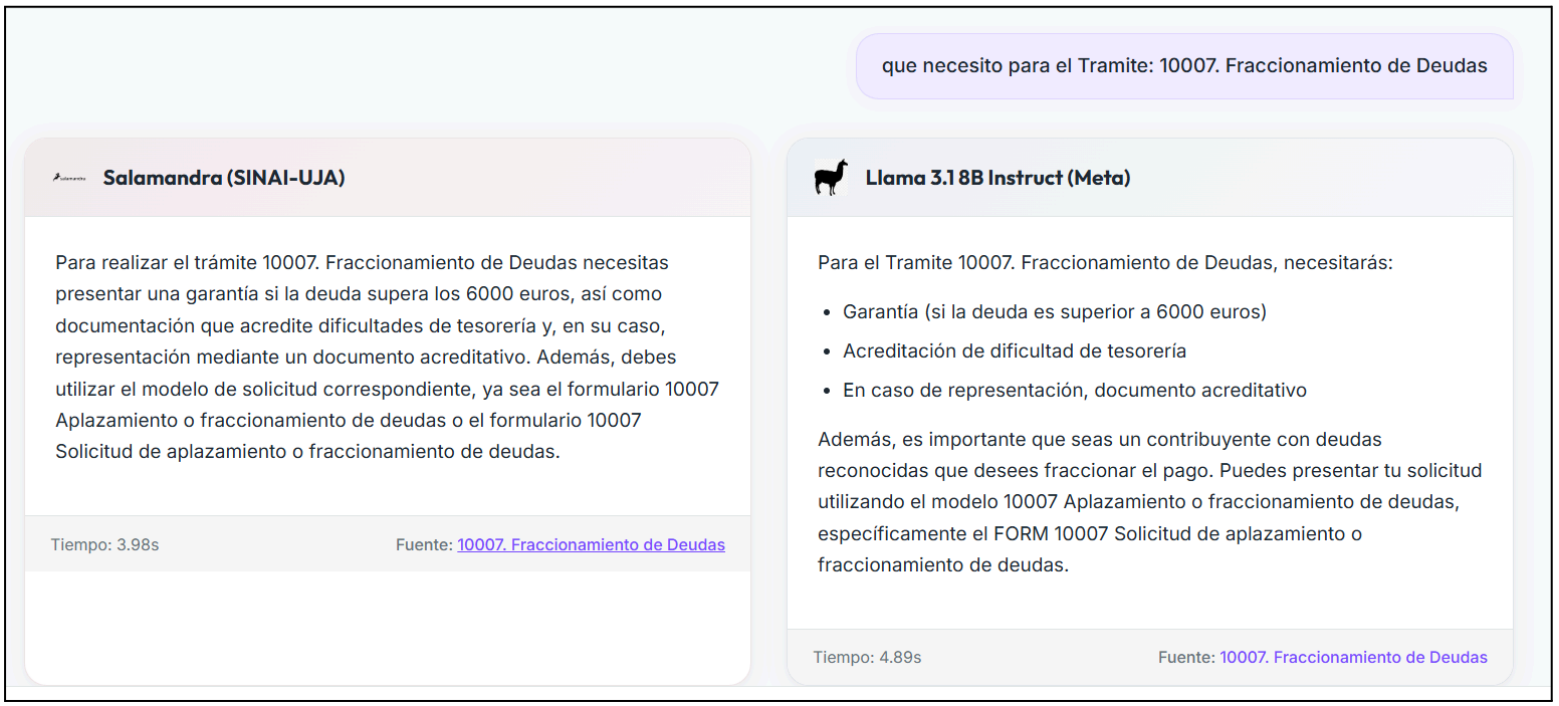


Ilustración 7.3: Detalle de hipervínculos oficiales a la sede electrónica.

RNF-5. Monitorización del Tiempo de Respuesta
Este requisito funcional se ejecuta mediante la inyección del módulo nativo **time** de Python dentro del bucle de control asíncrono de los generadores de tokens. **Al abrirse la conexión de streaming** con cada modelo, el backend captura una marca de tiempo inicial. Una vez que el modelo transmite el **token de cierre o finalización de secuencia** ([FIN]), el sistema **registra la marca** de tiempo de parada

El software calcula la diferencia exacta ($T_{final} - T_{inicial}$) para aislar la latencia computacional de cada IA. Este valor se inyecta en los metadatos transmitidos a la Interfaz y se almacena en el registro de persistencia local. La interfaz de usuario captura este parámetro y lo muestra por pantalla (ej. "Tiempo de respuesta: 2.34s") debajo del pie de cada columna, permitiendo al usuario realizar una comparación entre los tiempos de respuesta de ambos modelos. Podemos comprobar en la Ilustración 7.4 los tiempos de respuesta de ambos modelos, 1.04 segundos para Salamandra / ALIA y 1.12 segundos para Llama.



Ilustración 7.4: Comparativa de tiempos de respuesta.

7.7. Balance General de la Evaluación

La convergencia de la evaluación científica y la verificación funcional del software determina el éxito de la implementación del asistente virtual. Las métricas probabilísticas del framework Ragas han certificado que el pipeline de recuperación y la selección estratégica del embedding legal **bge-m3-es-legal-tmp-6** garantizan un **acceso riguroso y ordenado** a la normativa municipal, blindando el sistema en su primera fase.

Paralelamente, el estudio comparativo de los generadores ha permitido delimitar con precisión la compensación entre la **obediencia estricta al prompt** que ofrece una arquitectura global como Llama-3.1 y la capacidad de **resolución autónoma** fundamentada en el conocimiento interno especializado que aporta Salamandra (ALIA-legal). Tras analizar los resultados, se concluye que **Llama-3.1-8B-Instruct es el modelo que mejor funciona** para este escenario específico, ya que su adherencia estricta a las reglas mitiga el riesgo de desviación. Por tanto, se establece como el motor generador definitivo del sistema.

Finalmente, las pruebas funcionales de caja negra han demostrado que el backend asíncrono responde con resiliencia, gestionando el flujo concurrente mediante arquitecturas de streaming eficientes. En conclusión, el sistema satisface el **100 %** de las necesidades e indicaciones técnicas y funcionales definidas para dotar a la ciudadanía de una herramienta de acceso **transparente** y **robusta** a la información pública local.

Capítulo 8. Conclusión y trabajos futuros

8.1 Conclusiones Generales

La finalización de este Trabajo Fin de Grado permite **validar** la **hipótesis** de partida y confirmar el cumplimiento del **objetivo** general del proyecto: la optimización del acceso y la interoperabilidad de la información de los trámites administrativos del Ayuntamiento de Jaén mediante un asistente virtual inteligente basado en una arquitectura de Generación Aumentada por Recuperación (RAG).

Los resultados obtenidos en la fase de evaluación técnica aportan tres conclusiones fundamentales:

- **Validación de la Hipótesis**: Se ha demostrado matemáticamente que las limitaciones semánticas de los Modelos de Lenguaje de Gran Escala ante el léxico administrativo no se mitigan aumentando el volumen de parámetros del modelo, sino mediante la inyección precisa de contexto. El **motor de recuperación denso FAISS** y el modelo de **embeddings** especializado ***dariolopez/bge-m3-es-legal-tmp-6***, ha alcanzado una **precisión de contexto del 86,56%**, demostrando que la organización estructural y la vectorización jurídica son los verdaderos pilares de la veracidad informativa en la administración pública.
- **Contención de Alucinaciones y Disciplina de Prompt**: La restricción del espacio de respuesta a los fragmentos normativos del Ayuntamiento de Jaén ha actuado como un cortafuegos algorítmico robusto. Sin embargo, la auditoría manual realizada ha revelado un dato importante en la disciplina de seguimiento de instrucciones del prompt entre los modelos evaluados. El modelo **Llama-3.1-8B-Instruct** demostró una **adherencia superior y más estricta a las reglas del prompt** del sistema, sigue fielmente las instrucciones de seguridad y las restricciones negativas ante la ausencia de contexto ineficaz. Por el contrario, **ALIA-es-legal-administrative-7B-Instruct (Salamandra)** tiende a priorizar su conocimiento interno. Debido a su **profundo preentrenamiento en el dominio administrativo español**, tendía a evitar las reglas de contención del prompt para responder de forma autónoma usando sus pesos sinápticos locales, resolviendo la consulta de manera independiente pero asumiendo más riesgo de desviación respecto al contexto necesario.
- **Evolución Dinámica del Modelo y Optimización Pos-evaluación**: Hay que destacar que el comportamiento del modelo Salamandra documentado durante la fase de evaluación de este trabajo estuvo marcado debido a que el modelo resultaba abusar de la verbosidad. Esta tendencia a extenderse en las respuestas favoreció e infló positivamente su puntuación en **Relevancia (0.6992)**, ya que al generar textos más largos y explicativos aumentaba la probabilidad de que el algoritmo de evaluación inversa de Ragas detectara una alta coincidencia semántica con la consulta del usuario. No obstante, en fechas recientes, los compañeros del grupo de investigación SINAI han desplegado una **actualización** y refinamiento sobre el modelo **Salamandra** en el clúster ADA. Las pruebas de **caja negra** realizadas tras este ajuste confirman que el modelo ha corregido dicha tendencia, generando en la actualidad respuestas más **breves**, **concisas** y **directas** al ciudadano, optimizando su comportamiento para el entorno de producción real. Con esto también se ha conseguido que al ser más breve y concisa su generación, Salamandra es más fiel a

seguir el prompt en comparación a su versión anterior.

Este escenario justifica el desarrollo de la interfaz web en **doble columna** implementada en este TFG, la cual ha servido como una excelente herramienta de validación y comparación en tiempo real. Al no depender de un único enfoque, el sistema unifica la **estricta alineación al prompt de Llama** con la **riqueza en el dominio administrativo y la constante evolución de Salamandra**, consolidando una herramienta de transparencia pública robusta, asíncrona y plenamente adaptada a la ciudadanía de Jaén.

Sin embargo, a la luz de los resultados y métricas obtenidas, la evaluación determina de forma clara que el sistema se queda con Llama-3.1-8B-Instruct como el modelo que mejor funciona. Su capacidad para actuar como cortafuegos frente a alucinaciones lo convierte en la opción definitiva para un entorno de producción orientado a la ciudadanía. De este modo, aunque la arquitectura dual quede consolidada como un valioso **banco de pruebas** interactivo para que los investigadores del grupo SINAI evalúen futuras versiones de sus modelos, la recomendación técnica final de este proyecto establece a **Llama** como el motor generador óptimo y más seguro para la administración pública.

8.2 Trabajos Futuros

A partir de las limitaciones identificadas en la fase de evaluación y el comportamiento dinámico de los modelos, se proponen las siguientes líneas de investigación y desarrollo técnico:

8.2.1 Refinamiento de Prompting

Debido a la reciente actualización de Salamandra, se plantea como trabajo futuro el diseñar plantillas de **prompts de diferente longitud** y técnicas **Few-Shot[28] In-Context Learning**. El objetivo es evaluar si **instrucciones más acotadas** y con **ejemplos muy concisos** consiguen optimizar la salida de ambos modelos, forzando a Llama a mantener su disciplina y garantizando que Salamandra consolide de forma definitiva su nuevo patrón de respuestas breves y directas.

8.2.2 Evaluación de Embeddings

Aunque el codificador **bge-m3-es-legal-tmp-6** ha demostrado un rendimiento bastante favorable frente a modelos generalistas, el ecosistema de procesamiento del lenguaje natural evoluciona de manera constante. Concretamente, coincidiendo con las fases finales de redacción de este documento, el **grupo de investigación SINAI** ha publicado sus propios modelos de **embeddings especializados**. Debido a que el banco de pruebas y la evaluación del Retriever ya se encontraban completamente consolidados y cerrados, debido a las restricciones temporales TFG impidieron probar este modelo y volver a analizar el corpus bajo esta nueva arquitectura.

Por consiguiente, se establece como línea de investigación la **sustitución del codificador** actual por los nuevos **vectores densos del grupo SINAI**. Se evaluará su rendimiento comparativo frente al modelo de Darío López, analizando su comportamiento ante variaciones de **tamaño de fragmento** (chunk size) y **solapamiento** (overlap), con el fin de determinar si la especialización del grupo optimiza aún más la recuperación en el dominio normativo de Jaén.

[28] Few-Shot: https://www.ibm.com/es-es/think/topics/few-shot-learning

Asimismo, dado la continua evolución en el campo de la inteligencia artificial, se plantea como trabajo futuro la integración de **nuevos modelos y arquitecturas de embeddings** que se publiquen tras la finalización de este proyecto. De este modo, el sistema se mantendrá **actualizado**, permitiendo evaluar si los futuros lanzamientos superan las métricas obtenidas por las soluciones analizadas en este documento.

8.2.3. Implementación de un Reranker Administrativo

Los resultados del análisis manual determinaron que en 25 casos el motor FAISS sufrió de ambigüedad por campos parecidos, confundiendo filas con semántica muy similar (como "Tasas" y "Plazos"). Para resolver este cuello de botella, se proyecta la implementación de una arquitectura de **recuperación en dos etapas** (Two-Stage Retrieval)[29] mediante la incorporación de un **Reranker especializado** en el dominio legal español, el grupo **SINAI** dispone de uno, el cual se plantea probar en dicha línea de investigación. El motor FAISS realizará una primera extracción de los Top-10 de documentos basados en similitud de vectores densos. Posteriormente, un modelo de clasificación cruzada (Cross-Encoder), con conocimiento de la jerarquía normativa, **recalculará la relación** entre la pregunta y los 10 fragmentos, **reordenando el ranking** para asegurar que el chunk óptimo se posicione en el Top-1 de la carga útil del prompt.

[29] Reranker: https://www.pinecone.io/learn/series/rag/rerankers/

Bibliografía

"Abierto vs Cerrado." *Modelos de IA de Código Abierto vs Código Cerrado*, https://4geeks.com/es/lesson/ias-open-source.

"Catálogo de procedimientos." *Sede electrónica - Ayuntamiento de Jaén*, https://sede.aytojaen.es/sta/CarpetaPublic/doEvent?APP_CODE=STA&PAGE_CODE=CATALOGO.

Chan, Xin, et al. "Scaling Synthetic Data Creation with 1,000,000,000 Personas." *arXiv*, 2024, https://arxiv.org/abs/2406.20094.

Chase, Harrison. "langchain: The agent engineering platform. · GitHub." *GitHub*, 2022, https://github.com/langchain-ai/langchain.

Chen, Jianlv, et al. "BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation." *arxiv*, 2024, https://arxiv.org/abs/2402.03216.

Cormack, Gordon V., et al. "Reciprocal Rank Fusion outperforms Condorcet andIndividual Rank Learning Methods." 2009, https://dl.acm.org/doi/epdf/10.1145/1571941.1572114.

"Cuánto cobra un ingeniero informático en España." *Randstad*, 21 January 2026, https://www.randstad.es/contenidos360/orientacion-laboral/cuanto-cobra-un-ingeniero-informatico-en-espana/. Accessed 22 June 2026.

"Deep Learning o Aprendizaje profundo : ¿qué es?" *Liora*, 19 February 2026, https://liora.io/es/deep-learning-definicion.

Es, Shahul, et al. "RAGAS: Automated Evaluation of Retrieval Augmented Generation." *arXiv preprint*, 2023, p. 8, https://arxiv.org/abs/2309.15217.

Fowler, Martin. *UML distilled*. Addison-Wesley, 2004. Accessed 22 June 2026.

Gao, Yunfan, et al. "Retrieval-Augmented Generation for Large Language Models: A Survey." *arXiv preprint*, 2023, https://arxiv.org/abs/2312.10997.

Gemma Team. "Gemma: Open Models Based on Gemini Research and Technology." *arxiv*, https://arxiv.org/abs/2403.08295.

Institute, Project Management. *Guide to the Project Management Body of Knowledge (PMBOK® Guide)–Sixth Edition*. Project Management Institute, 2017, https://trainupinstitute.com/wp-content/uploads/2022/03/Project-Management-Institute-A-Guide-to-the-Project-Management-Body-of-Knowledge-PMBOK%C2%AE-Guide%E2%80%93Sixth-Edition-Project-Management-Institute-2017.pdf. Accessed 22 June 2026.

Jegou, Hervé, et al. “Faiss: A library for efficient similarity search - Engineering at Meta.” *Engineering at Meta*, 29 March 2017, https://engineering.fb.com/2017/03/29/data-infrastructure/faiss-a-library-for-efficient-similarity-search/.

Ji, Ziwei, et al. “Survey of Hallucination in Natural Language Generation.” *ACM Computing Surveys*, vol. 55, no. 12, 2023, pp. 1-38, https://dl.acm.org/doi/10.1145/3571730.

Johnson, Jeff, et al. “Billion-Scale Similarity Search with GPUs.” *IEEE Transactions on Big Data*, vol. 7, no. 3, 2019, pp. 535-547, https://ieeexplore.ieee.org/document/8733051.

Jurafsky, Daniel, and James H. Martin. *Speech and Language Processing*, 6 January 2026, https://web.stanford.edu/~jurafsky/slp3/.

Lewis, Patrick, et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.” *Advances in Neural Information Processing Systems*, vol. 33, 2020, p. 16, https://proceedings.neurips.cc/paper/2020/file/6b493230205f780e1bc26945df7481e5-Paper.pdf.

Lin, Tianyang, et al. “A Survey of Transformers.” *AI Open*, vol. 3, 2022, pp. 111-132, https://www.sciencedirect.com/science/article/pii/S2666651022000146.

Llama Team. “The Llama 3 Herd of Models.” *arxiv*, 2024, https://arxiv.org/abs/2407.21783.

Manning, Christopher D., et al. *Introduction to information retrieval*. Cambridge University Press, 2008.

McConnell, Steve. *Software Estimation*. Microsoft Press, 2006, https://ptgmedia.pearsoncmg.com/images/9780735605350/samplepages/9780735605350.pdf. Accessed 22 June 2026.

Nielsen, Jakob. *Usability Engineering*. Elsevier Science, 1994. Accessed 22 June 2026.

"Orden PJC/297/2026, por la que se desarrollan las normas legales de cotización a la Seguridad Social." *BOE.es*, 30 March 2026, https://www.boe.es/diario_boe/txt.php?id=BOE-A-2026-7296. Accessed 22 June 2026.

Qwen Team. "Qwen Technical Report." *arxiv*, 2023, https://arxiv.org/abs/2309.16609.

Ramírez, Sebastián. "FastAPI framework, high performance, easy to learn, fast to code, ready for production · GitHub." *GitHub*, 2018, https://github.com/fastapi/fastapi.

"SLM, LLM." *SLM, LLM, RAG y Fine-tuning: Pilares de la IA Generativa Moderna | datos.gob.es*, 10 September 2024, https://datos.gob.es/es/blog/slm-llm-rag-y-fine-tuning-pilares-de-la-ia-generativa-moderna. Accessed 23 June 2026.

Sommerville, Ian. *Software Engineering*. Pearson, 2011. Accessed 22 June 2026.

Vaswani, Ashish, et al. "Attention Is All You Need." *Advances in Neural Information Processing Systems*, vol. 30, 2017, p. 11, https://proceedings.neurips.cc/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.

Wolf, Thomas, et al. "Transformers: State-of-the-Art Natural Language Processing." vol. Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, 2020, https://aclanthology.org/2020.emnlp-demos.6/.

Zheng, Lianmin, et al. "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena." *Advances in Neural Information Processing Systems*, vol. 36, 2023, p. 29, https://proceedings.neurips.cc/paper_files/paper/2023/file/91f18a1287b398d378ef22505bf41832-Paper-Datasets_and_Benchmarks.pdf.